The Ego Empire – CEO Narcissism and Corporate Expansion

Research Overview

M&A Activity is one of the most important research topics in Finance.

Narcissism is a multifaceted personality trait often associated with heightened competitiveness and a desire for dominance.

Narcissism can be understood as the overconfidence of the executive, when they don't make rational M&A decisions and is believed to be one of the drivers for poor M&A decision-making

Empirical research shows that most M&A deals deliver only small returns or negative long-run post-acquisition performance for acquirers. Prior research suggests that such characteristics significantly influence CEOs' strategic decision-making and, consequently, organizational outcomes.

Methodology

This study uses a panel data at the firm–year level to test whether CEOs with higher narcissism—proxied by linguistic self-referencing are associated with more aggressive corporate expansion.

We measures CEO narcissism using the Self-Reference Ratio (SRR) word frequency analysis from executive interview transcripts on Youtube to prove the hypothesis: Does CEO narcissism (measured by linguistic self-referencing ratio in unscripted video interviews as SRR) drive excessive "Empire Building"? Are narcissistic executives more likely to pursue aggressive M&A activities and asset growth, prioritizing size and status over operational efficiency?

$$SRR_{Proportion} = \frac{\text{First-Person Singular}}{\text{First-Person Singular} + \text{First-Person Plural}}$$

Data sources:

Interview dataset: from this course

M&A outcomes: M&A transaction data (Capital IQ - WRDS), aggregated to firm–year.

Financial & controls: Firm fundamentals (Compustat - WRDS), executive controls (ExecuComp - WRDS).

GROUP MEMBERS:

Ngoc Cam Huong Vo 26****
Thi Hong Cam Nguyen 32****

Contents:

---

1. Importing, Data Cleaning & Inspection

In [1]:

```
#import library
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import statsmodels.formula.api as smf
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

1.1 Interview dataset

Tthe course dataset `youtube_data_anonymized.parquet` which contains executive interviews, along with identifiers (e.g., exec_fullname, ticker, first_person_singular, singular_pronoun_ratio, numerosity_total_words).

Process: We start with wrangling data (transform datatype, cleaning and firm-year-executive aggregation). The reason of aggregating is to build a panel at the firm-year-executive level. We also construct normalized SRR scaled by total words so that short interviews with less words do not dominate the measure and prevent mechanical correlation with verbose ceo (ceo speak longer than average)

Output: a firm-year dataset containing SRR metrics of different ceo in different year and firm and tickeror gvkey to merge with other dataset

In [2]:

```
# Load and prepare data

data = pd.read_parquet("youtube_data_anonymized.parquet", engine="pyarrow"  )
pd.set_option('display.max_columns', None)
data
```

Out[2]:

	exec_fullname	coname	coname_2	video_id_anonymized	speaker_id	speaker_similarity	fulltitles	video_title	channel_id_anonymized	video_duration	view_count	video_description
0	Brent J. Beardall, CPA	WAFD INC	Washington Federal Inc.	-5320224913879327568	SPEAKER_00	0.819808	Chief Financial Officer, Executive Vice Presid...	WAFD Bank reports loan originations down 70% i...	1641393960067410226	4:48	2,627 views	Brent Beardall, president and CEO of WAFD bank...
1	Brent J. Beardall, CPA	WAFD INC	Washington Federal Inc.	-4339905836685796780	SPEAKER_02	0.882299	Chief Financial Officer, Executive Vice Presid...	WaFd CEO on regional bank challenges, commerci...	1641393960067410226	5:07	1,774 views	Brent Beardall, WaFd Bank president and CEO, j...
2	Brent J. Beardall, CPA	WAFD INC	Washington Federal Inc.	-7479805287864656696	SPEAKER_01	0.861989	Chief Financial Officer, Executive Vice Presid...	Western U.S. states see net positive migration...	1641393960067410226	5:08	7,123 views	Brent Beardall, president and CEO of Washingto...
3	Brent J. Beardall, CPA	WAFD INC	Washington Federal Inc.	1997219334077199719	SPEAKER_02	0.855025	Chief Financial Officer, Executive Vice Presid...	Regional banks 'actually very strong,' WaFd Ba...	278641420618151631	6:13	1,467 views	New York Community Bancorp (NYCB) will receive...
4	Brent J. Beardall, CPA	WAFD INC	Washington Federal Inc.	2169099132758973428	SPEAKER_02	0.917461	Chief Financial Officer, Executive Vice Presid...	WaFD Bank's Brent Beardall on regional banks a...	1641393960067410226	5:46	1,938 views	Brent Beardall, president and CEO of WAFD, joi...
...	...	...	...	...	...	...	...	...	...	...	...	...
24250	Kevin D. Bailey	VF CORP	VF Corp.	4203938220259004970	SPEAKER_00	0.999137	Executive VP and Group President of Asia-Paci...	Asia is our leading indicator of recovery: VF ...	6078421526510234977	3:07	260 views	VF Corporation's Kevin Bailey explains the str...
24251	Curtis A. Campbell	BLOCK H & R INC	H&R Block Inc.	2910008019541895626	SPEAKER_01	0.999205	Chief Product Officer & President of Global Co...	How Gen Z is shaping the future of personal fi...	278641420618151631	4:52	7,309 views	#genz #yahoofinance #personalfinance \n\nAccor...
24252	Tammy L. Browning	KELLY SERVICES INC -CL A	Kelly Services, Inc.	2318678781407289009	SPEAKER_02	0.999619	Senior VP & President of KellyOCG Senior VP & ...	May jobs report: 'The talent is not fully back...	278641420618151631	6:34	942 views	#Mayjobsreport #jobsreport #employment \nKelly...
24253	Joseph L. Hooley, Esq.	STATE STREET CORP	State Street Corp.	-669633334309017044	SPEAKER_00	0.999716	Chairman, Chief Executive Officer, President, ...	State Street CEO Discusses Long Term Investing...	-363012820510499262	9:17	6,513 views	May.01 -- State Street Chairman and CEO, Jay H...
24254	Deepa Soni	HARTFORD INSURANCE GROUP INC	Hartford Financial Services Group Inc.	6709728959766294434	SPEAKER_01	0.998997	Executive VP and Head of Technology, Data, Ana...	The Hartford's Deepa Soni on customer-centric ...	-6712639624239791107	57:22	1,321 views	

24255 rows × 68 columns

In [3]:

```
def wrangle(file):
    # Read parquet file
    data = pd.read_parquet(file, engine="pyarrow")

    # Choose columns needed for the research
    columns = ["video_id_anonymized",
        "exec_fullname",
        "coname",
        "coname_2",
        "ticker",
        "video_title",
        "video_duration",
        "view_count",
        "publish_date",
        "gvkey",
        "gsector",
        "first_person_singular",
        "first_person_plural",
        "singular_pronoun_ratio",
        "numerosity_numbers",
        "numerosity_total_words",
        "numerosity_ratio",
    ]
    data = data[columns]

    # Change publish_date to datetime type
    data["publish_date"] = pd.to_datetime(data["publish_date"], errors="coerce")
    # Create a column year from publish_date without decimal
    data["publish_year"] = data["publish_date"].dt.year.astype("Int64")

    # Create a column appending gvkey and exec_fullname
    data["gvkey_exec"] = data["gvkey"].astype(str) + "_" + data["exec_fullname"].astype(str)

    # Create column gvkey_year by combining gvkey and publish_year
    data["gvkey_year"] = data["gvkey"].astype(str) + "_" + data["publish_year"].astype(str)

    # Exclude rows where company has less thanninterview and numerosity_total_words < 500
    data = (
        data
        .groupby("gvkey_exec")
        .filter(lambda g: (len(g) >= 3) or (g["numerosity_total_words"].sum() >= 500))
    )

    # Create a column SRR as singular_pronoun_ratio
    data["SRR"] = data["singular_pronoun_ratio"]
    #remove rows where publish_year is null
    data = data[data['publish_year'].notnull()]

    #remove 0 in gvkey so that they can later merge with another table
    data["gvkey"] = data["gvkey"].str.lstrip("0")


    return data

# Load and wrangle data

file="youtube_data_anonymized.parquet"
interview= wrangle(file)


# Aggregate with multi-level columns ( to normalize by total words then )
interview = (
    interview
    .groupby(['gvkey', 'ticker', 'exec_fullname', 'publish_year'])
    .agg({
        'SRR': ['mean', 'median'],
        'numerosity_total_words': 'sum'
    })
    .reset_index()
)

# Flatten and rename columns properly
interview.columns = [
    'gvkey', 'ticker', 'exec_fullname', 'publish_year',
    'SRR_mean', 'SRR_median', 'total_words'  # Correct names
]

# Normalize SRR by total words
interview['normalized_SRR_mean'] = (
    interview['SRR_mean'] / interview['total_words'].replace(0, float('nan'))
)

interview['normalized_SRR_median'] = (
    interview['SRR_median'] / interview['total_words'].replace(0, float('nan'))
)

#extract ticker list for merging with transaction data later
ticker_list = interview['ticker'].unique().tolist()

# Overview of interview data
pd.set_option('display.max_columns', None)
print("\n Interview Overview: \n")
display(interview.head())
print("\n Interview description:\n")
display(interview.describe())
print("\n Interview info:\n")
display(interview.info())
```

```
Interview Overview:
```

	gvkey	ticker	exec_fullname	publish_year	SRR_mean	SRR_median	total_words	normalized_SRR_mean	normalized_SRR_median
0	10016	SWK	Donald Allan, Jr.	2023	0.344519	0.344519	7597	0.000045	0.000045
1	10016	SWK	Donald Allan, Jr.	2024	0.225352	0.225352	943	0.000239	0.000239
2	10016	SWK	James M. Loree	2019	0.117647	0.117647	376	0.000313	0.000313
3	10016	SWK	James M. Loree	2020	0.192308	0.192308	1052	0.000183	0.000183
4	10016	SWK	James M. Loree	2021	0.124046	0.113946	2582	0.000048	0.000044

```
Interview description:
```

	publish_year	SRR_mean	SRR_median	total_words	normalized_SRR_mean	normalized_SRR_median
count	6508.0	6508.000000	6508.000000	6508.000000	6508.000000	6508.000000
mean	2018.50799	0.326100	0.322474	3941.735556	0.000421	0.000421
std	4.357093	0.213247	0.217808	9618.602901	0.001338	0.001339
min	2007.0	0.000000	0.000000	23.000000	0.000000	0.000000
25%	2015.0	0.161554	0.153846	650.750000	0.000060	0.000058
50%	2019.0	0.285570	0.277170	1410.500000	0.000150	0.000148
75%	2022.0	0.450065	0.445157	3806.750000	0.000368	0.000371
max	2025.0	1.000000	1.000000	290660.000000	0.043478	0.043478

```
Interview info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6508 entries, 0 to 6507
Data columns (total 9 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   gvkey                  6508 non-null   object
 1   ticker                 6508 non-null   object
 2   exec_fullname          6508 non-null   object
 3   publish_year           6508 non-null   Int64
 4   SRR_mean               6508 non-null   float64
 5   SRR_median             6508 non-null   float64
 6   total_words            6508 non-null   int64
 7   normalized_SRR_mean    6508 non-null   float64
 8   normalized_SRR_median  6508 non-null   float64
dtypes: Int64(1), float64(4), int64(1), object(3)
memory usage: 464.1+ KB
None
```

In [4]:

```
#display number of distinct tickers
num_tickers = interview['ticker'].nunique()
print(f"Number of distinct tickers in interview: {num_tickers}")
display(num_tickers)
#display number of distinct gvkeys
num_gvkeys = interview['gvkey'].nunique()
print(f"Number of distinct gvkeys : {num_gvkeys}")
display(num_gvkeys)
#display the number of observations in interview data
num_observations = interview.shape[0]
print(f"Number of observations in interview data: {num_observations}")
display(num_observations)
#display the number of distinct executives in interview data
num_executives = interview['exec_fullname'].nunique()
print(f"Number of distinct executives in interview data: {num_executives}")
display(num_executives)
```

```
Number of distinct tickers in interview: 932
932
Number of distinct gvkeys : 921
921
Number of observations in interview data: 6508
6508
Number of distinct executives in interview data: 1835
1835
```

1.2 Compustat dataset

These annual firm fundamentals are used to construct standard corporate finance controls and growth measures at the firm-year level.

Chosen variables for this project:

- Calculated fundamentals: `Tobin's Q`, `Market-to-Book`, `Leverage`, `ROA`, `EBIT`, `Acquisition intensity ratio`
- Constructed based on other fundamentals: `Asset growth`, `Sales growth`, `PPE growth`, `Capex intensity`
- All financial controls are aligned to the same calendar/fiscal year definition used in the firm-year panel, and lagged where needed.

Below is some fundamentals in the dataset:

Column	Meaning (short description)
costat	Company status code (active/inactive, etc.)
curcd	Reporting currency code (e.g., USD, EUR)
gvkey	Compustat firm identifier
datadate	Fiscal period end date
conm	Company name
tic	Ticker symbol
cusip	CUSIP security identifier
fyr	Fiscal year-end month (1–12)
gind	GICS industry group code
gsector	GICS sector code
gsubind	GICS sub-industry code
acqmeth	Acquisition method flag (purchase, pooling, etc.)
fyear	Fiscal year (numeric year)
act	Current assets
at	Total assets
bkvlps	Book value per share
ceq	Common equity
che	Cash and short-term investments
dlc	Debt in current liabilities (short-term debt)
dltt	Long-term debt
ppent	Net property, plant, and equipment
re	Retained earnings
seq	Stockholders' equity (total)
uceq	Common/parent equity (alt. to CEQ when missing)
ebit	Earnings before interest and taxes
epspi	Earnings per share incl. special items
revt	Total revenue
sale	Net sales
xrd	Research and development expense
aqc	Acquisitions of subsidiaries (cash outflow)
capx	Capital expenditures
esubc	Equity in earnings of unconsolidated subsidiaries
fincf	Net cash flow from financing activities
oancf	Net cash flow from operating activities
cshi	Common shares issued
csho	Common shares outstanding
dt	Total debt (often dltt + dlc)
teq	Total equity
mkvalt	Market value of the firm
prcc_f	Fiscal year-end closing price (split-adjusted)

In [5]:

```
# read csv file
com= pd.read_csv("compustat.csv")
com= pd.read_csv("compustat.csv")
com
```

Out[5]:

	costat	curcd	datafmt	indfmt	consol	gvkey	datadate	conm	tic	cusip	fyr	gind	gsector	gsubind	acqmeth	fyear	act	at	bkvlps	ceq	che	dlc
0	A	USD	STD	INDL	C	1045	2005-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2005	6164.0	29495.0	-8.0883	-1478.0	4324.0	1239.0
1	A	USD	STD	INDL	C	1045	2006-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2006	6902.0	29145.0	-2.7270	-606.0	5183.0	1349.0
2	A	USD	STD	INDL	C	1045	2007-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2007	7229.0	28571.0	10.6537	2657.0	4963.0	1049.0
3	A	USD	STD	INDL	C	1045	2008-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2008	5935.0	25175.0	-10.5216	-2935.0	3566.0	1956.0
4	A	USD	STD	INDL	C	1045	2009-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2009	6642.0	25438.0	-10.4893	-3489.0	4859.0	1114.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19511	A	USD	STD	INDL	C	316056	2020-12-31	ALLEGION PLC	ALLE	G0176J109	12	201020	20	20102010	NaN	2020	1145.0	3069.4	9.0930	829.4	480.4	27.8
19512	A	USD	STD	INDL	C	316056	2021-12-31	ALLEGION PLC	ALLE	G0176J109	12	201020	20	20102010	NaN	2021	1117.6	3051.0	8.6050	759.1	397.9	41.7
19513	A	USD	STD	INDL	C	316056	2022-12-31	ALLEGION PLC	ALLE	G0176J109	12	201020	20	20102010	AP	2022	1214.6	3991.2	10.7202	941.8	288.0	44.4
19514	A	USD	STD	INDL	C	316056	2023-12-31	ALLEGION PLC	ALLE	G0176J109	12	201020	20	20102010	AP	2023	1360.9	4311.5	15.0654	1318.3	468.1	446.3
19515	A	USD	STD	INDL	C	316056	2024-12-31	ALLEGION PLC	ALLE	G0176J109	12	201020	20	20102010	NaN	2024	1422.3	4487.8	17.3984	1500.7	503.8	58.6

19516 rows × 43 columns

In [6]:

```
# Create lagged variables

group = com.groupby("gvkey")
com["at_lag"]   = group["at"].shift(1)
com["revt_lag"] = group["revt"].shift(1)
com["revt_lag2"] = group["revt"].shift(2)  # for sales growth (t-1) if needed
com["ppent_lag"] = group["ppent"].shift(1)

#create new columns assets growth, sales growth, ppe growth, capex intensity growth, M&A intensity growth
com['Sales_Growth'] = (com['revt'] - com['revt_lag']) / com['revt_lag']
com['PPE_Growth'] = (com['ppent'] - com['ppent_lag']) / com['ppent_lag']
com['Capex_Intensity_Growth'] = (com['capx'] / com['at'] - (com['capx'].shift(1) / com['at'].shift(1))) / (com['capx'].shift(1) / com['at'].shift(1))
com['MA_Intensity_Growth'] = (com['aqc'] / com['at'] - (com['aqc'].shift(1) / com['at'].shift(1))) / (com['aqc'].shift(1) / com['at'].shift(1))

# add the columns TobinQ, Market_to_Book, Leverage, ROA, EBITDA_Margin, acquisition intensity ratio
#TobinQ and Size
com['TobinQ'] = (com['mkvalt'] ) / com['at']
com["log_at_lag"] = np.log(com["at_lag"])
# Debt and Leverage
com['Leverage'] = (com['dltt']+ com['dlc']) / com['at']
# Return on Assets
com['ROA'] = com['ebit'] / com['at']
#Ebitda Margin
com['EBIT_Margin'] = com['ebit'] / com['revt']
# Acquisition Expenditure ratio
com['Aqc_Ratio'] = com['aqc'] / com['at']
# Cashflow ratio
com["CF"] = com["oancf"] / com["at"]

# add column gvkey_year
com['gvkey_year'] = com['gvkey'].astype(str) + '_' + com['fyear'].astype(str)

#remove rows with missing values in the columns aqc, at, revt, ebit
com = com.dropna(subset=['aqc', 'at','revt','ebit'])

# Overview of compustat data
pd.set_option('display.max_columns', None)
print("\n compustat Overview: \n")
display(com.head())

#display number of distinct gvkeys
num_gvkeys = interview['gvkey'].nunique()
print(f"Number of distinct gvkeys in compustat data: {num_gvkeys}")
display(num_gvkeys)
```

```
compustat Overview:
```

	costat	curcd	datafmt	indfmt	consol	gvkey	datadate	conm	tic	cusip	fyr	gind	gsector	gsubind	acqmeth	fyear	act	at	bkvlps	ceq	che	dlc	dltt
0	A	USD	STD	INDL	C	1045	2005-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2005	6164.0	29495.0	-8.0883	-1478.0	4324.0	1239.0	13456.0
1	A	USD	STD	INDL	C	1045	2006-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2006	6902.0	29145.0	-2.7270	-606.0	5183.0	1349.0	12041.0
2	A	USD	STD	INDL	C	1045	2007-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2007	7229.0	28571.0	10.6537	2657.0	4963.0	1049.0	10093.0
3	A	USD	STD	INDL	C	1045	2008-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2008	5935.0	25175.0	-10.5216	-2935.0	3566.0	1956.0	9001.0
4	A	USD	STD	INDL	C	1045	2009-12-31	AMERICAN AIRLINES GROUP INC	AAL	02376R102	12	203020	20	20302010	NaN	2009	6642.0	25438.0	-10.4893	-3489.0	4859.0	1114.0	10583.0

```
Number of distinct gvkeys in compustat data: 921
921
```

1.3 Executive Data Set

Source: Execucomp (WRDS)

CEO controls include: `Age`, `Gender`, `Tenure (tenure_years)`, `Equity-based compensation`. We calculate `tenure_years` based on `year` and `becameceo` columns

In [7]:

```
var_desc = {
    "coname": "Company Name",
    "cusip": "CUSIP and Issue Number",
    "gvkey": "Company ID Number",
    "spindex": "Industry Group",
    "ticker": "Ticker Symbol",
    "execid": "Executive ID number",
    "exec_fname": "First Name",
    "exec_fullname": "Full Name",
    "exec_lname": "Last Name",
    "exec_mname": "Middle Name",
    "gender": "Gender",
    "becameceo": "Date Became CEO",
    "co_per_rol": "ID number for each executive/company combination",
    "leftofc": "Date Left as CEO",
    "age": "Executive's Age",
    "bonus": "Bonus ($)",
    "option_awards": "Value of Option Awards - FAS 123R ($)",
    "opt_unex_exer_est_val": "Estimated Value of In-the-Money Unexercised Exercisable Options ($)",
    "opt_unex_exer_num": "Unexercised Exercisable Options",
    "salary": "Salary ($)",
    "shrown_excl_opts": "Shares Owned - Options Excluded",
    "shrown_excl_opts_pct": "Percentage of Total Shares Owned - Options Excluded",
    "stock_awards": "Value of Stock Awards - FAS 123R ($)",
    "tdc1": "Total Compensation (Salary + Bonus + Other Annual + Restricted Stock Grants + LTIP Payouts + All Other + Value of Option Grants)",
    "total_alt2_pct": "Total Compensation (options exercised/stock vested) Percent Change Year-to-Year (%)",
    "total_curr": "Total Current Compensation (Salary + Bonus)",
    "year": "Fiscal Year"
}


exu= pd.read_csv("executive.csv")

exu.head()
```

Out[7]:

	gvkey	year	coname	cusip	spindex	ticker	execid	exec_fname	exec_fullname	exec_lname	exec_mname	gender	becameceo	co_per_rol	leftofc	reason	age	bonus	option_awards
0	1045	2005	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	MALE	2003-04-25	14221	2011-11-28	NaN	47.0	0.0	NaN
1	1045	2006	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	MALE	2003-04-25	14221	2011-11-28	NaN	48.0	0.0	851.398
2	1045	2007	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	MALE	2003-04-25	14221	2011-11-28	NaN	48.0	0.0	949.500
3	1045	2008	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	MALE	2003-04-25	14221	2011-11-28	NaN	49.0	0.0	1078.220
4	1045	2009	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	MALE	2003-04-25	14221	2011-11-28	NaN	50.0	0.0	957.580

In [8]:

```
#extract the factor before the comma in exec_fullname
exu['exec_fullname'] = exu['exec_fullname'].str.split(",").str[0]
#tranform gender column to binary 0 and 1
exu['gender'] = exu['gender'].map({'MALE': 0, 'FEMALE': 1})

exu = exu.sort_values(["gvkey", "execid", "year"])

# number of comp years per CEO–firm
exu["n_years_ceo_firm"] = (
    exu.groupby(["gvkey", "execid"])["year"].transform("nunique")
)

exu["crit1"] = exu["n_years_ceo_firm"] >= 2
# 1) Parse to datetime (handles strings)
exu["becameceo_dt"] = pd.to_datetime(exu["becameceo"], errors="coerce")

# 2) Extract year as integer
exu["becameceo_year"] = exu["becameceo_dt"].dt.year

# 3) Tenure in years (first CEO year = 1)
exu["tenure_years"] = exu["year"] - exu["becameceo_year"] + 1
# where tenure_years is NaN
exu_null_tenure = exu[exu["tenure_years"].isna()]
exu["tenure_years"] = pd.to_numeric(exu["tenure_years"], errors="coerce")
exu
```

Out[8]:

	gvkey	year	coname	cusip	spindex	ticker	execid	exec_fname	exec_fullname	exec_lname	exec_mname	gender	becameceo	co_per_rol	leftofc	reason	age	bonus	option_awards
0	1045	2005	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	0	2003-04-25	14221	2011-11-28	NaN	47.0	0.0	NaN
1	1045	2006	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	0	2003-04-25	14221	2011-11-28	NaN	48.0	0.0	851.398
2	1045	2007	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	0	2003-04-25	14221	2011-11-28	NaN	48.0	0.0	949.500
3	1045	2008	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	0	2003-04-25	14221	2011-11-28	NaN	49.0	0.0	1078.220
4	1045	2009	AMERICAN AIRLINES GROUP INC	NaN	2030	AAL	14591	Gerard	Gerard J. Arpey	Arpey	J.	0	2003-04-25	14221	2011-11-28	NaN	50.0	0.0	957.580
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
89411	316056	2024	ALLEGION PLC	NaN	2010	ALLE	63775	David	David S. Ilardi	Ilardi	S.	0	NaN	70216	NaN	NaN	45.0	0.0	218.758
89412	316056	2022	ALLEGION PLC	NaN	2010	ALLE	63776	John	John H. Stone	Stone	H.	0	2022-07-11	70217	NaN	NaN	51.0	1250.0	2000.016
89413	316056	2023	ALLEGION PLC	NaN	2010	ALLE	63776	John	John H. Stone	Stone	H.	0	2022-07-11	70217	NaN	NaN	52.0	0.0	1375.011
89414	316056	2024	ALLEGION PLC	NaN	2010	ALLE	63776	John	John H. Stone	Stone	H.	0	2022-07-11	70217	NaN	NaN	53.0	0.0	1375.035
89415	316056	2023	ALLEGION PLC	NaN	2010	ALLE	65693	Robert	Robert C. Martens	Martens	C.	0	NaN	72477	NaN	NaN	52.0	0.0	137.501

89417 rows × 33 columns

In [9]:

```
#display number of distinct gvkeys
num_gvkeys = exu['gvkey'].nunique()
print(f"Number of distinct gvkeys in execuive data: {num_gvkeys}")
display(num_gvkeys)
#display the number of distinct executives in executive data
num_executives = exu['execid'].nunique()
print(f"Number of distinct executives in executive data: {num_executives}")
display(num_executives)
```

```
Number of distinct gvkeys in execuive data: 966
966
Number of distinct executives in executive data: 16702
16702
```

1.4 M&A Volume to define “Empire Building” Metrics

Source: Capital IQ - M&A transaction data of `WRDS` to construct firm-year acquisition outcomes.

M&A volume is the sum of transaction size across all `completed` deals in that year.

Process:

- This data set don't use gvkey or ticker as key, instead it uses companyid, that's why we need to have another support table to change ticker to companyid
- Create firm identifier (ticker): Apply a scraping-based enrichment step to map tickers for transaction counterparties.
- Currency conversion to USD: transaction size is reported in the local currency of the deal. We convert all deal values to USD using an exchange-rate API. Some exchange currency are not available in api, we compute it manually using the actual currency rate
- Filter deals: keep Merger/Acquisition type transactions and restrict to completed/closed deals.
- Firm-year aggregation: sum transaction values by firm-year to obtain transactionsize_usd.

In [ ]:

```
import wrds
db = wrds.Connection(wrds_username="", wrds_password="")

# only fetch 0 rows but include column metadata
df = db.raw_sql("select * from ciq.wrds_ciqsymbol limit 0;")

# example: ['TSLA','MSFT',...]
ticker_list = ticker_list

# 1) build "'TSLA','MSFT',..." for SQL
tickers_sql = ",".join(f"'{t}'" for t in ticker_list)

# 2) insert into the query
sql = f"""
    select distinct companyid,
           companyname,
           symbolvalue as ticker,
           exchangeid,
           exchangename,
           startdate,
           enddate,
           objectid
    from ciq.wrds_ciqsymbol
    where symbolvalue in ({tickers_sql})
      and symboltypecat = 'ticker'
      and primaryflag_trd = 1;
"""

tickers = db.raw_sql(sql)
```

```
Loading library list...
Done
```

In [11]:

```
# tranform the column compnayid to exclude decimal
tickers['companyid'] = tickers['companyid'].astype(int).astype(str)
#choose distinct companyid and companyname, symbolvalue as ticker from tickers
tickers = tickers[['companyid', 'companyname', 'ticker']].drop_duplicates()
#get column companyid from tickers with no decimals
companyids = tickers['companyid'].astype(int).drop_duplicates().tolist()
# extract columns companyid, companyname, ticker to csv
tickers.to_csv("tickers.csv", index=False)
```

In [12]:

```
# Transaction of Global and North America deals

transaction= pd.read_csv('transactionma.csv')
transaction2=pd.read_csv('transactionma_addin.csv')
#append transaction2 to transaction
transaction = pd.concat([transaction, transaction2], ignore_index=True)
#chnage name column relcompanyid to companyid
transaction = transaction.rename(columns={'relcompanyid': 'companyid'})
#change companyid to integer
transaction['companyid'] = transaction['companyid'].astype(int)
transaction["companyid"] = transaction["companyid"].astype("string")
tickers["companyid"] = tickers["companyid"].astype("string")
# Convert companyid,tickers to string
transaction["companyid"] = transaction["companyid"].astype("string")
tickers["companyid"] = tickers["companyid"].astype("string")
# Merge Transaction data again with tickee to get ticker column
transaction_final = pd.merge(
    transaction,
    tickers,
    on="companyid",
    how="left",
)
# Keep rows where transactiontype is Merger/Acquisition
transaction_final = transaction_final[transaction_final['transactiontype'] == 'Merger/Acquisition']
# keep just closed transactions
transaction_final = transaction_final[transaction_final['lateststatus'] == 'Closed']
# save transaction_final to csv
transaction_final.to_csv('transaction_final.csv', index=False)
#remove NaN values in the column transactionsize
transaction_final = transaction_final.dropna(subset=['transactionsize'])
```

In [13]:

```
# Set the maximum width of a column to 50 characters
pd.set_option('display.max_colwidth', 50)

# Now view your data
transaction_final.head()
```

Out[13]:

	companyid	announceddate	transactionid	transactiontype	tgtcompanyid	tgtcompanyname	tgtcompanytype	transtitle	relcompanyname	relcompanystatus	compreltype	lateststatus
2	18671	2006-09-20	29226721	Merger/Acquisition	23271637	DSM Pharmaceutical Products, Inc., Pharma Chem...	Assets/Products	NaN	Albemarle Corporation	Operating	Merged Entity	Closed
5	18671	2008-05-05	43503823	Merger/Acquisition	3181768	Sorbent Technologies Corporation	Private Company	NaN	Albemarle Corporation	Operating	Merged Entity	Closed
6	18671	2008-07-14	47261409	Merger/Acquisition	47261400	Remaining 25% stake in Ningbo Jinhai and 25% s...	Assets/Products	NaN	Albemarle Corporation	Operating	Merged Entity	Closed
7	18671	2010-08-02	110184287	Merger/Acquisition	998289	Albemarle UK Ltd.	Private Company	NaN	Albemarle Corporation	Operating	Prior Subsidiary/Operating Unit	Closed
10	18671	2013-10-08	247902595	Merger/Acquisition	247902562	Cambridge Chemical Company Ltd	Private Company	NaN	Albemarle Corporation	Operating	Prior Subsidiary/Operating Unit	Closed

In [14]:

```
# Because the transactionsize is in local currency of target company each company, we will converse all local currency to usd
curr_list=transaction_final["localcurrency"].dropna().unique().tolist()

import requests


url = "https://api.frankfurter.dev/v1/2000-01-01..2025-12-31"
params = {
    "base": "USD",
    "to": ",".join(curr_list)   # ask only for these currencies
}

r = requests.get(url, params=params)
data = r.json()["rates"]

fx = (
    pd.DataFrame(data)
      .T.reset_index()
      .rename(columns={"index": "date"})
)

fx["date"] = pd.to_datetime(fx["date"])
fx["USD"] = 1.0
```

```
available = set(fx.columns) - {"date"}
requested = set(curr_list)
missing = requested - available
print("Missing currencies:", sorted(missing))
#there are just around 34 currency in this api, we are still missing a lot of currency
```

```
Missing currencies: ['AED', 'ARS', 'BDT', 'BHD', 'BWP', 'CLP', 'DEM', 'EGP', 'GNF', 'JMD', 'KES', 'KWD', 'LKR', 'MAD', 'NAD', 'NGN', 'OMR', 'PEN', 'PKR', 'PYG', 'RSD', 'SAR', 'TN
D', 'TTD', 'TWD', 'UAH', 'VND', 'XOF', 'ZWD']
```

In [15]:

```
#Those exchange rate from missing currency will be manually collected on the internet on the date 23.12.2025
rates = {
    "AED": 3.67218,
    "ARS": 1450.28,
    "BDT": 121.085,
    "BHD": 0.37385,
    "BWP": 13.2637,
    "CLP": 909.613,
    "DEM": 1.66599,
    "EGP": 47.4849,
    "GNF": 8671.22,
    "JMD": 158.606,
    "KES": 128.214,
    "KWD": 0.306608,
    "LKR": 307.937,
    "MAD": 9.06353,
    "NAD": 16.7138,
    "NGN": 1458.67,
    "OMR": 0.38363,
    "PEN": 3.34372,
    "PKR": 279.753,
    "PYG": 6670.66,
    "RSD": 99.7486,
    "SAR": 3.74849,
    "TND": 2.90038,
    "TTD": 6.69745,
    "TWD": 31.5063,
    "UAH": 41.9477,
    "VND": 26308.3,
    "XOF": 558.748,
    "ZWD": 373.3,
}

# add those to column fx
for ccy, rate in rates.items():
    fx[ccy] = rate

#unpivot the column
fx_long = fx.melt(id_vars="date", var_name="localcurrency", value_name="per_usd")

#merge fx_long to transaction_final on announceddate and localcurrency to get the exchange rate
transaction_final["announceddate"] = pd.to_datetime(transaction_final["announceddate"])
transaction_final = transaction_final.merge(
    fx_long,
    left_on=["announceddate", "localcurrency"],
    right_on=["date", "localcurrency"],
    how="left"
)
```

In [16]:

```
fx.head(10)
```

Out[16]:

	date	AUD	CAD	CHF	CYP	CZK	DKK	EUR	GBP	HKD	HUF	ISK	JPY	KRW	LVL	NOK	NZD	PLN	SEK	SGD	ZAR	BGN	TRY	CNY	HRK
0	1999-12-30	1.5351	1.4541	1.5978	0.57403	35.938	7.4092	0.99542	0.61885	7.7676	253.53	72.497	102.26	1132.07	0.58541	8.0395	1.9268	4.1397	8.5233	1.6641	6.1587	NaN	NaN	NaN	NaN
1	2000-01-03	1.5209	1.4447	1.5900	0.57156	35.741	7.3740	0.99108	0.61903	7.7923	252.26	72.379	101.83	1129.85	0.58632	7.9901	1.9159	4.1462	8.4757	1.6619	6.1460	NaN	NaN	NaN	NaN
2	2000-01-04	1.5213	1.4494	1.5578	0.56041	35.197	7.2226	0.97040	0.61097	7.7827	246.99	71.732	102.75	1123.07	0.58282	7.9088	1.9161	4.1167	8.3663	1.6542	6.1220	NaN	NaN	NaN	NaN
3	2000-01-05	1.5213	1.4530	1.5490	0.55729	35.047	7.1802	0.96451	0.60995	7.7767	245.48	71.663	103.53	1134.34	0.58218	7.9147	1.9248	4.1114	8.3348	1.6550	6.0725	NaN	NaN	NaN	NaN
4	2000-01-06	1.5237	1.4527	1.5468	0.55651	34.889	7.1661	0.96265	0.60666	7.7823	244.95	71.958	104.66	1147.38	0.58337	7.8966	1.9315	4.1002	8.3216	1.6645	6.0940	NaN	NaN	NaN	NaN
5	2000-01-07	1.5303	1.4595	1.5635	0.56162	35.032	7.2380	0.97238	0.60891	7.7820	247.60	72.141	105.11	1137.22	0.58557	7.9682	1.9391	4.0740	8.4063	1.6624	6.0670	NaN	NaN	NaN	NaN
6	2000-01-10	1.5238	1.4540	1.5729	0.56447	35.182	7.2781	0.97761	0.61091	7.7692	248.99	72.343	104.86	1131.92	0.58520	8.0066	1.9340	4.0636	8.4632	1.6595	6.0830	NaN	NaN	NaN	NaN
7	2000-01-11	1.5273	1.4590	1.5691	0.56318	35.071	7.2586	0.97504	0.60950	7.7793	248.36	72.358	106.13	1146.17	0.58649	8.0026	1.9370	4.0725	8.4458	1.6707	6.0750	NaN	NaN	NaN	NaN
8	2000-01-12	1.5185	1.4568	1.5617	0.56041	34.913	7.2228	0.97012	0.60662	7.7734	247.17	71.692	105.84	1143.29	0.58508	7.9705	1.9288	4.0587	8.4134	1.6672	6.0784	NaN	NaN	NaN	NaN
9	2000-01-13	1.5035	1.4497	1.5678	0.56209	35.178	7.2450	0.97314	0.60870	7.7828	247.95	71.837	105.97	1135.67	0.58603	7.9598	1.9117	4.0631	8.4152	1.6679	6.0630	NaN	NaN	NaN	NaN

In [17]:

```
fx_long.head(10)
```

Out[17]:

	date	localcurrency	per_usd
0	1999-12-30	AUD	1.5351
1	2000-01-03	AUD	1.5209
2	2000-01-04	AUD	1.5213
3	2000-01-05	AUD	1.5213
4	2000-01-06	AUD	1.5237
5	2000-01-07	AUD	1.5303
6	2000-01-10	AUD	1.5238
7	2000-01-11	AUD	1.5273
8	2000-01-12	AUD	1.5185
9	2000-01-13	AUD	1.5035

In [18]:

```
# calculate transaction size in usd
transaction_final["transactionsize_usd"] = (transaction_final["transactionsize"] / transaction_final["per_usd"])
# drop rows with NaN in transactionsize_usd
transaction_final = transaction_final.dropna(subset=['transactionsize_usd'])
# Group all transactions by firm and year
#new column year from announceddate
transaction_final['fyear'] = pd.to_datetime(transaction_final['announceddate'], errors='coerce').dt.year
# ensure fiscal year is integer
transaction_final["fyear"] = transaction_final["fyear"].astype(int)

# group company by ticker and fyear and take the sum of transactionsize_usd and transactionsize and transactioncount
transaction_grouped = (
    transaction_final
    .groupby(['ticker', 'fyear','relcompanyname'], as_index=False)
    .agg({
        'transactionsize_usd': 'sum',
        'transactionsize': 'sum'
    })
)
transaction_grouped
```

Out[18]:

	ticker	fyear	relcompanyname	transactionsize_usd	transactionsize
0	A	2005	Agilent Technologies, Inc.	3617.000000	3617.0
1	A	2005	Resolute FP Canada Inc.	982.000000	982.0
2	A	2006	Agilent Technologies, Inc.	176.000000	176.0
3	A	2006	AstraZeneca AB	35.271876	250.0
4	A	2006	Labeyrie Fine Foods PLC	21.576342	17.0
...	...	...	...	...	...
12921	ZTS	2016	Zoetis Inc.	122.000000	122.0
12922	ZTS	2017	Zoetis Inc.	111.498972	145.0
12923	ZTS	2018	Zoetis Inc.	1995.000000	1995.0
12924	ZTS	2022	Zoetis Inc.	50.000000	50.0
12925	ZTS	2023	Zoetis Inc.	384.444797	329.0

12926 rows × 5 columns

In [19]:

```
#display number of unique tickers
print("Number of unique tickers in compustat data:", transaction_grouped['ticker'].nunique())
```

```
Number of unique tickers in compustat data: 907
```

1.5 Merge all datasets

Merge all components into the final firm-year panel keyed by `gvkey`, `year`, `executive`

- Merge interview with financial fundamentals by `gvkey` and `year`
- Merge interview, financial fundamentals with executive controls by `gvkey`, `year` and `executive`

Result : a firm-year dataset combining (i) narcissism proxies (SRR), (ii) M&A "empire building" outcomes, (iii) firm financial controls, and (iv) CEO characteristics for regression analysis.

In [20]:

```
# Make sure both keys have same dtype
interview["gvkey"] = interview["gvkey"].astype(str)
com["gvkey"] = com["gvkey"].astype(str)
exu["gvkey"] = exu["gvkey"].astype(str)
# Ensure year is numeric on both sides
interview["publish_year"] = pd.to_numeric(
    interview["publish_year"], errors="coerce"
)
com["fyear"] = pd.to_numeric(com["fyear"], errors="coerce")

# Now merge with compustat data
merged_interview_com = pd.merge(
    interview,
    com,
    left_on=["gvkey", "publish_year"],
    right_on=["gvkey", "fyear"],
    how="left",
)
#continue to merge with executive data
merged_interview_com_exu = pd.merge(
    merged_interview_com,
    exu,
    left_on=["gvkey", "publish_year", "exec_fullname"],
    right_on=["gvkey", "year", "exec_fullname"],
    how="left",
)
#drop rows with NaN in tenure_years and tobinQ
merged_interview_com_exu = merged_interview_com_exu.dropna(subset=['TobinQ'])
#change column name ticker_x to ticker
merged_interview_com_exu = merged_interview_com_exu.rename(columns={'ticker_x': 'ticker'})
#left join with transaction data
final_merged = pd.merge(
    merged_interview_com_exu,
    transaction_grouped,
    left_on=["ticker", "publish_year"],
    right_on=["ticker", "fyear"],
    how="left",
)
#remove rows where TobinQ is NaN and transactionsize_usd is NaN
final_merged = final_merged.dropna(subset=['TobinQ', 'transactionsize_usd'])
#remove rows where TobinQ is NaN and transactionsize_usd is NaN
final_merged = final_merged.dropna(subset=['TobinQ', 'transactionsize_usd'])
final_merged = final_merged.dropna(subset=["execid"])
final_merged
```

```
/var/folders/6h/8882zf396gl7nvkkt7wy23dh0000gn/T/ipykernel_29549/3734429452.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  com["gvkey"] = com["gvkey"].astype(str)
/var/folders/6h/8882zf396gl7nvkkt7wy23dh0000gn/T/ipykernel_29549/3734429452.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  com["fyear"] = pd.to_numeric(com["fyear"], errors="coerce")
```

Out[20]:

	gvkey	ticker	exec_fullname	publish_year	SRR_mean	SRR_median	total_words	normalized_SRR_mean	normalized_SRR_median	costat	curcd	datafmt	indfmt	consol	datadate	
1	10016	SWK	James M. Loree	2019	0.117647	0.117647	376	0.000313	0.000313	A	USD	STD	INDL	C	2019-12-31	STANLEY B & DECKE
2	10016	SWK	James M. Loree	2020	0.192308	0.192308	1052	0.000183	0.000183	A	USD	STD	INDL	C	2020-12-31	STANLEY B & DECKE
3	10016	SWK	James M. Loree	2020	0.192308	0.192308	1052	0.000183	0.000183	A	USD	STD	INDL	C	2020-12-31	STANLEY B & DECKE
4	10016	SWK	James M. Loree	2021	0.124046	0.113946	2582	0.000048	0.000044	A	USD	STD	INDL	C	2021-12-31	STANLEY B & DECKE
5	10016	SWK	James M. Loree	2021	0.124046	0.113946	2582	0.000048	0.000044	A	USD	STD	INDL	C	2021-12-31	STANLEY B & DECKE
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
5828	9899	T	Randall L. Stephenson	2018	0.435752	0.350340	6624	0.000066	0.000053	A	USD	STD	INDL	C	2018-12-31	AT&
5829	9899	T	Randall L. Stephenson	2019	0.266093	0.242857	6182	0.000043	0.000039	A	USD	STD	INDL	C	2019-12-31	AT&
5830	9899	T	Randall L. Stephenson	2019	0.266093	0.242857	6182	0.000043	0.000039	A	USD	STD	INDL	C	2019-12-31	AT&
5831	9899	T	Randall L. Stephenson	2020	0.368559	0.359886	3515	0.000105	0.000102	A	USD	STD	INDL	C	2020-12-31	AT&
5834	9904	SWN	Steven L. Mueller	2012	0.115942	0.115942	964	0.000120	0.000120	I	USD	STD	INDL	C	2012-12-31	SOUTHWES ENERG

2567 rows × 102 columns

In [21]:

```
#number of unique gvkeys in final_merged
num_gvkeys_final = final_merged['gvkey'].nunique()
print(f"Number of unique gvkeys in final merged data: {num_gvkeys_final}")
display(num_gvkeys_final)
#number of unique executive in final_merged
num_executives_final = final_merged['execid'].nunique()
print(f"Number of unique executives in final merged data: {num_executives_final}")
final_merged.to_csv('final_merged.csv', index=False)
display(num_executives_final)
```

```
Number of unique gvkeys in final merged data: 539
539
Number of unique executives in final merged data: 849
849
```

Tenure year is Tenure year being CEO, the executive can join company and become vice president for a long time and then became CEO, for example Dennis P. Vermillion joined Avista in 1985 and has held a series of staff and management roles (including president of Avista Energy from 2001–2007, senior vice president, president of Avista Utilities, and president of Avista Corp) before becoming CEO in 2019. So negative tenure years make sense

In [22]:

```
final_merged[["transactionsize_usd","revt_lag","CF","bonus","option_awards","stock_awards"]].describe()
```

Out[22]:

	transactionsize_usd	revt_lag	CF	bonus	option_awards	stock_awards
count	2567.000000	2564.000000	2567.000000	2567.000000	2567.000000	2567.000000
mean	2751.656857	34993.829395	0.101530	431.997590	2574.869905	7741.525548
std	14670.505491	63422.949882	0.076860	1511.916572	11649.249853	15352.946273
min	0.000000	10.029000	-0.331062	0.000000	0.000000	0.000000
25%	22.862110	3889.000000	0.056567	0.000000	0.000000	2397.715000
50%	188.136399	10701.000000	0.093801	0.000000	0.000000	5280.000000
75%	1125.000000	36216.000000	0.137867	0.000000	2195.400000	9559.842000
max	389694.211213	645737.000000	0.574269	19350.000000	245953.429000	394592.295000

There is a massive skewness in trasaction value and revenue. We will use log later to normalize the data

---

2. Build a completed panel data and Exploratory Data Analysis (EDA)

To mitigate omitted-variable bias and isolate the association between CEOs' self-referential language (SRR) and subsequent acquisition outcomes, we include a standard set of firm- and CEO-level controls that capture investment opportunities, financial capacity and constraints, managerial discretion, and incentive alignment.

Controls	Variables
Investment opportunities	Tobin's Q
Financial capacity	Size (log assets), Revenue, Leverage
Performance	EBIT Margin
Growth	Sales Growth
CEO characteristics	Tenure, Age, Gender, Equity compensation measures

We control for `Tobin’s Q` used for firms' growth prospects and the marginal value of investment opportunities. Firms with higher Tobin's Q are more likely to expand and undertake acquisitions for value-enhancing reasons.

We control for `lagged revenue (revt_lag)` as a measure of firm scale, since larger firms mechanically engage in larger transactions and may also differ in their media visibility and communication context. `Leverage` is added to account for financing constraints and capital structure effects, as highly levered firms may face tighter borrowing capacity and different acquisition strategies.

We control for `acquisition intensity (Aqc_Ratio)` to capture persistent firm-level tendencies toward acquisitive growth. This helps distinguish SRR-related variation from stable strategic differences in corporate expansion.

`age` is included to account for systematic differences in risk preferences, career stage, and leadership style that may affect both language use and acquisition behavior. CEO `tenure (tenure_years)` proxies for managerial influence and discretion: longer-tenured CEOs often have greater latitude to pursue strategic initiatives, including acquisitions, and tenure may also correlate with communication style. We include `gender` to account for demographic differences in communication patterns and potential differences in board scrutiny or stakeholder responses that may affect strategic decision-making.

We include `CEO share ownership excluding options (shrown_excl_opts_pct)` to capture alignment with shareholders ("skin in the game"). Higher ownership can constrain empire-building incentives or, alternatively, reflect control and bargaining power; in either case it is a key determinant of acquisition behavior. We also control for equity-based compensation and option-related wealth incentives through `log option awards (log_option_awards)`, `log stock awards (log_stock_awards)`. These variables proxy for risk-taking incentives and the CEO's wealth exposure to stock-price movements, both of which can influence the propensity to pursue acquisitions and the scale of transactions.

We observed that transaction sizes, revenues, CEO bonus , stock awards, stock awards are right-skewed (many small values, few huge values). To reduce skewness we transformed them to log and visualize them before regression.

To create an unified dataset for our CEO narcissism–M&A analysis, we merged multiple data sources into a single firm-year panel. During this integration, we observed that some firm-year observations were incomplete—for example, certain years contained interview-based SRR measures but lacked matched M&A transaction values, or had missing Compustat/CEO control variables. These inconsistencies can lead to different sample sizes across model specifications and make results difficult to compare. To ensure a consistent and comparable estimation sample across all regressions, we removed firm-year rows where any required variable for the analysis was missing (SRR measures, M&A outcomes, firm financial controls, and CEO characteristics).

Before cleaning step we have 2567 records. After cleaning step, our final dataset consisted of 965 records providing a robust foundation for further analysis.

In [23]:

```
# choose only columns needed for the regression
final_data = final_merged[["publish_year",
    "gvkey",
    "exec_fullname",
    "age",
    "bonus",
    "option_awards",
    "salary",
    "stock_awards",
    "gsector","normalized_SRR_mean",
    "normalized_SRR_median",
    "log_at_lag",
    "at",
    "Sales_Growth",
    "PPE_Growth",
    "TobinQ",
    "Leverage",
    "ROA",
    "EBIT_Margin",
    "Aqc_Ratio",
    "CF",
    "tenure_years",
    "gender",
    "transactionsize_usd",
    "shrown_excl_opts_pct",
    "opt_unex_exer_est_val",
    "opt_unex_exer_num",
    "prcc_f",
    "at_lag",
    "revt_lag",
    "SRR_mean",
    "SRR_median"

]]
# Sort by CEO and year, then shift transactionsize_usd forward (next year)
final_data['transactionsize_usd_nextyear'] = (
    final_merged
    .sort_values(['gvkey', 'exec_fullname', 'publish_year'])
    .groupby(['gvkey', 'exec_fullname'])['transactionsize_usd']
    .shift(-1)  # Negative shift = future value
)

# Create transactionsize_usd_next2year
final_data['transactionsize_usd_next2year'] = (
    final_merged
    .sort_values(['gvkey', 'exec_fullname', 'publish_year'])
    .groupby(['gvkey', 'exec_fullname'])['transactionsize_usd']
    .shift(-2)  # Negative shift = future value
)

# Update your selection to include it
final_data = final_data[[
    "publish_year", "gvkey", "exec_fullname", "age", "bonus", "option_awards",
    "salary", "stock_awards", "gsector", "normalized_SRR_mean", "normalized_SRR_median",
    "log_at_lag", "at", "Sales_Growth", "PPE_Growth", "TobinQ", "Leverage",
    "ROA", "EBIT_Margin", "Aqc_Ratio", "CF", "tenure_years", "gender",
    "transactionsize_usd", "transactionsize_usd_nextyear", "transactionsize_usd_next2year", # Added here
    "shrown_excl_opts_pct", "opt_unex_exer_est_val", "opt_unex_exer_num",
    "prcc_f", "at_lag", "revt_lag","SRR_mean",
    "SRR_median"
]]

final_data = final_data.replace([np.inf, -np.inf], np.nan)  # turn inf into NaN
final_data = final_data.dropna(how="any")                   # drop rows with any NaN

#create log_transaction_size_usd column
final_data['log_transaction_size_usd_nextyear'] = np.log(final_data['transactionsize_usd_nextyear'] + 1)  # add 1 to avoid log(0)
final_data['log_transaction_size_usd'] = np.log(final_data['transactionsize_usd'] + 1)  # add 1 to avoid log(0)
final_data['log_transaction_size_usd_next2year'] = np.log(final_data['transactionsize_usd_next2year'] + 1)  # add 1 to avoid log(0)
final_data
#create log_revt_lag column
final_data['log_revt_lag'] = np.log(final_data['revt_lag'] + 1)  # add 1 to avoid log(0)
final_data

# log bonus
final_data["log_bonus"] = np.log(final_data["bonus"] + 1)

# log option awards
final_data["log_option_awards"] = np.log(final_data["option_awards"] + 1)

# log stock awards
final_data["log_stock_awards"] = np.log(final_data["stock_awards"] + 1)
# Check the structure
print("--- Data Info ---")
print(final_data.info())

# Check summary statistics
print("\n--- Summary Statistics ---")
display(final_data[["age", "bonus", "option_awards", "salary", "stock_awards","log_transaction_size_usd","log_transaction_size_usd_nextyear",
                    "log_transaction_size_usd_next2year","log_revt_lag","log_bonus","log_option_awards","log_stock_awards"]].describe())
```

```
--- Data Info ---
<class 'pandas.core.frame.DataFrame'>
Index: 965 entries, 1 to 5829
Data columns (total 41 columns):
 #   Column                              Non-Null Count  Dtype
---  ------                              --------------  -----
 0   publish_year                        965 non-null    Int64
 1   gvkey                               965 non-null    object
 2   exec_fullname                       965 non-null    object
 3   age                                 965 non-null    float64
 4   bonus                               965 non-null    float64
 5   option_awards                       965 non-null    float64
 6   salary                              965 non-null    float64
 7   stock_awards                        965 non-null    float64
 8   gsector                             965 non-null    float64
 9   normalized_SRR_mean                 965 non-null    float64
 10  normalized_SRR_median               965 non-null    float64
 11  log_at_lag                          965 non-null    float64
 12  at                                  965 non-null    float64
 13  Sales_Growth                        965 non-null    float64
 14  PPE_Growth                          965 non-null    float64
 15  TobinQ                              965 non-null    float64
 16  Leverage                            965 non-null    float64
 17  ROA                                 965 non-null    float64
 18  EBIT_Margin                         965 non-null    float64
 19  Aqc_Ratio                           965 non-null    float64
 20  CF                                  965 non-null    float64
 21  tenure_years                        965 non-null    float64
 22  gender                              965 non-null    float64
 23  transactionsize_usd                 965 non-null    float64
 24  transactionsize_usd_nextyear        965 non-null    float64
 25  transactionsize_usd_next2year       965 non-null    float64
 26  shrown_excl_opts_pct                965 non-null    float64
 27  opt_unex_exer_est_val               965 non-null    float64
 28  opt_unex_exer_num                   965 non-null    float64
 29  prcc_f                              965 non-null    float64
 30  at_lag                              965 non-null    float64
 31  revt_lag                            965 non-null    float64
 32  SRR_mean                            965 non-null    float64
 33  SRR_median                          965 non-null    float64
 34  log_transaction_size_usd_nextyear   965 non-null    float64
 35  log_transaction_size_usd            965 non-null    float64
 36  log_transaction_size_usd_next2year  965 non-null    float64
 37  log_revt_lag                        965 non-null    float64
 38  log_bonus                           965 non-null    float64
 39  log_option_awards                   965 non-null    float64
 40  log_stock_awards                    965 non-null    float64
dtypes: Int64(1), float64(38), object(2)
memory usage: 317.6+ KB
None

--- Summary Statistics ---
```

```
/var/folders/6h/8882zf396gl7nvkkt7wy23dh0000gn/T/ipykernel_29549/1500480180.py:36: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  final_data['transactionsize_usd_nextyear'] = (
/var/folders/6h/8882zf396gl7nvkkt7wy23dh0000gn/T/ipykernel_29549/1500480180.py:44: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  final_data['transactionsize_usd_next2year'] = (
```

	age	bonus	option_awards	salary	stock_awards	log_transaction_size_usd	log_transaction_size_usd_nextyear	log_transaction_size_usd_next2year	log_revt_lag	log_bon
count	965.000000	965.000000	965.000000	965.000000	965.000000	965.000000	965.000000	965.000000	965.000000	965.00000
mean	55.900518	349.127816	4168.286699	1153.030971	8359.569846	4.899640	4.956017	4.963760	9.468198	0.9244
std	6.896723	1438.796118	16718.317894	551.152095	16749.865100	2.687455	2.688190	2.696521	1.500813	2.4422
min	31.000000	0.000000	0.000000	0.001000	0.000000	0.000000	0.000000	0.000000	4.207450	0.0000
25%	52.000000	0.000000	0.000000	900.000000	2800.005000	2.890372	2.970964	2.959184	8.521384	0.0000
50%	56.000000	0.000000	45.543000	1100.000000	5999.877000	4.982006	5.041573	5.010635	9.445204	0.0000
75%	60.000000	0.000000	3099.994000	1461.248000	10000.000000	6.823286	6.906755	6.908755	10.519970	0.0000
max	88.000000	19350.000000	245953.429000	5600.000000	376180.000000	12.873120	12.873120	12.873120	13.253327	9.8704

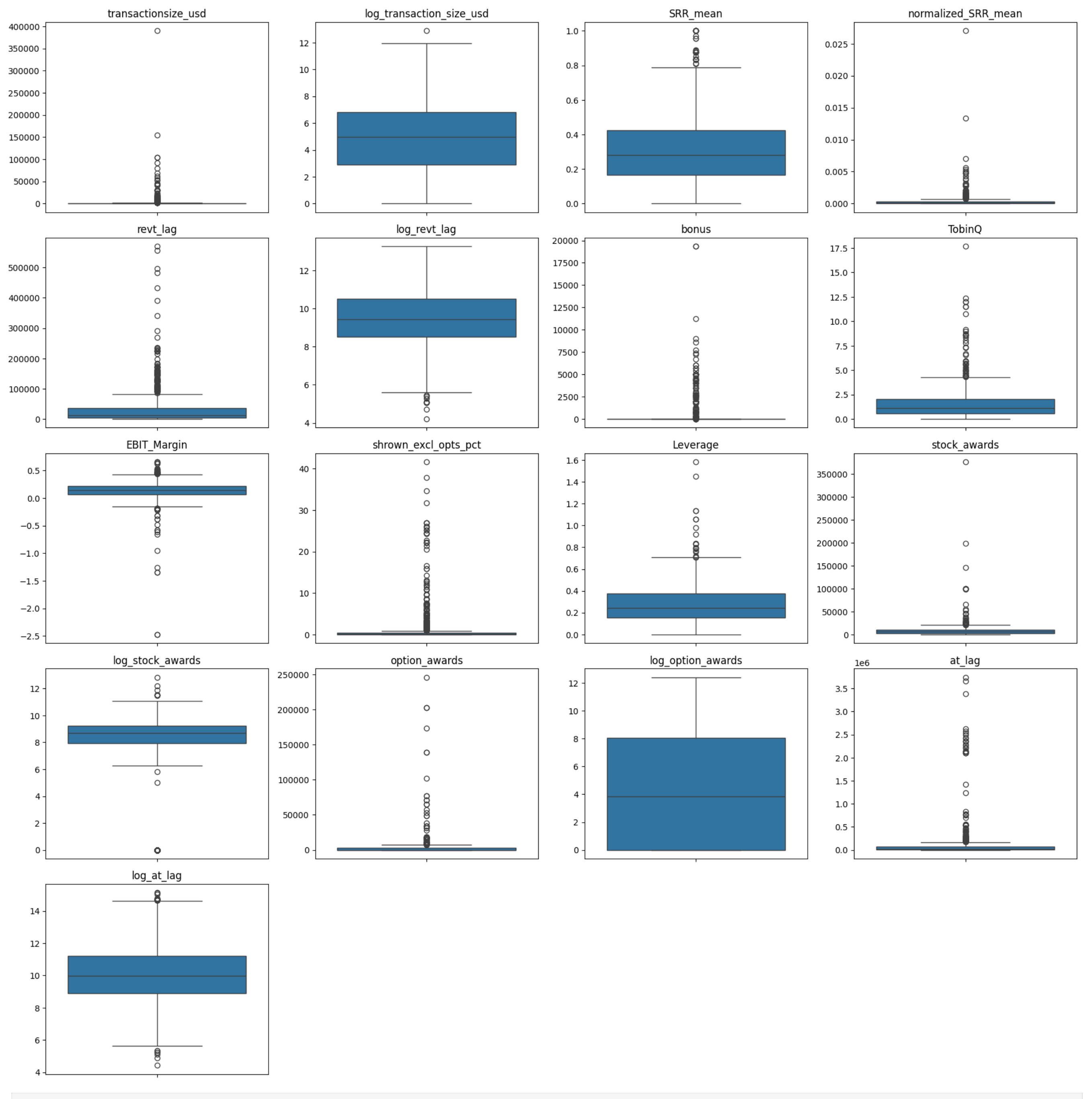
2.1 Data visualization

Purpose: We use boxplot to identify influential outliers that could drive results and see variables after log transformation

Interpret: Some raw measures perform right-skewed outliners: transactionsize_usd, revt_lag, bonus, at_lag, option_awards, stock_awards. Those are much better than after transformation.

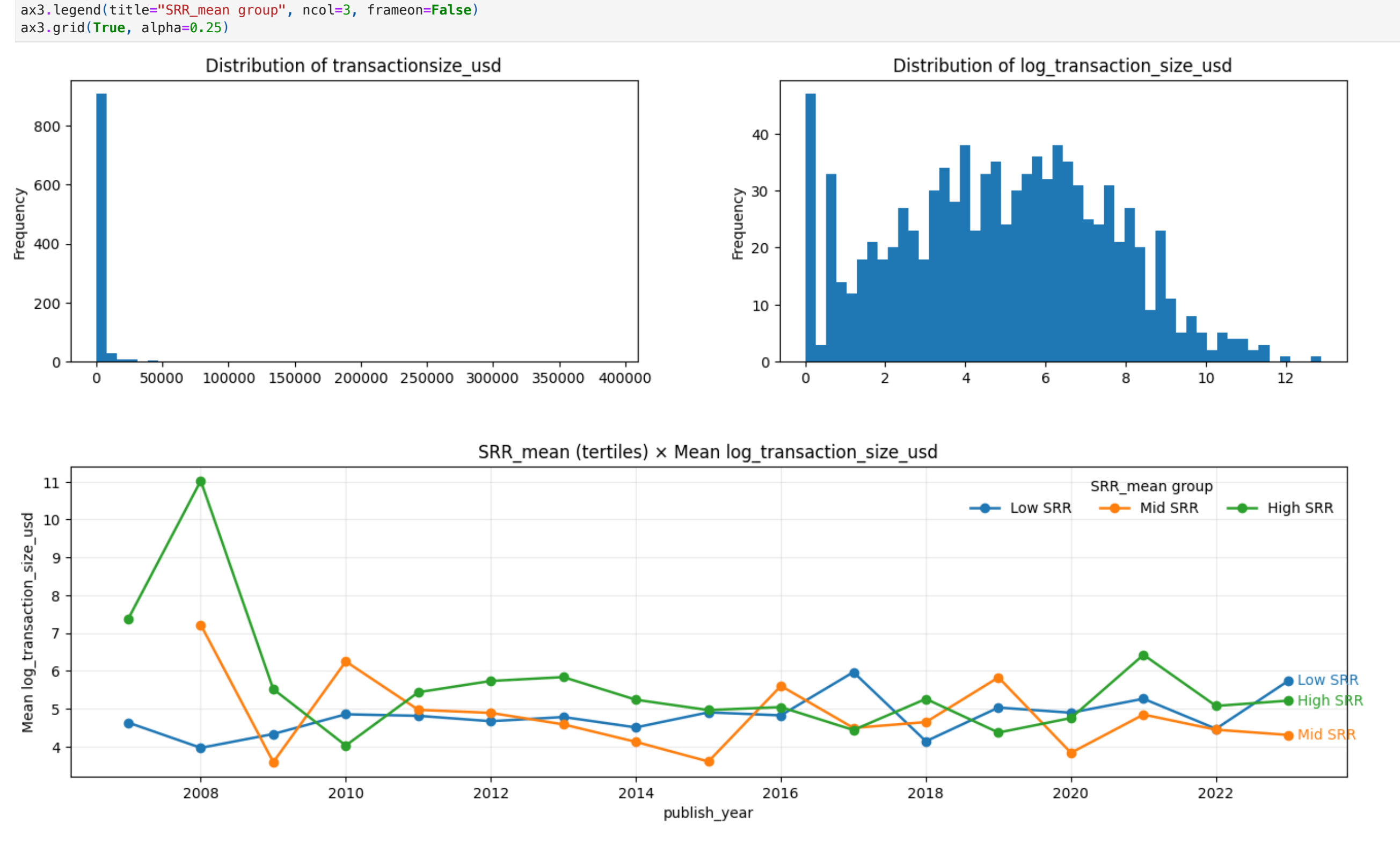
In [24]:

```
# Variables
vars_to_plot = [
    "transactionsize_usd",
    "log_transaction_size_usd",
    "SRR_mean",
    "normalized_SRR_mean",
    "revt_lag",
    "log_revt_lag",
    "bonus",
    "TobinQ",
    "EBIT_Margin",
    "shrown_excl_opts_pct",
    "Leverage",
    "stock_awards","log_stock_awards",
    "option_awards","log_option_awards",
    "at_lag","log_at_lag",

]

# Create a figure with 5 rows and 4 columns
fig, axes = plt.subplots(5, 4, figsize=(18, 18))
axes = axes.flatten()

# Boxplots
for i, col in enumerate(vars_to_plot):
    d = final_data[col].dropna()
    sns.boxplot(y=d, ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel("")
    axes[i].set_ylabel("")

# Turn off unused subplots
for j in range(len(vars_to_plot), 20):
    axes[j].axis("off")

plt.tight_layout()
plt.show()
```



In [25]:

```
df = final_data.copy()


# --- keep only what we need ---
cols = ["transactionsize_usd", "log_transaction_size_usd", "SRR_mean", "normalized_SRR_mean", "publish_year"]
d = df[cols].dropna(subset=["log_transaction_size_usd", "publish_year"]).copy()
d["publish_year"] = d["publish_year"].astype(int)

# ========================
# Figure layout:
# Row 1: 1x2 distributions
# Row 2: SRR interaction (spans both columns)
# Row 3: normalized SRR interaction (spans both columns)
# ========================
fig = plt.figure(figsize=(16, 14))
gs = fig.add_gridspec(nrows=3, ncols=2, height_ratios=[1, 1.1, 1.1], hspace=0.35, wspace=0.25)

# ---------- Top row: distributions ----------
ax1 = fig.add_subplot(gs[0, 0])
raw = d["transactionsize_usd"].dropna()
ax1.hist(raw, bins=50)
ax1.set_title("Distribution of transactionsize_usd")
ax1.set_ylabel("Frequency")

ax2 = fig.add_subplot(gs[0, 1])
logv = d["log_transaction_size_usd"].dropna()
ax2.hist(logv, bins=50)
ax2.set_title("Distribution of log_transaction_size_usd")
ax2.set_ylabel("Frequency")


# ---------- Bottom 1: SRR_mean interaction over years ----------
ax3 = fig.add_subplot(gs[1, :])

tmp = d.dropna(subset=["SRR_mean", "log_transaction_size_usd", "publish_year"]).copy()
tmp["SRR_group"] = pd.qcut(tmp["SRR_mean"], q=3, labels=["Low SRR", "Mid SRR", "High SRR"])

g1 = (tmp.groupby(["publish_year", "SRR_group"])["log_transaction_size_usd"]
        .mean()
        .reset_index()
        .pivot(index="publish_year", columns="SRR_group", values="log_transaction_size_usd")
        .sort_index())

# Choose colors (explicitly requested by you)
color_map = {
    "Low SRR":  "tab:blue",
    "Mid SRR":  "tab:orange",
    "High SRR": "tab:green",
}

for grp in g1.columns:
    y = g1[grp]
    ax3.plot(
        g1.index, y,
        marker="o",
        linewidth=1.8,
        color=color_map.get(grp, None),
        label=grp
    )

    # ---- Label the line on the chart (at the last available point) ----
    last_valid_year = y.dropna().index.max()
    if pd.notna(last_valid_year):
        last_y = y.loc[last_valid_year]
        ax3.annotate(
            grp,
            xy=(last_valid_year, last_y),
            xytext=(6, 0),             # small right offset
            textcoords="offset points",
            va="center",
            fontsize=10,
            color=color_map.get(grp, "black")
        )

ax3.set_title("SRR_mean (tertiles) × Mean log_transaction_size_usd")
ax3.set_xlabel("publish_year")
ax3.set_ylabel("Mean log_transaction_size_usd")
ax3.legend(title="SRR_mean group", ncol=3, frameon=False)
ax3.grid(True, alpha=0.25)
```



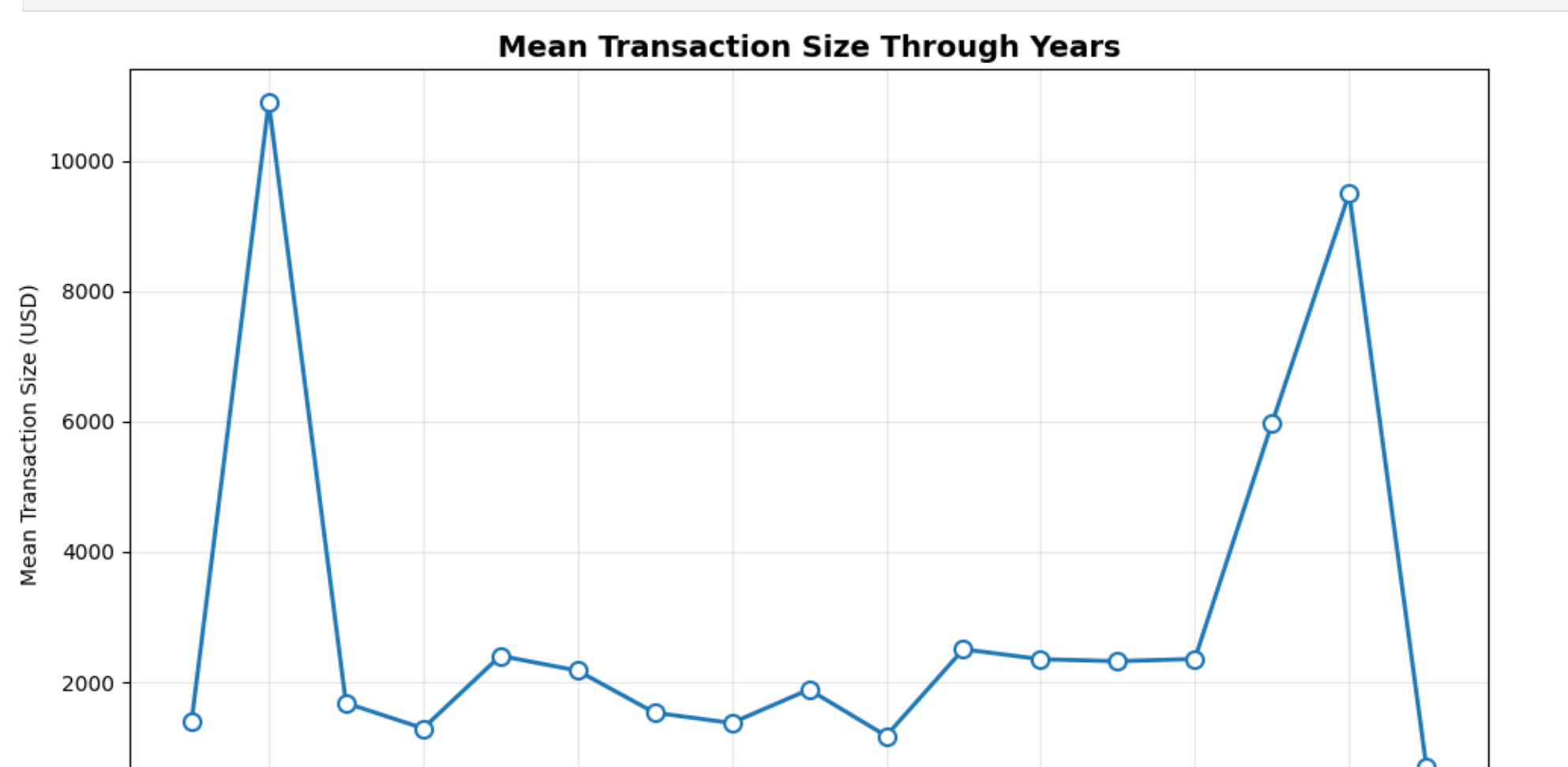
Transaction size is really skewed, log transaction will be used to normalize the distribitution

The extreme separation in 2007–2010 is likely driven by a small number of observations and/or influential mega-deals, making group means in those years less stable.

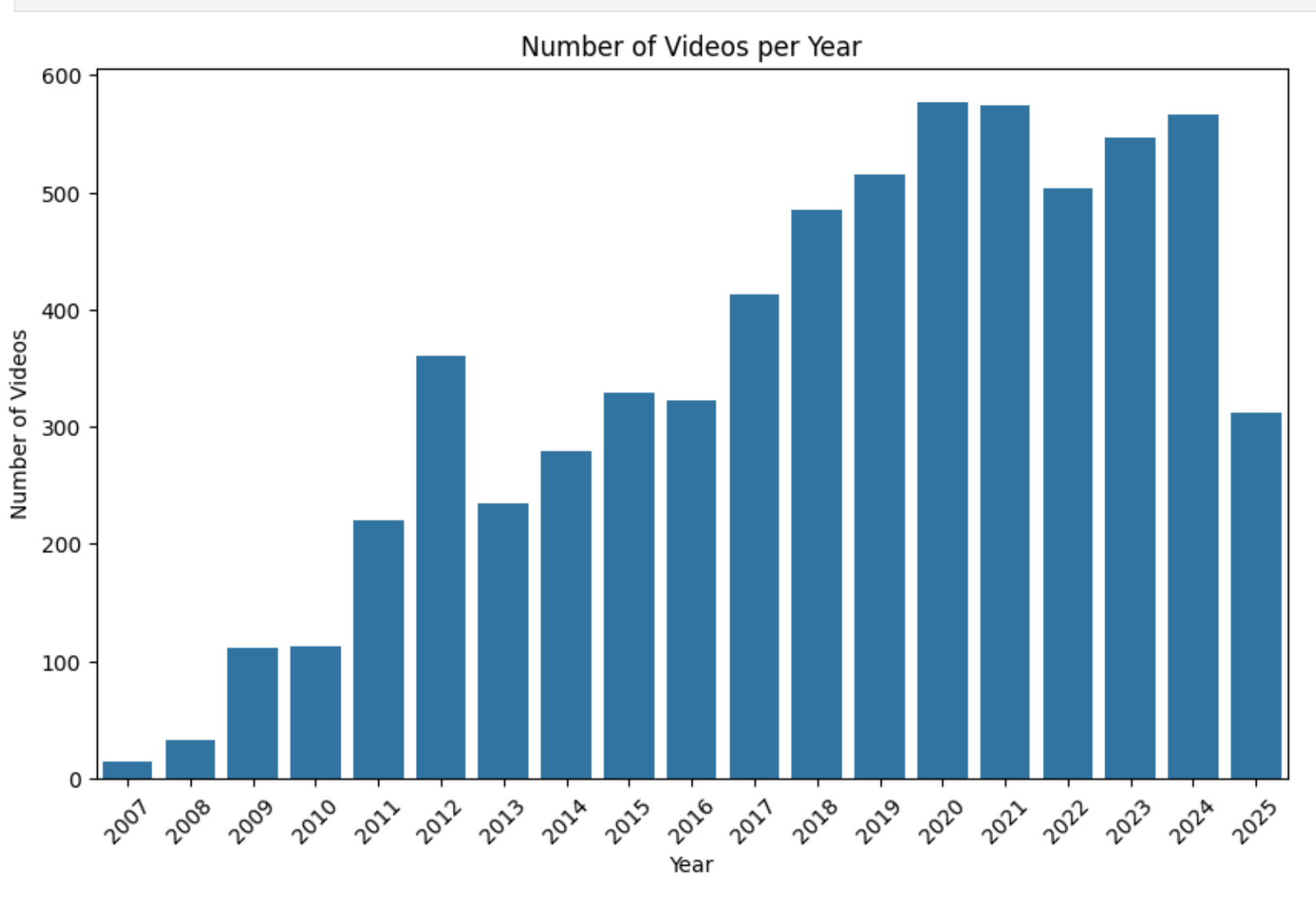
Visually, there is no stable pattern for high SRR always make larger deals each year. In first period (2007-2009) SRR is very unstable suggests that there were too few observations.

In [26]:

```
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator
# Calculate mean transaction size per year
mean_data = (
    final_data.groupby("publish_year", as_index=False)["transactionsize_usd"]
    .mean()
    .rename(columns={"transactionsize_usd": "mean_transactionsize_usd"})
)
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(
    mean_data["publish_year"],
    mean_data["mean_transactionsize_usd"],
    marker="o", linewidth=2,
    markersize=8, markerfacecolor="white", markeredgewidth=1.5
)

ax.set_title("Mean Transaction Size Through Years", fontsize=14, fontweight="bold")
ax.set_xlabel("Fiscal Year")
ax.set_ylabel("Mean Transaction Size (USD)")

ax.xaxis.set_major_locator(MaxNLocator(integer=True))
ax.tick_params(axis="x", rotation=45)

ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

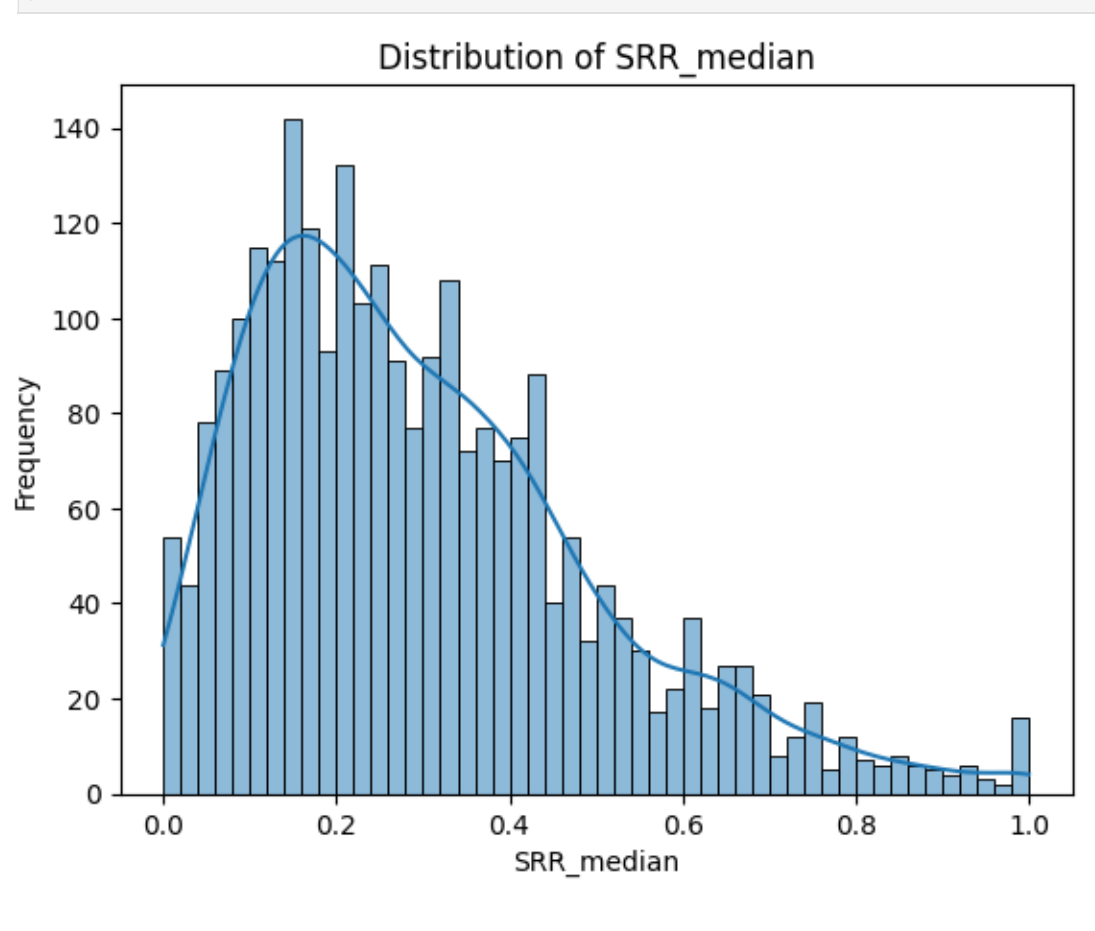


In [27]:
```python
#plot the number of videos per year
plt.figure(figsize=(10, 6))
sns.countplot(x='publish_year', data=interview, order=sorted(interview['publish_year'].dropna().unique()))
plt.title('Number of Videos per Year')
plt.xlabel('Year')
plt.ylabel('Number of Videos')
plt.xticks(rotation=45)
plt.show()
```



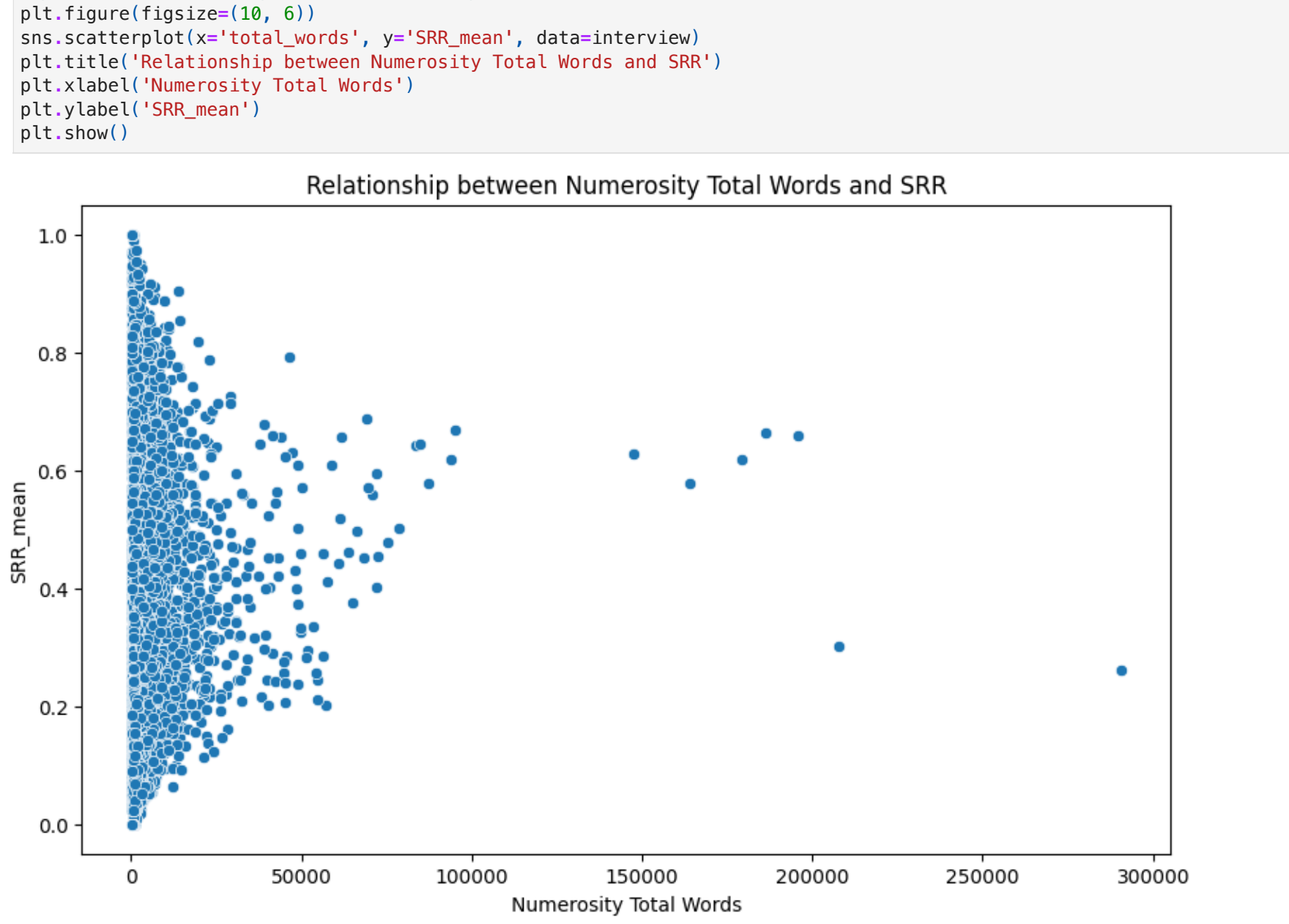
The bar chart shows a strong upward trend in the number of videos over time. Peaking around 2020–2021 (~570–580 videos)

In [28]:
```python
# plot the distribution of SRR_median
sns.histplot(final_merged['SRR_median'], bins=50, kde=True)
plt.title('Distribution of SRR_median')
plt.xlabel('SRR_median')
plt.ylabel('Frequency')
plt.show()
```



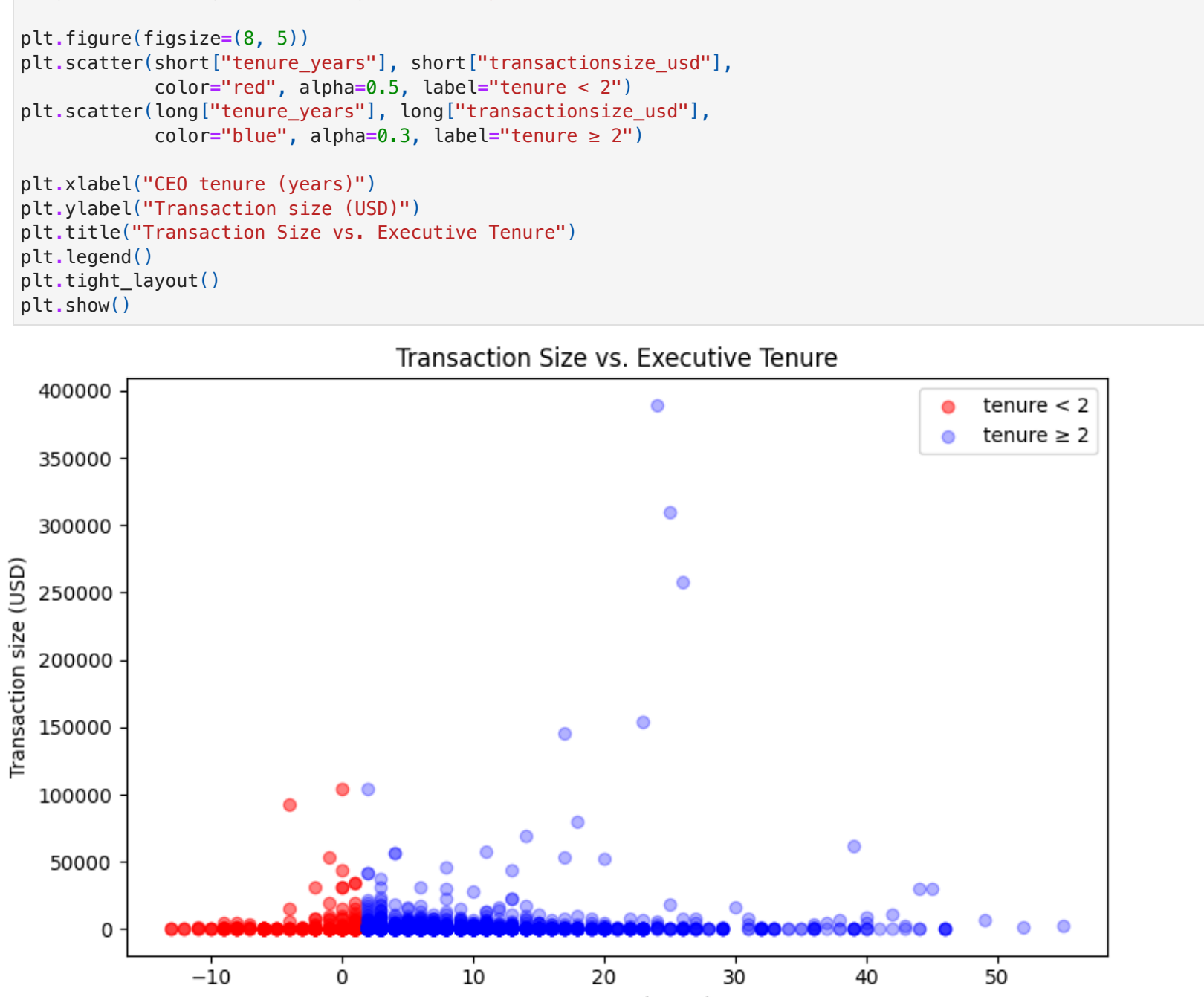
The histogram shows SRR_median is right-skewed: most observations cluster at low values (roughly 0.1–0.3, peaking around ~0.15–0.2), and the frequency declines steadily as SRR_median increases, with a long tail extending toward 1.0 (few very high-SRR cases).

In [29]:
```python
# Word Count Binning Analysis
interview["word_bin"] = pd.cut(
    interview["total_words"],
    bins=[0, 100, 300, 500, 1000, 5000]
)
bin_means = interview.groupby("word_bin")["SRR_mean"].mean()
fig, ax1 = plt.subplots(figsize=(10, 6))
bin_means.plot(kind="bar", ax=ax1, color="skyblue")
ax1.set_ylabel("Mean SRR")
ax1.set_xlabel(" Total Words (bins)")
ax2 = ax1.twinx()
bin_counts = interview["word_bin"].value_counts().sort_index()
bin_counts.plot(kind="line", ax=ax2, color="orange", marker="o")
ax2.set_ylabel("Number of Interviews")
plt.title("Mean SRR by Word Count Bins")
plt.show()
```



Mean SRR is highest in very short interviews (0–100 words) and generally declines as word count increases, stabilizing around ~0.27–0.33 for longer interviews. SRR estimates from short transcripts are noisier and more extreme, while longer interviews provide more stable averages.

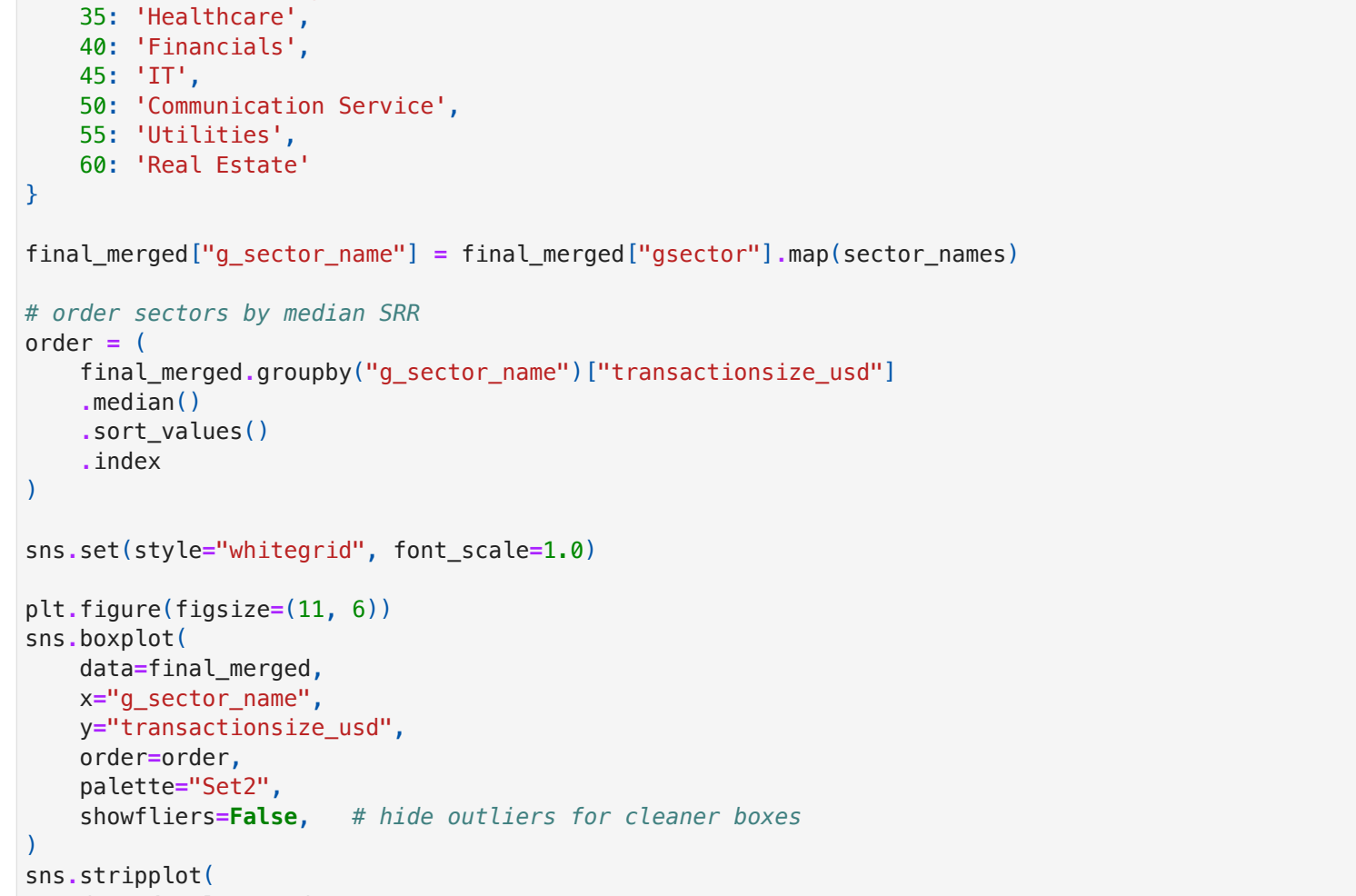
In [30]:
```python
# plot the relationship between numerosity_total_words and SRR
plt.figure(figsize=(10, 6))
sns.scatterplot(x='total_words', y='SRR_mean', data=interview)
plt.title('Relationship between Numerosity Total Words and SRR')
plt.xlabel('Numerosity Total Words')
plt.ylabel('SRR_mean')
plt.show()
```



Very short interviews show high dispersion (SRR ranges from near 0 to near 1), while as word count increases, SRR values become more concentrated around mid-levels (roughly 0.2–0.6). This indicates SRR is noisier in short transcripts and more stable for longer interviews.

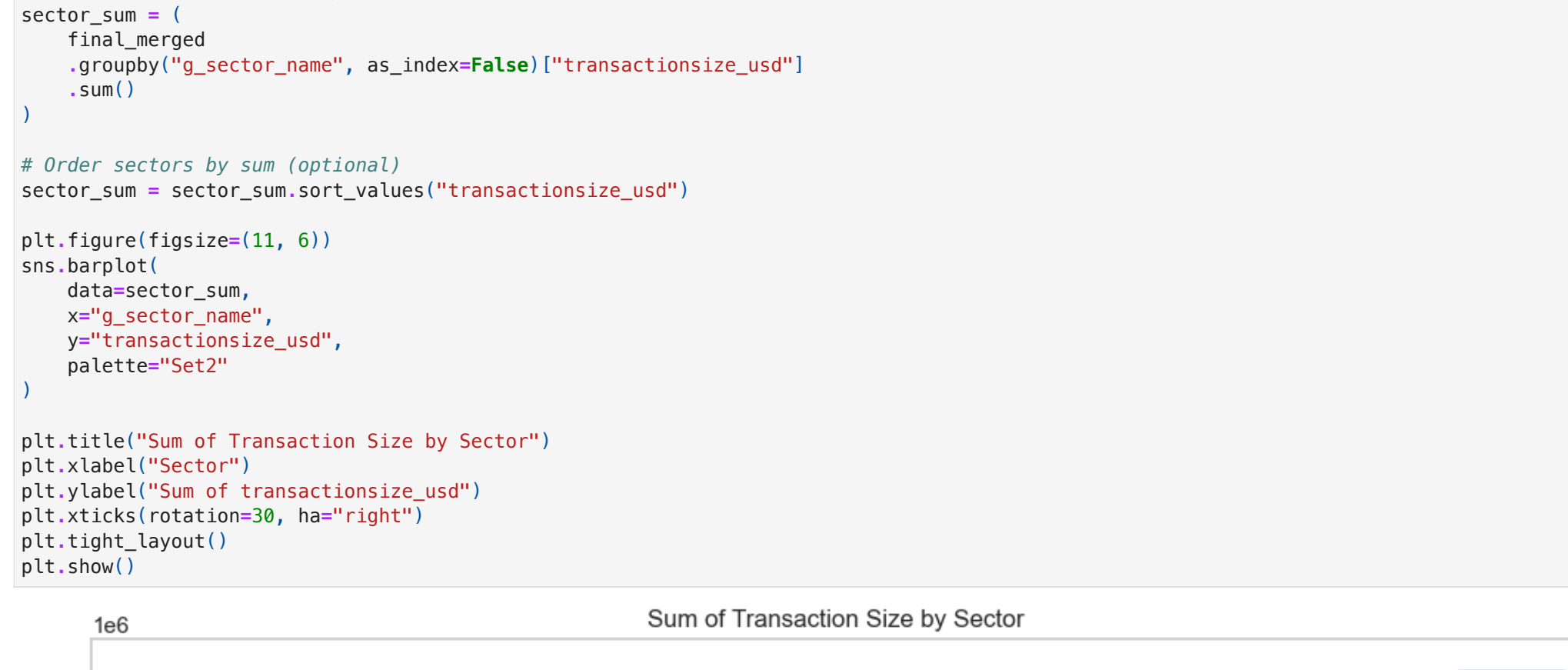
Short interviews show high SRR -> SRR should be normalized

In [31]:
```python
short = final_merged[final_merged["tenure_years"] < 2]
long  = final_merged[final_merged["tenure_years"] >= 2]

plt.figure(figsize=(8, 5))
plt.scatter(short["tenure_years"], short["transactionsize_usd"],
            color="red", alpha=0.5, label="tenure < 2")
plt.scatter(long["tenure_years"], long["transactionsize_usd"],
            color="blue", alpha=0.3, label="tenure ≥ 2")

plt.xlabel("CEO tenure (years)")
plt.ylabel("Transaction size (USD)")
plt.title("Transaction Size vs. Executive Tenure")
plt.legend()
plt.tight_layout()
plt.show()
```



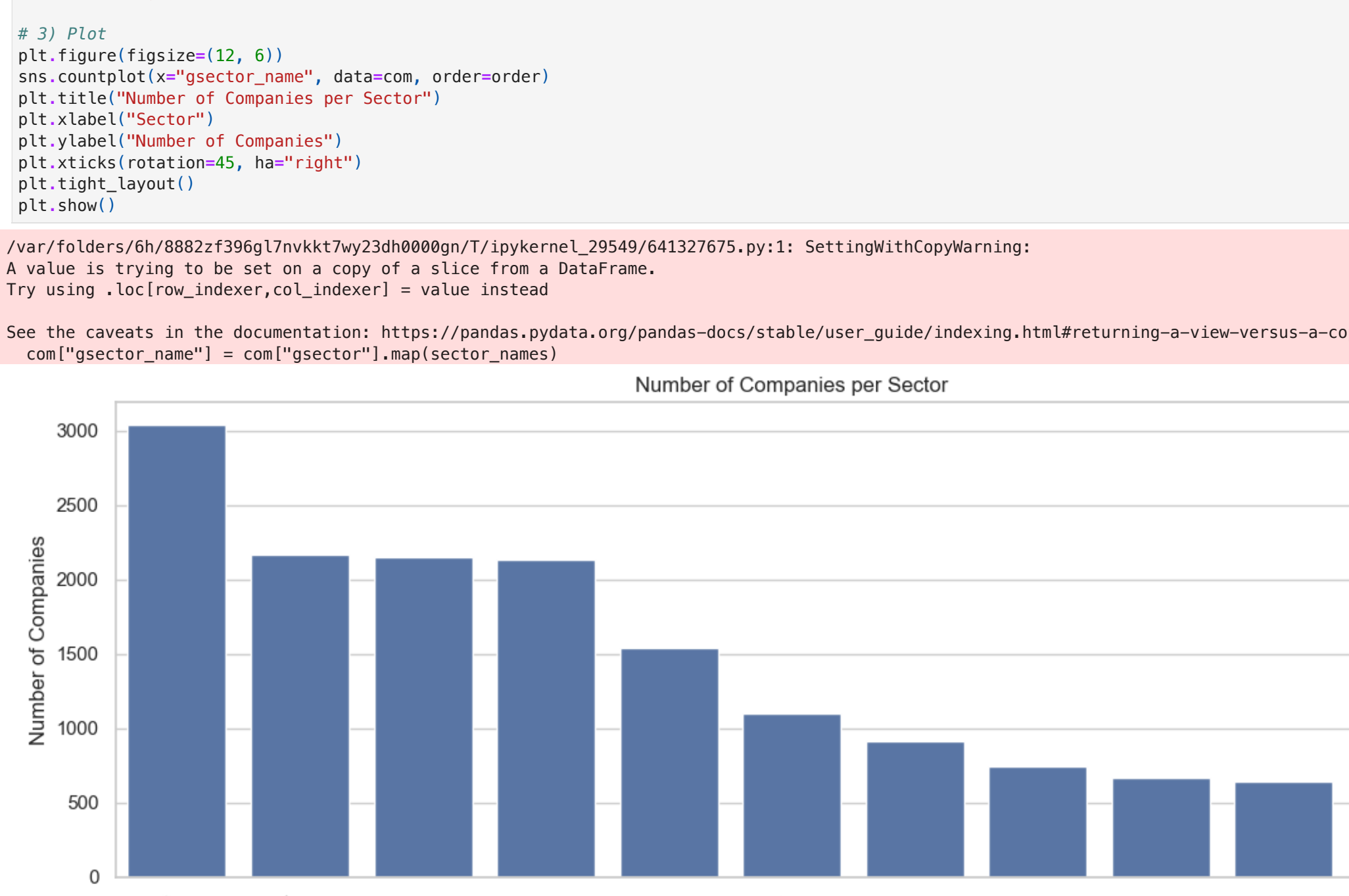
Deal size varies a lot both for early and later-tenure CEOs. And an acquisition deal is usually approved by the company's board of directors, not only CEO. So we dont exlude tenure year<2. Research reports that CEO with more tenure year tend to make more shareholder value in M&A. In this study it will be just controlled variable, we focus on SRR.

In [32]:
```python
# ensure sector names exist
sector_names = {
    10: 'Energy',
    15: 'Materials',
    20: 'Industrials',
    25: 'Commercial Discretionary',
    30: 'Consumer Staples',
    35: 'Healthcare',
    40: 'Financials',
    45: 'IT',
    50: 'Communication Service',
    55: 'Utilities',
    60: 'Real Estate'
}

final_merged["g_sector_name"] = final_merged["gsector"].map(sector_names)

# order sectors by median SRR
order = (
    final_merged.groupby("g_sector_name")["transactionsize_usd"]
    .median()
    .sort_values()
    .index
)

sns.set(style="whitegrid", font_scale=1.0)

plt.figure(figsize=(11, 6))
sns.boxplot(
    data=final_merged,
    x="g_sector_name",
    y="transactionsize_usd",
    order=order,
    palette="Set2",
    showfliers=False,   # hide outliers for cleaner boxes
)
sns.stripplot(
    data=final_merged,
    x="g_sector_name",
    y="normalized_SRR_mean",
    order=order,
    color="black",
    size=2,
    alpha=0.3
)

plt.title("Transaction Size by Sector")
plt.xlabel("Sector")
plt.ylabel("transactionsize_usd")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```



The boxplots show that Financials (and to a lesser extent Healthcare and Energy) have the largest typical deal sizes.

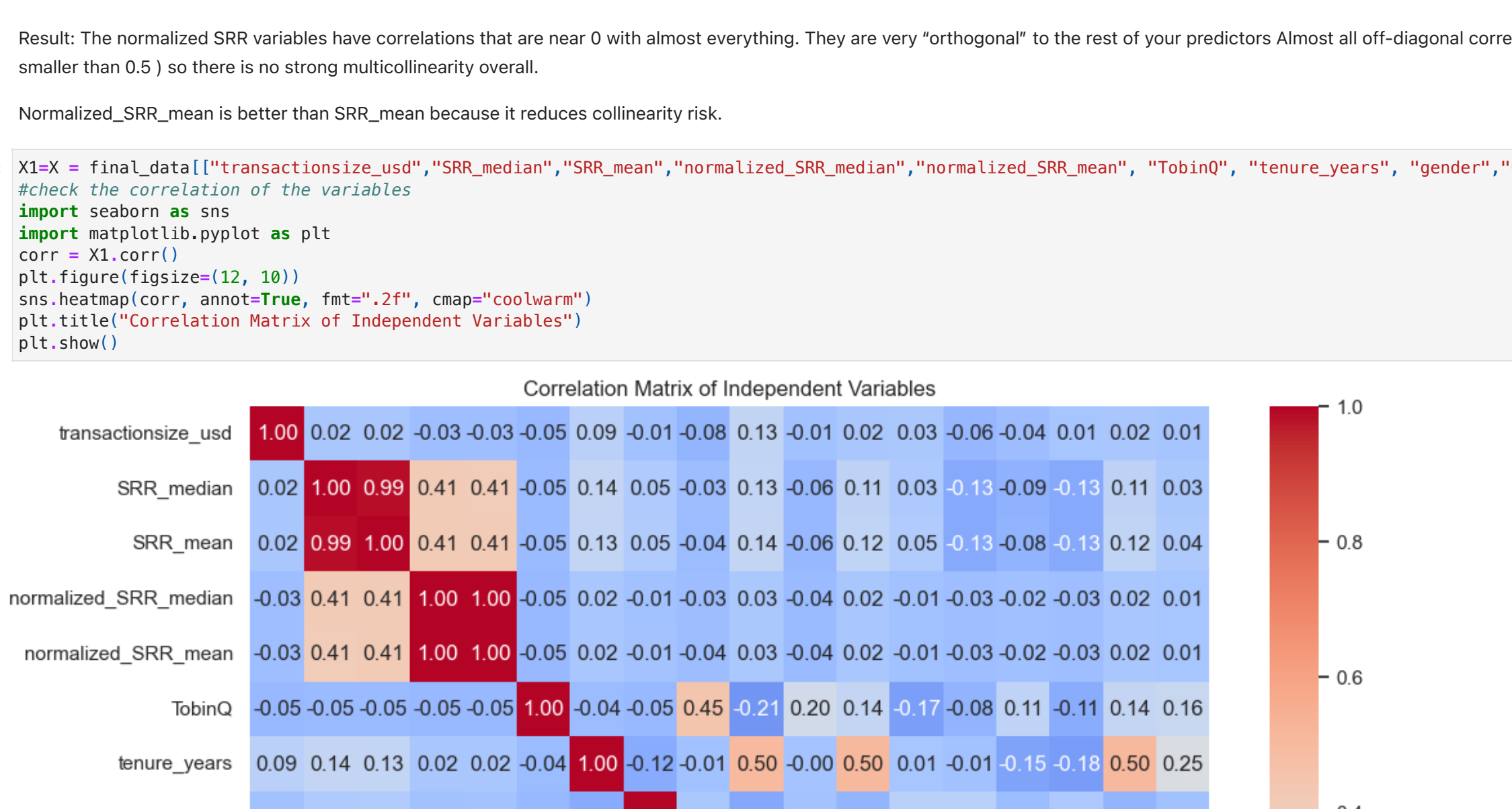
In [33]:
```python
# Sum of transaction size by sector
sector_sum = (
    final_merged
    .groupby("g_sector_name", as_index=False)["transactionsize_usd"]
    .sum()
)

# Order sectors by sum (optional)
sector_sum = sector_sum.sort_values("transactionsize_usd")

plt.figure(figsize=(11, 6))
sns.barplot(
    data=sector_sum,
    x="g_sector_name",
    y="transactionsize_usd",
    palette="Set2"
)

plt.title("Sum of Transaction Size by Sector")
plt.xlabel("Sector")
plt.ylabel("Sum of transactionsize_usd")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```



Financials(and to a lesser extent, Healthcare and Energy) have the largest typical deal sizes.

In [34]:
```python
com["gsector_name"] = com["gsector"].map(sector_names)
# 2) Order bars by count (descending)
order = com["gsector_name"].value_counts().index

# 3) Plot
plt.figure(figsize=(12, 6))
sns.countplot(x="gsector_name", data=com, order=order)
plt.title("Number of Companies per Sector")
plt.xlabel("Sector")
plt.ylabel("Number of Companies")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
/var/folders/6h/8882zf396gl7nvkkt7wy23dh0000gn/T/ipykernel_29549/641327675.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  com["gsector_name"] = com["gsector"].map(sector_names)
```



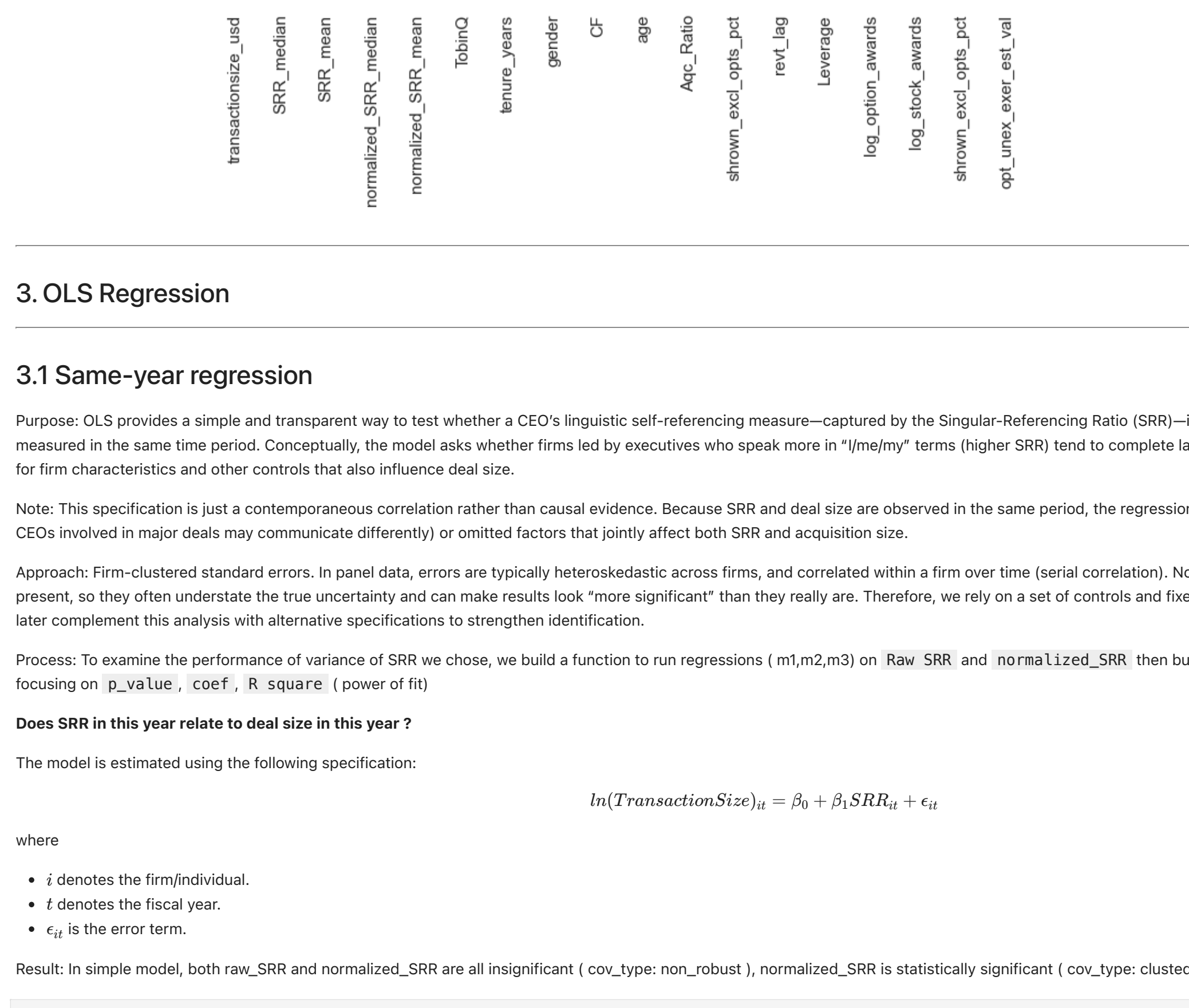
Commercial Discretionaty dominate by a large margin, followed by IT , Financials and Industrials. All other sectors contribute comparatively smaller totals, indicating that aggregate deal activity is driven mostly by these high-volume industries.

2.2 Correlation of variables

Result: The normalized SRR variables have correlations that are near 0 with almost everything. They are very "orthogonal" to the rest of your predictors Almost all off-diagonal correlations are small (close to 0 and smaller than 0.5) so there is no strong multicollinearity overall.

Normalized_SRR_mean is better than SRR_mean because it reduces collinearity risk.

In [35]:
```python
X1=X = final_data[["transactionsize_usd","SRR_median","SRR_mean","normalized_SRR_median","normalized_SRR_mean", "TobinQ", "tenure_years", "gender","CF","age","Aqc_Ratio","shrown
#check the correlation of the variables
import seaborn as sns
import matplotlib.pyplot as plt
corr = X1.corr()
plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix of Independent Variables")
plt.show()
```



---

3. OLS Regression

3.1 Same-year regression

Purpose: OLS provides a simple and transparent way to test whether a CEO's linguistic self-referencing measure—captured by the Singular-Referencing Ratio (SRR)—is related to M&A deal size when both are measured in the same time period. Conceptually, the model asks whether firms led by executives who speak more in "I/me/my" terms (higher SRR) tend to complete larger deals in that same year, after accounting for firm characteristics and other controls that also influence deal size.

Note: This specification is just a contemporaneous correlation rather than causal evidence. Because SRR and deal size are observed in the same period, the regression cannot fully rule out reverse causality (e.g., CEOs involved in major deals may communicate differently) or omitted factors that jointly affect both SRR and acquisition size.

Approach: Firm-clustered standard errors. In panel data, errors are typically heteroskedastic across firms, and correlated within a firm over time (serial correlation). Non-robust SEs assume neither of these is present, so they often understate the true uncertainty and can make results look "more significant" than they really are. Therefore, we rely on a set of controls and fixed effects, firm-clustered SE by gvkey and we later complement this analysis with alternative specifications to strengthen identification.

Process: To examine the performance of variance of SRR we chose, we build a function to run regressions (m1,m2,m3) on `Raw SRR` and `normalized_SRR` then build a function to demonstrate the OLS result, focusing on `p_value` , `coef` , `R square` (power of fit)

Does SRR in this year relate to deal size in this year ?

The model is estimated using the following specification:

$$ln(TransactionSize)_{it} = \beta_0 + \beta_1 SRR_{it} + \epsilon_{it} \tag{1}$$

where

- i denotes the firm/individual.
- t denotes the fiscal year.
- ϵ_{it} is the error term.

Result: In simple model, both raw_SRR and normalized_SRR are all insignificant (cov_type: non_robust), normalized_SRR is statistically significant (cov_type: clusted).

In [36]:
```python
def simple_ols(data, dependent_var, independent_vars):
    """
    OLS model: dependent_var ~ const + independent_var
    """
    rows = []
    for x in independent_vars:
        d = data[[dependent_var, x]].dropna()
        X = sm.add_constant(d[[x]])
        y = d[dependent_var]

        res = sm.OLS(y, X).fit()

        rows.append({
            "model": x,
            "coef": res.params[x],
            "std_err": res.bse[x],
            "p_value": res.pvalues[x],
            "t_stat": res.tvalues[x],
            "F_stat": res.fvalue,
            "Probabilitiy_F_stat": res.f_pvalue,
            "R_squared": res.rsquared,
            "adj_R_squared": res.rsquared_adj,
            "[0.025": res.conf_int().loc[x, 0],
            "0.975]": res.conf_int().loc[x, 1],
            "kurtosis": res.resid.kurtosis,
            "skewness": res.resid.skew(),
            "Cond. No.": res.condition_number,
            "n": int(res.nobs)
        })

    table = pd.DataFrame(rows).set_index("model")
    return table[["coef", "std_err", "p_value", "t_stat", "R_squared", "adj_R_squared", "n"]]

# use it
m1_sameyear_nocl = simple_ols(
    final_data,
    dependent_var="log_transaction_size_usd",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"]
)

print("Simple regression on log_transaction_size_usd, nonrobust: \n")

display(m1_sameyear_nocl)
```

```
Simple regression on log_transaction_size_usd, nonrobust:
```

	coef	std_err	p_value	t_stat	R_squared	adj_R_squared	n
model							
SRR_median	0.669448	0.420968	0.112104	1.590260	0.002619	0.001584	965
SRR_mean	0.816221	0.432003	0.059140	1.889386	0.003693	0.002659	965
normalized_SRR_mean	-66.782098	74.455089	0.369973	-0.896945	0.000835	-0.000203	965
normalized_SRR_median	-67.594356	74.425515	0.363992	-0.908215	0.000856	-0.000182	965

In [37]:
```python
def simple_ols_clustered(data, dependent_var, independent_vars, cluster_col="gvkey"):
    """
    Runs separate bivariate OLS models: y ~ const + x
    Returns a compact table using firm-clustered standard errors.
    """
    rows = []
    for x in independent_vars:
        d = data[[dependent_var, x, cluster_col]].dropna()
        X = sm.add_constant(d[[x]])
        y = d[dependent_var]
        groups = d[cluster_col]

        res = sm.OLS(y, X).fit(
            cov_type="cluster",
            cov_kwds={"groups": groups, "use_correction": True}
        )

        rows.append({
            "model": x,
            "coef": res.params[x],
            "std_err": res.bse[x],
            "p_value": res.pvalues[x],
            "t_stat": res.tvalues[x],
            # With clustered SEs, model-wide F-stat may be unstable; keep if you want:
            "F_stat": getattr(res, "fvalue", np.nan),
            "R_squared": res.rsquared,
            "adj_R_squared": res.rsquared_adj,
            "n_obs": int(res.nobs),
            "n_clusters": d[cluster_col].nunique()
        })

    table = pd.DataFrame(rows).set_index("model")
    return table[["coef", "std_err", "p_value", "t_stat", "R_squared", "adj_R_squared", "n_obs", "n_clusters"]]

# ------- use it -----------------
m1_same_year = simple_ols_clustered(
    final_data,
    dependent_var="log_transaction_size_usd",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    cluster_col="gvkey"
)

print("Simple regression on log_transaction_size_usd, firm-clustered SE (gvkey):\n")
display(m1_same_year)
```

```
Simple regression on log_transaction_size_usd, firm-clustered SE (gvkey):
```

	coef	std_err	p_value	t_stat	R_squared	adj_R_squared	n_obs	n_clusters
model								
SRR_median	0.669448	0.494121	0.175473	1.354826	0.002619	0.001584	965	222
SRR_mean	0.816221	0.497932	0.101167	1.639222	0.003693	0.002659	965	222
normalized_SRR_mean	-66.782098	33.917111	0.048955	-1.968980	0.000835	-0.000203	965	222
normalized_SRR_median	-67.594356	33.981425	0.046684	-1.989156	0.000856	-0.000182	965	222

The control model (m2) is estimated using the following specification:

$$ln(TransactionSize)_{it} = \beta_0 + \beta_1 SRR_{it} + \sum_{k=1}^{n} \gamma_k \text{Controls}_{it,k} + \epsilon_{it} \tag{2}$$

where

- i denotes the firm/individual.
- t denotes the fiscal year.
- $Controls_{it,k}$ represents the k-th control variable for firm/individual i at time t.
- ϵ_{it} is the error term.

Result: Both SRR are insignificant (no clustering and with clustering). SRR does not have a statistically reliable association with current-year deal size.

In [38]:
```python
def ols_with_controls(data, dependent_var, independent_vars, control_vars):



    results_list = []
    models = {}

    for x in independent_vars:
        # Build formula
        formula_parts = [dependent_var, "~", x]

        # Add control variables
        for ctrl in control_vars:
            formula_parts.append(f"+ {ctrl}")

        formula = " ".join(formula_parts)

        # Fit model
        try:
            model = smf.ols(formula, data=data).fit()
            models[x] = model

            # Extract results for main independent variable
            results_list.append({
                'Independent_Variable': x,
                'Coefficient': model.params[x],
                'Std_Error': model.bse[x],
                'T_Stat': model.tvalues[x],
                'P_Value': model.pvalues[x],
                '[0.025': model.conf_int().loc[x, 0],
                '0.975]': model.conf_int().loc[x, 1],
                'R_Squared': model.rsquared,
                'Adj_R_Squared': model.rsquared_adj,
                'F_Stat': model.fvalue,
                'Prob_F_Stat': model.f_pvalue,

            })
        except Exception as e:
            print(f"Error fitting model for {x}: {str(e)}")
            continue

    results_df = pd.DataFrame(results_list)
    return results_df, models

#Regression with basic controls
control_vars = ["TobinQ", "tenure_years", "gender", "age", "log_stock_awards",
                "EBIT_Margin","shrown_excl_opts_pct","Leverage","log_option_awards","log_revt_lag","log_at_lag"]
m2_sameyear_nocl, models_basic = ols_with_controls(
    data=final_data,
    dependent_var="log_transaction_size_usd",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
)

print("Regression with Controls Results on log_transaction_size_usd with Controls:, no clustering\n")
display(m2_sameyear_nocl)
```

```
Regression with Controls Results on log_transaction_size_usd with Controls:, no clustering
```

	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	F_Stat	Prob_F_Stat
0	SRR_median	0.270576	0.414891	0.652161	0.514455	-0.543632	1.084783	0.110410	0.099196	9.846290	2.452449e-18
1	SRR_mean	0.356056	0.426407	0.835015	0.403919	-0.480750	1.192862	0.110664	0.099453	9.871752	2.165077e-18
2	normalized_SRR_mean	-19.862534	71.243908	-0.278796	0.780462	-159.675781	119.950712	0.110085	0.098867	9.813744	2.875997e-18
3	normalized_SRR_median	-20.260794	71.218074	-0.284489	0.776097	-160.023343	119.501755	0.110088	0.098870	9.814044	2.871772e-18

In [39]:
```python
formula = (
    "log_transaction_size_usd ~ normalized_SRR_median + TobinQ + tenure_years + gender + age + "
    "log_stock_awards + EBIT_Margin + shrown_excl_opts_pct + Leverage + "
    "log_option_awards + log_revt_lag + log_at_lag"
)

base = smf.ols(formula, data=final_data, missing="drop")
m2 = base.fit()  # No clustering - uses standard OLS SEs

print(m2.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     log_transaction_size_usd   R-squared:                       0.110
Model:                                  OLS   Adj. R-squared:                  0.099
Method:                       Least Squares   F-statistic:                     9.814
Date:                      Fri, 30 Jan 2026   Prob (F-statistic):           2.87e-18
Time:                              23:13:53   Log-Likelihood:                -2266.5
No. Observations:                       965   AIC:                             4559.
Df Residuals:                           952   BIC:                             4622.
Df Model:                                12
Covariance Type:                  nonrobust
=========================================================================================
                            coef    std err          t      P>|t|      [0.025      0.975]
-----------------------------------------------------------------------------------------
Intercept                 0.0286      0.907      0.031      0.975      -1.752       1.809
normalized_SRR_median   -20.2608     71.218     -0.284      0.776    -160.023     119.502
TobinQ                    0.1596      0.055      2.921      0.004       0.052       0.267
tenure_years              0.0029      0.013      0.224      0.823      -0.023       0.028
gender                    0.4373      0.343      1.273      0.203      -0.237       1.111
age                       0.0127      0.015      0.839      0.402      -0.017       0.042
log_stock_awards         -0.0020      0.033     -0.061      0.951      -0.067       0.063
EBIT_Margin               1.2352      0.428      2.888      0.004       0.396       2.075
shrown_excl_opts_pct      0.0213      0.025      0.861      0.390      -0.027       0.070
Leverage                 -0.3417      0.461     -0.741      0.459      -1.246       0.563
log_option_awards        -0.0438      0.021     -2.096      0.036      -0.085      -0.003
log_revt_lag             -0.1914      0.112     -1.712      0.087      -0.411       0.028
log_at_lag                0.5742      0.100      5.728      0.000       0.377       0.771
==============================================================================
Omnibus:                       31.298   Durbin-Watson:                   1.912
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               16.142
Skew:                          -0.105   Prob(JB):                     0.000312
Kurtosis:                       2.403   Cond. No.                     5.13e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 5.13e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

In [40]:
```python
def ols_with_controls_clustered(
    data,
    dependent_var,
    independent_vars,
    control_vars,
    cluster_col="gvkey",
    use_correction=True
):

    # remove duplicates while keeping order
    control_vars = list(dict.fromkeys(control_vars))

    results_list = []
    models = {}

    for x in independent_vars:
        formula = f"{dependent_var} ~ {x}"
        if control_vars:
            formula += " + " + " + ".join(control_vars)
        try:
            base = smf.ols(formula, data=data, missing="drop")

            # align groups to the exact estimation sample
            used_idx = base.data.row_labels
            groups = data.loc[used_idx, cluster_col]

            res = base.fit(
                cov_type="cluster",
                cov_kwds={"groups": groups, "use_correction": use_correction}
            )

            models[x] = res
            ci = res.conf_int()

            results_list.append({
                "Independent_Variable": x,
                "coef": res.params.get(x, np.nan),
                "Std_Error": res.bse.get(x, np.nan),
                "T_Stat": res.tvalues.get(x, np.nan),
                "P_Value": res.pvalues.get(x, np.nan),
                "[0.025": ci.loc[x, 0] if x in ci.index else np.nan,
                "0.975]": ci.loc[x, 1] if x in ci.index else np.nan,
                "R_Squared": res.rsquared,
                "Adj_R_Squared": res.rsquared_adj,
                # With clustered SEs, model-wide F can be unstable; keep if you want:
                "N_Clusters": int(pd.Series(groups).nunique())
            })

        except Exception as e:
            print(f"Error fitting model for {x}: {str(e)}")
            continue

    results_df = pd.DataFrame(results_list)
    return results_df, models


# --------- run it -----------
control_vars = ["TobinQ", "tenure_years", "gender", "age",
                "log_stock_awards",
                "EBIT_Margin", "shrown_excl_opts_pct", "Leverage",
                "log_option_awards", "log_revt_lag", "log_at_lag"]

m2_same_year, models_basic_cl = ols_with_controls_clustered(
    data=final_data,
    dependent_var="log_transaction_size_usd",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    cluster_col="gvkey"
)

print("Regression with Controls Results on log_transaction_size_usd (clustered by gvkey):\n")
display(m2_same_year)
```

```
Regression with Controls Results on log_transaction_size_usd (clustered by gvkey):
```

	Independent_Variable	coef	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	N_Clusters
0	SRR_median	0.270576	0.393389	0.687807	0.491574	-0.500453	1.041605	0.110410	0.099196	222
1	SRR_mean	0.356056	0.400473	0.889089	0.373955	-0.428856	1.140968	0.110664	0.099453	222
2	normalized_SRR_mean	-19.862534	32.193188	-0.616979	0.537248	-82.960024	43.234956	0.110085	0.098867	222
3	normalized_SRR_median	-20.260794	32.277204	-0.627712	0.530193	-83.522952	43.001364	0.110088	0.098870	222

The full model (m3) is estimated using the following specification:

$$ln(TransactionSize)_{it} = \beta_0 + \beta_1 SRR_{it} + \sum_{k=1}^{n} \gamma_k \text{Controls}_{it,k} + \alpha_{it} + \delta_t + \epsilon_{it} \quad (3)$$

where

- i denotes the firm/individual.
- t denotes the fiscal year.
- α_i and δ_t represent entity and time fixed effects respectively.
- $Controls_{it,k}$ represents the k-th control variable for firm/individual i at time t.
- ϵ_{it} is the error term.

Result: After controlling for covariates and fixed effects, SRR (mean/median and normalized) still does not significantly explain current-year log transaction size.

In [41]:

```
# convert gvkey, gsector to categorical type
final_data["gvkey"] = final_data["gvkey"].astype("category")
final_data["gsector"] = final_data["gsector"].astype("category")
# Convert publish_year, gender to float
final_data["publish_year"] = final_data["publish_year"].astype(float)
final_data["gender"] = final_data["gender"].astype(float)
```

In [42]:

```
def ols_with_controls_and_fe(
    data: pd.DataFrame,
    dependent_var: str,
    independent_vars: list,
    control_vars: list,
    fe_specs: dict,
    cov_type: str = "nonrobust",  # keep as-is
):
    """
    Runs OLS with controls + fixed effects for each FE spec and each independent var.
    Returns:
      models_fe[fe_name][x] = fitted statsmodels result
    """
    controls_str = " + ".join(control_vars) if control_vars else "1"

    models_fe = {}
    for fe_name, fe_terms in fe_specs.items():
        models_fe[fe_name] = {}
        for x in independent_vars:
            formula = f"{dependent_var} ~ {x} + {controls_str} + {fe_terms}"
            models_fe[fe_name][x] = smf.ols(formula, data=data).fit(cov_type=cov_type)

    return models_fe


def fe_results_table_nocl(models_fe: dict):
    rows = []
    for fe_name, sub in models_fe.items():
        for x, res in sub.items():
            ci = res.conf_int()
            rows.append({
                "FE_spec": fe_name,
                "Independent_Variable": x,
                "Coefficient": res.params.get(x, float("nan")),
                "Std_Error": res.bse.get(x, float("nan")),
                "T_Stat": res.tvalues.get(x, float("nan")),
                "P_Value": res.pvalues.get(x, float("nan")),
                "[0.025": ci.loc[x, 0] if x in ci.index else float("nan"),
                "0.975]": ci.loc[x, 1] if x in ci.index else float("nan"),
                "R_Squared": res.rsquared,
                "Adj_R_Squared": res.rsquared_adj,
                "F_Stat": res.fvalue,
                "Prob_F_Stat": res.f_pvalue,

            })
    return pd.DataFrame(rows)


# ---- inputs ----
control_vars = [
    "TobinQ", "tenure_years", "gender", "age",
    "log_stock_awards",
    "EBIT_Margin", "shrown_excl_opts_pct", "Leverage",
    "log_option_awards", "log_revt_lag", "log_at_lag"
]

"""
we decide to consider four fixed effect specifications to control time invariant characteristic of each unit(firm or sector)
and time varying macroeconomic factors (year) which may affect both independent and dependent variables, so that the coeffcients
can be interpreted as within-firm or within-sector effects over time.
"""

fe_specs = {
    "FE1: sector":        "C(gsector)",
    "FE2: sector + year": "C(gsector) + C(publish_year)",
    "FE3: firm ":   "C(gvkey) ",
    "FE4: sector + firm": "C(gsector) + C(gvkey)",
    "FE5: year": "C(publish_year)"
}

m3_sameyear_nocl = ols_with_controls_and_fe(
    data=final_data,
    dependent_var="log_transaction_size_usd",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="nonrobust"
)


m3_sameyear_nocl = fe_results_table_nocl(m3_sameyear_nocl)
print("Regression with Controls and Fixed Effects Results on log_transaction_size_usd:, no cluster \n")
display(m3_sameyear_nocl)
```

```
Regression with Controls and Fixed Effects Results on log_transaction_size_usd:, no cluster
```

	FE_spec	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	F_Stat	Prob_F_Stat
0	FE1: sector	SRR_median	-0.045327	0.423328	-0.107074	0.914753	-0.876101	0.785447	0.128621	0.108270	6.320224	1.650321e-17
1	FE1: sector	SRR_mean	0.000926	0.436059	0.002123	0.998306	-0.854834	0.856686	0.128610	0.108259	6.319626	1.658462e-17
2	FE1: sector	normalized_SRR_mean	-17.675723	71.336462	-0.247780	0.804359	-157.672495	122.321048	0.128667	0.108317	6.322829	1.615314e-17
3	FE1: sector	normalized_SRR_median	-17.853952	71.303894	-0.250392	0.802339	-157.786811	122.078906	0.128668	0.108319	6.322897	1.614411e-17
4	FE2: sector + year	SRR_median	0.021914	0.437456	0.050095	0.960058	-0.836605	0.880434	0.146435	0.111408	4.180577	3.853274e-15
5	FE2: sector + year	SRR_mean	0.116297	0.451821	0.257396	0.796930	-0.770414	1.003008	0.146494	0.111469	4.182543	3.758957e-15
6	FE2: sector + year	normalized_SRR_mean	3.167614	72.077478	0.043947	0.964956	-138.286536	144.621764	0.146435	0.111407	4.180559	3.854140e-15
7	FE2: sector + year	normalized_SRR_median	2.578818	72.038100	0.035798	0.971451	-138.798051	143.955688	0.146434	0.111406	4.180539	3.855114e-15
8	FE3: firm	SRR_median	-0.447099	0.561614	-0.796096	0.426235	-1.549667	0.655470	0.425314	0.242137	2.321879	1.597365e-17
9	FE3: firm	SRR_mean	-0.294058	0.578925	-0.507937	0.611651	-1.430612	0.842497	0.425018	0.241748	2.319076	1.780284e-17
10	FE3: firm	normalized_SRR_mean	-46.241982	83.918686	-0.551033	0.581779	-210.992365	118.508400	0.425054	0.241795	2.319416	1.757000e-17
11	FE3: firm	normalized_SRR_median	-46.466175	83.909304	-0.553767	0.579908	-211.198139	118.265789	0.425056	0.241798	2.319439	1.755470e-17
12	FE4: sector + firm	SRR_median	-0.447099	0.561614	-0.796096	0.426235	-1.549667	0.655470	0.425314	0.242137	2.321879	1.597365e-17
13	FE4: sector + firm	SRR_mean	-0.294058	0.578925	-0.507937	0.611651	-1.430612	0.842497	0.425018	0.241748	2.319076	1.780284e-17
14	FE4: sector + firm	normalized_SRR_mean	-46.241982	83.918686	-0.551033	0.581779	-210.992365	118.508400	0.425054	0.241795	2.319416	1.757000e-17
15	FE4: sector + firm	normalized_SRR_median	-46.466175	83.909304	-0.553767	0.579908	-211.198139	118.265789	0.425056	0.241798	2.319439	1.755470e-17
16	FE5: year	SRR_median	0.327269	0.431097	0.759155	0.447951	-0.518759	1.173297	0.126678	0.100553	4.848898	7.900352e-15
17	FE5: year	SRR_mean	0.461195	0.444214	1.038227	0.299432	-0.410575	1.332965	0.127145	0.101034	4.869399	6.444727e-15
18	FE5: year	normalized_SRR_mean	-4.589097	72.058002	-0.063686	0.949234	-146.003047	136.824854	0.126144	0.100003	4.825510	9.965563e-15
19	FE5: year	normalized_SRR_median	-5.329713	72.025318	-0.073998	0.941028	-146.679522	136.020096	0.126145	0.100004	4.825568	9.959824e-15

In [43]:

```
m3 = smf.ols(
    "log_transaction_size_usd ~ normalized_SRR_mean + TobinQ + tenure_years + gender+ age "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage+ log_revt_lag+ log_at_lag+C(gsector)",
    data=final_data).fit()
m3.summary()
```

Out[43]:

OLS Regression Results

Dep. Variable:	log_transaction_size_usd	**R-squared:**	0.124
Model:	OLS	**Adj. R-squared:**	0.105
Method:	Least Squares	**F-statistic:**	6.379
Date:	Fri, 30 Jan 2026	**Prob (F-statistic):**	4.52e-17
Time:	23:13:53	**Log-Likelihood:**	-2258.7
No. Observations:	965	**AIC:**	4561.
Df Residuals:	943	**BIC:**	4669.
Df Model:	21		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-0.6222	1.054	-0.590	0.555	-2.691	1.447
C(gsector)[T.15.0]	-0.1075	0.551	-0.195	0.845	-1.188	0.973
C(gsector)[T.20.0]	0.3052	0.473	0.646	0.519	-0.622	1.233
C(gsector)[T.25.0]	0.4154	0.481	0.863	0.388	-0.529	1.360
C(gsector)[T.30.0]	0.0049	0.510	0.010	0.992	-0.997	1.007
C(gsector)[T.35.0]	0.8995	0.498	1.807	0.071	-0.077	1.876
C(gsector)[T.40.0]	1.3399	0.562	2.384	0.017	0.237	2.443
C(gsector)[T.45.0]	0.6504	0.459	1.417	0.157	-0.250	1.551
C(gsector)[T.50.0]	1.2628	0.516	2.446	0.015	0.250	2.276
C(gsector)[T.55.0]	0.1035	0.743	0.139	0.889	-1.355	1.561
C(gsector)[T.60.0]	0.6685	0.776	0.862	0.389	-0.853	2.191
normalized_SRR_mean	-12.8298	71.438	-0.180	0.858	-153.025	127.366
TobinQ	0.1466	0.058	2.540	0.011	0.033	0.260
tenure_years	0.0070	0.013	0.528	0.597	-0.019	0.033
gender	0.3971	0.351	1.131	0.258	-0.292	1.086
age	0.0134	0.015	0.877	0.381	-0.017	0.043
log_stock_awards	0.0102	0.034	0.302	0.763	-0.056	0.076
EBIT_Margin	0.9954	0.441	2.259	0.024	0.131	1.860
shrown_excl_opts_pct	0.0096	0.026	0.373	0.709	-0.041	0.060
Leverage	0.0824	0.525	0.157	0.875	-0.948	1.113
log_revt_lag	-0.0175	0.152	-0.115	0.909	-0.316	0.281
log_at_lag	0.3793	0.142	2.676	0.008	0.101	0.657

Omnibus:	33.087	**Durbin-Watson:**	1.927
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	16.439
Skew:	-0.093	**Prob(JB):**	0.000269
Kurtosis:	2.388	**Cond. No.**	5.15e+04

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 5.15e+04. This might indicate that there are
strong multicollinearity or other numerical problems.

In [44]:

```
def ols_with_controls_and_fe_clusterd(
    data: pd.DataFrame,
    dependent_var: str,
    independent_vars: list,
    control_vars: list,
    fe_specs: dict,
    cov_type: str = "cluster",     # "nonrobust", "HC1", "HC3", "cluster"
    cluster_col: str | None = None,  # firm id for clustering
    use_correction: bool = True
):
    control_vars = list(dict.fromkeys(control_vars))
    controls_str = " + ".join(control_vars) if control_vars else ""  # <- don't force "1"

    models_fe = {}
    for fe_name, fe_terms in fe_specs.items():
        models_fe[fe_name] = {}
        for x in independent_vars:
            rhs_parts = [x]
            if controls_str:
                rhs_parts.append(controls_str)
            if fe_terms and fe_terms.strip():
                rhs_parts.append(fe_terms)

            formula = f"{dependent_var} ~ " + " + ".join(rhs_parts)

            base = smf.ols(formula, data=data, missing="drop")

            if cov_type == "cluster":
                if not cluster_col:
                    raise ValueError("cov_type='cluster' requires cluster_col (e.g., 'gvkey').")

                used_idx = base.data.row_labels
                groups = data.loc[used_idx, cluster_col]

                res = base.fit(
                    cov_type="cluster",
                    cov_kwds={"groups": groups, "use_correction": use_correction}
                )
            elif cov_type == "nonrobust":
                res = base.fit()
            else:
                res = base.fit(cov_type=cov_type)

            models_fe[fe_name][x] = res

    return models_fe


def fe_results_table(models_fe: dict, data: pd.DataFrame, cluster_col: str | None = None):
    rows = []
    for fe_name, sub in models_fe.items():
        for x, res in sub.items():
            ci = res.conf_int()
            used_idx = res.model.data.row_labels

            n_clusters = np.nan
            if cluster_col is not None:
                n_clusters = int(data.loc[used_idx, cluster_col].nunique())

            rows.append({
                "FE_spec": fe_name,
                "Independent_Variable": x,
                "Coefficient": res.params.get(x, np.nan),
                "Std_Error": res.bse.get(x, np.nan),
                "T_Stat": res.tvalues.get(x, np.nan),
                "P_Value": res.pvalues.get(x, np.nan),
                "[0.025": ci.loc[x, 0] if x in ci.index else np.nan,
                "0.975]": ci.loc[x, 1] if x in ci.index else np.nan,
                "R_Squared": res.rsquared,
                "Adj_R_Squared": res.rsquared_adj,
                "N_Clusters": n_clusters,
                "N_Obs": int(res.nobs),
            })
    return pd.DataFrame(rows)


# ---- run clustered by firm ----

control_vars = [
    "TobinQ", "tenure_years", "gender", "age",
    "log_stock_awards",
    "EBIT_Margin", "shrown_excl_opts_pct", "Leverage",
    "log_option_awards", "log_revt_lag", "log_at_lag"
]

"""
we decide to consider four fixed effect specifications to control time invariant characteristic of each unit(firm or sector)
and time varying macroeconomic factors (year) which may affect both independent and dependent variables, so that the coeffcients
can be interpreted as within-firm or within-sector effects over time.
"""


fe_specs = {
    "FE1: sector":        "C(gsector)",
    "FE2: sector + year": "C(gsector) + C(publish_year)",
    "FE3: firm ":   "C(gvkey) ",
    "FE4: sector + firm": "C(gsector) + C(gvkey)",
    "FE5: year": "C(publish_year)"
}


models_same_year = ols_with_controls_and_fe_clusterd(
    data=final_data,
    dependent_var="log_transaction_size_usd",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="cluster",
    cluster_col="gvkey"
)

m3_sameyear = fe_results_table(models_same_year, data=final_data, cluster_col="gvkey")

print("Regression with Controls and Fixed Effects (clustered by firm gvkey):\n")
display(m3_sameyear)
```

```
Regression with Controls and Fixed Effects (clustered by firm gvkey):
```

	FE_spec	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	N_Clusters	N_Obs
0	FE1: sector	SRR_median	-0.045327	0.430679	-0.105246	0.916181	-0.889442	0.798788	0.128621	0.108270	222	965
1	FE1: sector	SRR_mean	0.000926	0.443336	0.002089	0.998334	-0.867996	0.869848	0.128610	0.108259	222	965
2	FE1: sector	normalized_SRR_mean	-17.675723	32.813193	-0.538677	0.590110	-81.988400	46.636954	0.128667	0.108317	222	965
3	FE1: sector	normalized_SRR_median	-17.853952	32.862525	-0.543292	0.586929	-82.263318	46.555414	0.128668	0.108319	222	965
4	FE2: sector + year	SRR_median	0.021914	0.425172	0.051542	0.958894	-0.811407	0.855235	0.146435	0.111408	222	965
5	FE2: sector + year	SRR_mean	0.116297	0.435664	0.266942	0.789514	-0.737589	0.970183	0.146494	0.111469	222	965
6	FE2: sector + year	normalized_SRR_mean	3.167614	30.464737	0.103976	0.917188	-56.542173	62.877401	0.146435	0.111407	222	965
7	FE2: sector + year	normalized_SRR_median	2.578818	30.560194	0.084385	0.932750	-57.318061	62.475697	0.146434	0.111406	222	965
8	FE3: firm	SRR_median	-0.447099	0.488698	-0.914877	0.360256	-1.404929	0.510731	0.425314	0.242137	222	965
9	FE3: firm	SRR_mean	-0.294058	0.515950	-0.569934	0.568722	-1.305301	0.717186	0.425018	0.241748	222	965
10	FE3: firm	normalized_SRR_mean	-46.241982	41.917482	-1.103167	0.269955	-128.398738	35.914773	0.425054	0.241795	222	965
11	FE3: firm	normalized_SRR_median	-46.466175	41.909133	-1.108736	0.267544	-128.606565	35.674216	0.425056	0.241798	222	965
12	FE4: sector + firm	SRR_median	-0.447099	0.492075	-0.908598	0.363562	-1.411548	0.517351	0.425314	0.242137	222	965
13	FE4: sector + firm	SRR_mean	-0.294058	0.519516	-0.566023	0.571378	-1.312289	0.724174	0.425018	0.241748	222	965
14	FE4: sector + firm	normalized_SRR_mean	-46.241982	42.207171	-1.095595	0.273256	-128.966518	36.482553	0.425054	0.241795	222	965
15	FE4: sector + firm	normalized_SRR_median	-46.466175	42.198764	-1.101126	0.270842	-129.174232	36.241882	0.425056	0.241798	222	965
16	FE5: year	SRR_median	0.327269	0.397967	0.822353	0.410876	-0.452731	1.107270	0.126678	0.100553	222	965
17	FE5: year	SRR_mean	0.461195	0.403829	1.142056	0.253431	-0.330295	1.252684	0.127145	0.101034	222	965
18	FE5: year	normalized_SRR_mean	-4.589097	32.598444	-0.140777	0.888046	-68.480873	59.302679	0.126144	0.100003	222	965
19	FE5: year	normalized_SRR_median	-5.329713	32.758683	-0.162696	0.870758	-69.535551	58.876125	0.126145	0.100004	222	965

3.2 Regression for M&A spending next year

This part is to answer the question: **Does SRR this year is associated to deal size next year?**

Result:

In simple model, SRR_median and SRR_mean is significant, but normalized SRR is not significant at 5% level (no clustering). While SRR and normalized_SRR are statistically significant (p_value < 0.05). Higher SRR today is associated with larger log deal size next year (cov_type: clustered)

In [45]:

```
import statsmodels.api as sm

X = final_data[["SRR_mean"]]
y = final_data["log_transaction_size_usd_nextyear"]
X = sm.add_constant(X)
model = sm.OLS(y, X , missing="drop").fit()
print(model.summary())
```

```
                                    OLS Regression Results
=============================================================================================
Dep. Variable:     log_transaction_size_usd_nextyear   R-squared:                       0.010
Model:                                           OLS   Adj. R-squared:                  0.009
Method:                                Least Squares   F-statistic:                     9.497
Date:                               Fri, 30 Jan 2026   Prob (F-statistic):            0.00212
Time:                                       23:13:53   Log-Likelihood:                -2318.3
No. Observations:                                965   AIC:                             4641.
Df Residuals:                                    963   BIC:                             4650.
Df Model:                                          1
Covariance Type:                           nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          4.5348      0.162     28.067      0.000       4.218       4.852
SRR_mean       1.3276      0.431      3.082      0.002       0.482       2.173
==============================================================================
Omnibus:                       40.762   Durbin-Watson:                   1.742
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               17.665
Skew:                          -0.007   Prob(JB):                     0.000146
Kurtosis:                       2.337   Cond. No.                         5.52
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [46]:

```
m1_nextyear_nocl = simple_ols(
    final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"]
)

print("Simple regression on log_transaction_size_usd_nextyear, nonrobust: \n")

display(m1_nextyear_nocl)
```

```
Simple regression on log_transaction_size_usd_nextyear, nonrobust:
```

model	coef	std_err	p_value	t_stat	R_squared	adj_R_squared	n
SRR_median	1.236708	0.419748	0.003293	2.946311	0.008934	0.007905	965
SRR_mean	1.327601	0.430803	0.002117	3.081691	0.009765	0.008737	965
normalized_SRR_mean	119.290238	74.407334	0.109217	1.603205	0.002662	0.001626	965
normalized_SRR_median	119.768814	74.377687	0.107665	1.610279	0.002685	0.001650	965

In [47]:

```
# ------- inputs -----------------
m1_nextyear = simple_ols_clustered(
    final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"]
)

print("\n Simple regression on log_transaction_size_usd_nextyear, firm-clustered SE (gvkey): \n")
display(m1_nextyear)
```

```
 Simple regression on log_transaction_size_usd_nextyear, firm-clustered SE (gvkey):
```

model	coef	std_err	p_value	t_stat	R_squared	adj_R_squared	n_obs	n_clusters
SRR_median	1.236708	0.497381	0.012903	2.486443	0.008934	0.007905	965	222
SRR_mean	1.327601	0.508296	0.009005	2.611864	0.009765	0.008737	965	222
normalized_SRR_mean	119.290238	59.052315	0.043375	2.020077	0.002662	0.001626	965	222
normalized_SRR_median	119.768814	59.169189	0.042952	2.024175	0.002685	0.001650	965	222

In [48]:

```
#Regression with basic controls

m2_nextyear_nocl, models_basic = ols_with_controls(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
)

print("Regression with Controls Results on log_transaction_size_usd_nextyear with Controls:, no clustering\n")
display(m2_nextyear_nocl)
```

```
Regression with Controls Results on log_transaction_size_usd_nextyear with Controls:, no clustering
```

	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	F_Stat	Prob_F_Stat
0	SRR_median	0.937836	0.419153	2.237457	0.025487	0.115266	1.760406	0.092539	0.081101	8.090091	1.337219e-14
1	SRR_mean	0.964728	0.430846	2.239147	0.025377	0.119210	1.810245	0.092546	0.081108	8.090783	1.332699e-14
2	normalized_SRR_mean	165.887062	71.950863	2.305560	0.021349	24.686445	307.087679	0.092832	0.081398	8.118355	1.164358e-14
3	normalized_SRR_median	166.735987	71.922687	2.318267	0.020646	25.590664	307.881309	0.092888	0.081454	8.123722	1.134150e-14

In [49]:

```
#Regression with basic controls

m2_nextyear, models_basic = ols_with_controls_clustered(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
)

print("Regression with Controls Results on log_transaction_size_usd_nextyear with Controls, firm-clustered SE (gvkey)::\n")
display(m2_nextyear)
```

```
Regression with Controls Results on log_transaction_size_usd_nextyear with Controls, firm-clustered SE (gvkey)::
```

	Independent_Variable	coef	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	N_Clusters
0	SRR_median	0.937836	0.420967	2.227812	0.025893	0.112755	1.762917	0.092539	0.081101	222
1	SRR_mean	0.964728	0.429879	2.244185	0.024821	0.122180	1.807275	0.092546	0.081108	222
2	normalized_SRR_mean	165.887062	69.157706	2.398678	0.016454	30.340449	301.433675	0.092832	0.081398	222
3	normalized_SRR_median	166.735987	69.295991	2.406142	0.016122	30.918340	302.553634	0.092888	0.081454	222

Raw SRR and normalized SRR are statistically significant But SRR in same_year regression was not statistically significant in the same model.

One possible hypothesis is that SRR has a lagged effect. It takes time to turn into a boardroom decision for a giant merger.

In [50]:

```
m2 = smf.ols(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_mean + TobinQ + tenure_years + gender+ age "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage+ log_revt_lag+ log_at_lag",
    data=final_data).fit()
m2.summary()
```

Out[50]:

OLS Regression Results

Dep. Variable:	log_transaction_size_usd_nextyear	**R-squared:**	0.089
Model:	OLS	**Adj. R-squared:**	0.078
Method:	Least Squares	**F-statistic:**	8.442
Date:	Fri, 30 Jan 2026	**Prob (F-statistic):**	2.58e-14
Time:	23:13:54	**Log-Likelihood:**	-2278.2
No. Observations:	965	**AIC:**	4580.
Df Residuals:	953	**BIC:**	4639.
Df Model:	11		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.6213	0.918	0.677	0.499	-1.180	2.422
normalized_SRR_mean	168.8879	72.058	2.344	0.019	27.477	310.299
TobinQ	0.1332	0.055	2.417	0.016	0.025	0.241
tenure_years	0.0066	0.013	0.513	0.608	-0.019	0.032
gender	0.4190	0.347	1.209	0.227	-0.261	1.099
age	-0.0027	0.015	-0.180	0.857	-0.033	0.027
log_stock_awards	-0.0092	0.034	-0.275	0.784	-0.075	0.057
EBIT_Margin	0.9462	0.432	2.189	0.029	0.098	1.795
shrown_excl_opts_pct	0.0245	0.025	0.979	0.328	-0.025	0.073
Leverage	-0.2047	0.466	-0.439	0.661	-1.120	0.710
log_revt_lag	-0.1093	0.111	-0.985	0.325	-0.327	0.108
log_at_lag	0.5114	0.100	5.128	0.000	0.316	0.707

Omnibus:	35.429	**Durbin-Watson:**	1.855
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	18.145
Skew:	-0.133	**Prob(JB):**	0.000115
Kurtosis:	2.383	**Cond. No.**	5.12e+04

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 5.12e+04. This might indicate that there are
strong multicollinearity or other numerical problems.

In [51]:

```
m2 = smf.ols(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_median + TobinQ + tenure_years + gender+ age "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage+ log_revt_lag+ log_at_lag",
    data=final_data).fit()
m2.summary()
```

Out[51]:

OLS Regression Results

Dep. Variable:	log_transaction_size_usd_nextyear	**R-squared:**	0.089
Model:	OLS	**Adj. R-squared:**	0.078
Method:	Least Squares	**F-statistic:**	8.448
Date:	Fri, 30 Jan 2026	**Prob (F-statistic):**	2.52e-14
Time:	23:13:54	**Log-Likelihood:**	-2278.1
No. Observations:	965	**AIC:**	4580.
Df Residuals:	953	**BIC:**	4639.
Df Model:	11		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.6175	0.918	0.673	0.501	-1.184	2.419
normalized_SRR_median	169.7422	72.030	2.357	0.019	28.386	311.098
TobinQ	0.1332	0.055	2.418	0.016	0.025	0.241
tenure_years	0.0066	0.013	0.513	0.608	-0.019	0.032
gender	0.4197	0.347	1.211	0.226	-0.260	1.100
age	-0.0027	0.015	-0.178	0.859	-0.033	0.027
log_stock_awards	-0.0092	0.034	-0.274	0.784	-0.075	0.057
EBIT_Margin	0.9455	0.432	2.187	0.029	0.097	1.794
shrown_excl_opts_pct	0.0244	0.025	0.978	0.328	-0.025	0.073
Leverage	-0.2044	0.466	-0.438	0.661	-1.119	0.711
log_revt_lag	-0.1092	0.111	-0.985	0.325	-0.327	0.108
log_at_lag	0.5115	0.100	5.129	0.000	0.316	0.707

Omnibus:	35.431	**Durbin-Watson:**	1.855
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	18.145
Skew:	-0.133	**Prob(JB):**	0.000115
Kurtosis:	2.383	**Cond. No.**	5.12e+04

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 5.12e+04. This might indicate that there are
strong multicollinearity or other numerical problems.

In [52]:

```
formula = (
    "log_transaction_size_usd_nextyear ~ normalized_SRR_median + TobinQ + tenure_years + gender + age + "
    "log_stock_awards + EBIT_Margin + shrown_excl_opts_pct + Leverage + "
    "log_option_awards + log_revt_lag + log_at_lag"
)

base = smf.ols(formula, data=final_data, missing="drop")
used_idx = base.data.row_labels
groups = final_data.loc[used_idx, "gvkey"]

m2 = base.fit(
    cov_type="cluster",
    cov_kwds={"groups": groups, "use_correction": True}
)

print(m2.summary())
```

```
                                    OLS Regression Results
=============================================================================================
Dep. Variable:     log_transaction_size_usd_nextyear   R-squared:                       0.093
Model:                                           OLS   Adj. R-squared:                  0.081
Method:                                Least Squares   F-statistic:                     5.295
Date:                               Fri, 30 Jan 2026   Prob (F-statistic):           7.65e-08
Time:                                       23:13:54   Log-Likelihood:                -2276.0
No. Observations:                                965   AIC:                             4578.
Df Residuals:                                    952   BIC:                             4641.
Df Model:                                         12
Covariance Type:                             cluster
=========================================================================================
                            coef    std err          z      P>|z|      [0.025      0.975]
-----------------------------------------------------------------------------------------
Intercept                 0.6464      1.017      0.636      0.525      -1.347       2.640
normalized_SRR_median   166.7360     69.296      2.406      0.016      30.918     302.554
TobinQ                    0.1424      0.059      2.411      0.016       0.027       0.258
tenure_years              0.0009      0.020      0.046      0.963      -0.039       0.040
gender                    0.3701      0.295      1.256      0.209      -0.207       0.948
age                      -0.0003      0.016     -0.017      0.986      -0.032       0.031
log_stock_awards         -0.0122      0.040     -0.302      0.763      -0.091       0.067
EBIT_Margin               0.9859      0.409      2.410      0.016       0.184       1.788
shrown_excl_opts_pct      0.0292      0.030      0.974      0.330      -0.030       0.088
Leverage                 -0.2165      0.586     -0.369      0.712      -1.366       0.933
log_option_awards        -0.0434      0.023     -1.876      0.061      -0.089       0.002
log_revt_lag             -0.0633      0.163     -0.388      0.698      -0.383       0.256
log_at_lag                0.4736      0.148      3.200      0.001       0.184       0.764
==============================================================================
Omnibus:                       33.453   Durbin-Watson:                   1.853
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               17.233
Skew:                          -0.122   Prob(JB):                     0.000181
Kurtosis:                       2.392   Cond. No.                     5.13e+04
==============================================================================

Notes:
[1] Standard Errors are robust to cluster correlation (cluster)
[2] The condition number is large, 5.13e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

normalized_SRR_median is positive and statistically significant at the 5% level

This supports our hypothesis, executives with higher SRR today tend to be associated with larger deal sizes next year.

Other significant controls:

- **Tobin's Q**: High-growth firms spend more on deals. A Tobin's Q above 1 signifies that the market values the firm's assets more than their replacement cost (indicating growth potential). The result shows that even when controlling for size and profit, the market's optimism about a company's future predicts larger deal sizes.
- **EBIT Margin**: Higher profitability leads to larger deals (p_value = 0.023 and coef 0.82). For every unit increase in margin, deal size sees a healthy bump. Firms don't just need to be big; they need to be efficient to justify larger deals.
- **Assets**: Firm size is a primary driver of deal scale. This is the most reliable predictor. Since the variable is logged, this suggests a power law relationship: as a firm's total assets increase, their deal capacity grows substantially. Because larger firms have the balance sheets to support bigger tickets.

Controls that are not significant:

- **tenure_years**: Whether a CEO has been there for 2 years or 20, the deal size is likely dictated by the firm's strategic needs and balance sheet capacity rather than the CEO's personal experience or "empire-building" phase.
- **Gender**: does not play a role in the magnitude of the transaction. This is consistent with several modern studies which find that while gender might influence the frequency or risk profile of deals, it doesn't fundamentally change the size of the check once a deal is approved.
- **shrown_excl_opts_pct** measures the percentage of the company held by shareholders (excluding options). Because two effects "tight leash on spending" and "mandate for aggressive growth" often cancel each other out, the variable can end up non-significant.
- **Leverage** perhaps is the most interesting non-significant result Highly leveraged firms might still pull off massive deals if they can secure bridge financing or use equity as a "currency" (stock-for-stock deals).It implies that a firm's current debt level does not act as a "ceiling" for next year's deal size.

To ensure the validity of our primary findings, we controlled for CEO characteristics (Tenure, Gender) and Capital structures (Leverage, Ownership Concentration). These factors were not found to have a statistically significant impact on next-year deal size, indicating that the scale of M&A activity is primarily driven by firm-level economic fundamentals (Assets, Profitability, and Tobin's Q) rather than individual or capital-structure constraints.

In [53]:

```
# Fixed Effects Regression Model

# convert gvkey, gsector to categorical type
final_data["gvkey"] = final_data["gvkey"].astype("category")
final_data["gsector"] = final_data["gsector"].astype("category")
# Convert publish_year, gender to float
final_data["publish_year"] = final_data["publish_year"].astype(float)
final_data["gender"] = final_data["gender"].astype(float)



m3_nextyear_nocl = ols_with_controls_and_fe(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="nonrobust"
)


results_m3 = fe_results_table_nocl(m3_nextyear_nocl)
print("Regression with Controls and Fixed Effects Results on log_transaction_size_usd_nextyear:\n")
display(results_m3)
```

```
Regression with Controls and Fixed Effects Results on log_transaction_size_usd_nextyear:
```

	FE_spec	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	F_Stat	Prob_F_Stat
0	FE1: sector	SRR_median	0.657448	0.427284	1.538669	0.124221	-0.181090	1.495987	0.112743	0.092022	5.440885	2.241001e-14
1	FE1: sector	SRR_mean	0.640093	0.440191	1.454126	0.146245	-0.223775	1.503961	0.112505	0.091778	5.427955	2.490479e-14

2	FE1: sector	normalized_SRR_mean	172.317030	71.876537	2.397403	0.016706	31.260368	313.373693	0.115908	0.095260	5.613612	5.461195e-15
3	FE1: sector	normalized_SRR_median	173.368918	71.840886	2.413235	0.016002	32.382219	314.355616	0.115979	0.095333	5.617504	5.290006e-15
4	FE2: sector + year	SRR_median	0.709219	0.443496	1.599157	0.110127	-0.161154	1.579593	0.123181	0.087200	3.423438	4.638564e-11
5	FE2: sector + year	SRR_mean	0.705351	0.458121	1.539662	0.123984	-0.193724	1.604427	0.123005	0.087016	3.417848	4.964575e-11
6	FE2: sector + year	normalized_SRR_mean	175.404662	72.946104	2.404579	0.016387	32.245807	318.563516	0.126216	0.090359	3.519953	1.429721e-11
7	FE2: sector + year	normalized_SRR_median	176.367161	72.903469	2.419187	0.015747	33.291979	319.442343	0.126282	0.090427	3.522062	1.393303e-11
8	FE3: firm	SRR_median	0.269903	0.569238	0.474147	0.635536	-0.847634	1.387439	0.409927	0.221847	2.179531	3.715197e-15
9	FE3: firm	SRR_mean	0.329885	0.586597	0.562370	0.574036	-0.821731	1.481500	0.410001	0.221944	2.180195	3.622877e-15
10	FE3: firm	normalized_SRR_mean	171.305315	84.815470	2.019741	0.043775	4.794354	337.816276	0.413022	0.225927	2.207558	1.282987e-15
11	FE3: firm	normalized_SRR_median	171.410135	84.805821	2.021207	0.043622	4.918115	337.902154	0.413026	0.225933	2.207601	1.280888e-15
12	FE4: sector + firm	SRR_median	0.269903	0.569238	0.474147	0.635536	-0.847634	1.387439	0.409927	0.221847	2.179531	3.715197e-15
13	FE4: sector + firm	SRR_mean	0.329885	0.586597	0.562370	0.574036	-0.821731	1.481500	0.410001	0.221944	2.180195	3.622877e-15
14	FE4: sector + firm	normalized_SRR_mean	171.305315	84.815470	2.019741	0.043775	4.794354	337.816276	0.413022	0.225927	2.207558	1.282987e-15
15	FE4: sector + firm	normalized_SRR_median	171.410135	84.805821	2.021207	0.043622	4.918115	337.902154	0.413026	0.225933	2.207601	1.280888e-15
16	FE5: year	SRR_median	0.992681	0.437069	2.271223	0.023360	0.134933	1.850429	0.102804	0.075965	3.830359	1.686592e-10
17	FE5: year	SRR_mean	1.036224	0.450456	2.300387	0.021645	0.152203	1.920245	0.102931	0.076095	3.835638	1.603300e-10
18	FE5: year	normalized_SRR_mean	165.682358	73.034411	2.268552	0.023523	22.352204	309.012512	0.102792	0.075953	3.829879	1.694378e-10
19	FE5: year	normalized_SRR_median	166.483325	72.999211	2.280618	0.022795	23.222251	309.744399	0.102845	0.076007	3.832052	1.659417e-10

In [54]:

```
m3_nextyear_clustered = ols_with_controls_and_fe_clusterd(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="cluster",
    cluster_col="gvkey"
)

m3_nextyear_clustered = fe_results_table(m3_nextyear_clustered, data=final_data, cluster_col="gvkey")

print("Regression with Controls and Fixed Effects (clustered by firm gvkey):\n")
display(m3_nextyear_clustered)
```

```
Regression with Controls and Fixed Effects (clustered by firm gvkey):
```

	FE_spec	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	N_Clusters	N_Obs
0	FE1: sector	SRR_median	0.657448	0.437373	1.503175	0.132794	-0.199787	1.514684	0.112743	0.092022	222	965
1	FE1: sector	SRR_mean	0.640093	0.455255	1.406010	0.159721	-0.252190	1.532376	0.112505	0.091778	222	965
2	FE1: sector	normalized_SRR_mean	172.317030	61.952161	2.781453	0.005412	50.893026	293.741035	0.115908	0.095260	222	965
3	FE1: sector	normalized_SRR_median	173.368918	62.141144	2.789922	0.005272	51.574514	295.163321	0.115979	0.095333	222	965
4	FE2: sector + year	SRR_median	0.709219	0.432620	1.639360	0.101138	-0.138700	1.557138	0.123181	0.087200	222	965
5	FE2: sector + year	SRR_mean	0.705351	0.455450	1.548693	0.121456	-0.187313	1.598016	0.123005	0.087016	222	965
6	FE2: sector + year	normalized_SRR_mean	175.404662	62.926814	2.787439	0.005313	52.070373	298.738951	0.126216	0.090359	222	965
7	FE2: sector + year	normalized_SRR_median	176.367161	63.112424	2.794492	0.005198	52.669084	300.065238	0.126282	0.090427	222	965
8	FE3: firm	SRR_median	0.269903	0.637434	0.423421	0.671988	-0.979445	1.519250	0.409927	0.221847	222	965
9	FE3: firm	SRR_mean	0.329885	0.660269	0.499622	0.617342	-0.964219	1.623988	0.410001	0.221944	222	965
10	FE3: firm	normalized_SRR_mean	171.305315	96.941310	1.767103	0.077211	-18.696162	361.306792	0.413022	0.225927	222	965
11	FE3: firm	normalized_SRR_median	171.410135	96.965549	1.767743	0.077104	-18.638850	361.459119	0.413026	0.225933	222	965
12	FE4: sector + firm	SRR_median	0.269903	0.641839	0.420515	0.674109	-0.988079	1.527884	0.409927	0.221847	222	965
13	FE4: sector + firm	SRR_mean	0.329885	0.664832	0.496192	0.619759	-0.973162	1.632931	0.410001	0.221944	222	965
14	FE4: sector + firm	normalized_SRR_mean	171.305315	97.611265	1.754975	0.079264	-20.009249	362.619879	0.413022	0.225927	222	965
15	FE4: sector + firm	normalized_SRR_median	171.410135	97.635672	1.755610	0.079155	-19.952265	362.772534	0.413026	0.225933	222	965
16	FE5: year	SRR_median	0.992681	0.420627	2.360002	0.018275	0.168267	1.817095	0.102804	0.075965	222	965
17	FE5: year	SRR_mean	1.036224	0.435375	2.380070	0.017309	0.182904	1.889544	0.102931	0.076095	222	965
18	FE5: year	normalized_SRR_mean	165.682358	70.134503	2.362352	0.018159	28.221257	303.143458	0.102792	0.075953	222	965
19	FE5: year	normalized_SRR_median	166.483325	70.258864	2.369570	0.017809	28.778482	304.188169	0.102845	0.076007	222	965

- normalized_SRR_mean and normalized_SRR_median is significant when we add controls for sector effects (FE1) and sector + year effects (FE2) and sector + firm effects (FE4) or firm effect (FE3) at 10% level significance

In [55]:

```
# 1) keep the models dict
m3_FE1_nextyear = ols_with_controls_and_fe_clusterd(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="nonrobust"
)


# 2) print the summary for FE1 sector and normalized_SRR_median

print(m3_FE1_nextyear["FE1: sector"]["normalized_SRR_median"].summary())
```

```
                                  OLS Regression Results
=========================================================================================
Dep. Variable:     log_transaction_size_usd_nextyear   R-squared:                       0.116
Model:                                           OLS   Adj. R-squared:                  0.095
Method:                                Least Squares   F-statistic:                     5.618
Date:                               Fri, 30 Jan 2026   Prob (F-statistic):           5.29e-15
Time:                                       23:13:55   Log-Likelihood:                -2263.6
No. Observations:                                965   AIC:                             4573.
Df Residuals:                                    942   BIC:                             4685.
Df Model:                                         22
Covariance Type:                           nonrobust
=========================================================================================
                            coef    std err          t      P>|t|      [0.025      0.975]
-----------------------------------------------------------------------------------------
Intercept                 0.2370      1.061      0.223      0.823      -1.846       2.320
C(gsector)[T.15.0]       -0.0178      0.558     -0.032      0.975      -1.113       1.078
C(gsector)[T.20.0]        0.1209      0.475      0.254      0.799      -0.812       1.054
C(gsector)[T.25.0]        0.1154      0.484      0.238      0.812      -0.835       1.065
C(gsector)[T.30.0]       -0.2391      0.513     -0.466      0.642      -1.246       0.768
C(gsector)[T.35.0]        0.7938      0.501      1.583      0.114      -0.190       1.778
C(gsector)[T.40.0]        1.2360      0.571      2.165      0.031       0.116       2.356
C(gsector)[T.45.0]        0.6587      0.463      1.423      0.155      -0.250       1.567
C(gsector)[T.50.0]        1.2999      0.520      2.498      0.013       0.278       2.321
C(gsector)[T.55.0]       -0.4347      0.751     -0.579      0.563      -1.908       1.039
C(gsector)[T.60.0]        0.7114      0.787      0.904      0.366      -0.833       2.256
normalized_SRR_median   173.3689     71.841      2.413      0.016      32.382     314.356
TobinQ                    0.1279      0.058      2.201      0.028       0.014       0.242
tenure_years              0.0037      0.014      0.268      0.788      -0.023       0.031
gender                    0.3188      0.354      0.901      0.368      -0.376       1.013
age                       0.0023      0.015      0.147      0.883      -0.028       0.033
log_stock_awards         -0.0049      0.034     -0.144      0.885      -0.072       0.062
EBIT_Margin               0.7350      0.445      1.652      0.099      -0.138       1.608
shrown_excl_opts_pct      0.0148      0.026      0.566      0.571      -0.036       0.066
Leverage                  0.3667      0.530      0.692      0.489      -0.673       1.406
log_option_awards        -0.0385      0.022     -1.750      0.080      -0.082       0.005
log_revt_lag              0.1659      0.153      1.083      0.279      -0.135       0.466
log_at_lag                0.2205      0.143      1.547      0.122      -0.059       0.500
==============================================================================
Omnibus:                       31.226   Durbin-Watson:                   1.904
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               16.125
Skew:                          -0.106   Prob(JB):                     0.000315
Kurtosis:                       2.403   Cond. No.                     5.16e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 5.16e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

In [56]:

```
# 1) keep the models dict
m3_FE1_nextyear = ols_with_controls_and_fe_clusterd(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="cluster",
    cluster_col="gvkey"
)


# 2) print the summary for FE1 sector and normalized_SRR_median

print(m3_FE1_nextyear["FE1: sector"]["normalized_SRR_median"].summary())
```

```
                                  OLS Regression Results
=========================================================================================
Dep. Variable:     log_transaction_size_usd_nextyear   R-squared:                       0.116
Model:                                           OLS   Adj. R-squared:                  0.095
Method:                                Least Squares   F-statistic:                     4.153
Date:                               Fri, 30 Jan 2026   Prob (F-statistic):           1.75e-08
Time:                                       23:13:55   Log-Likelihood:                -2263.6
No. Observations:                                965   AIC:                             4573.
Df Residuals:                                    942   BIC:                             4685.
Df Model:                                         22
Covariance Type:                             cluster
=========================================================================================
                            coef    std err          z      P>|z|      [0.025      0.975]
-----------------------------------------------------------------------------------------
Intercept                 0.2370      1.143      0.207      0.836      -2.004       2.478
C(gsector)[T.15.0]       -0.0178      0.464     -0.038      0.969      -0.927       0.891
C(gsector)[T.20.0]        0.1209      0.480      0.252      0.801      -0.819       1.061
C(gsector)[T.25.0]        0.1154      0.469      0.246      0.806      -0.804       1.035
C(gsector)[T.30.0]       -0.2391      0.457     -0.523      0.601      -1.135       0.657
C(gsector)[T.35.0]        0.7938      0.522      1.520      0.128      -0.229       1.817
C(gsector)[T.40.0]        1.2360      0.590      2.093      0.036       0.079       2.393
C(gsector)[T.45.0]        0.6587      0.456      1.444      0.149      -0.236       1.553
C(gsector)[T.50.0]        1.2999      0.411      3.163      0.002       0.494       2.105
C(gsector)[T.55.0]       -0.4347      0.705     -0.616      0.538      -1.817       0.948
C(gsector)[T.60.0]        0.7114      0.815      0.873      0.383      -0.886       2.309
normalized_SRR_median   173.3689     62.141      2.790      0.005      51.575     295.163
TobinQ                    0.1279      0.056      2.298      0.022       0.019       0.237
tenure_years              0.0037      0.019      0.197      0.844      -0.033       0.040
gender                    0.3188      0.294      1.085      0.278      -0.257       0.895
age                       0.0023      0.016      0.140      0.889      -0.030       0.034
log_stock_awards         -0.0049      0.040     -0.122      0.903      -0.084       0.074
EBIT_Margin               0.7350      0.416      1.768      0.077      -0.080       1.550
shrown_excl_opts_pct      0.0148      0.029      0.504      0.614      -0.043       0.072
Leverage                  0.3667      0.600      0.611      0.541      -0.809       1.542
log_option_awards        -0.0385      0.023     -1.670      0.095      -0.084       0.007
log_revt_lag              0.1659      0.203      0.819      0.413      -0.231       0.563
log_at_lag                0.2205      0.183      1.205      0.228      -0.138       0.579
==============================================================================
Omnibus:                       31.226   Durbin-Watson:                   1.904
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               16.125
Skew:                          -0.106   Prob(JB):                     0.000315
Kurtosis:                       2.403   Cond. No.                     5.16e+04
==============================================================================

Notes:
[1] Standard Errors are robust to cluster correlation (cluster)
[2] The condition number is large, 5.16e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Only Financials (gsector = 40) and Communication Service (gsector = 50) have significant positive coefficient. This is probably because financial firms are more likely to engage in M&A activities and their SRR scores may reflect their strategic communication related to such activities.

In [57]:

```
sector_names
```

Out[57]:

```
{10: 'Energy',
 15: 'Materials',
 20: 'Industrials',
 25: 'Commercial Discretionary',
 30: 'Consumer Staples',
 35: 'Healthcare',
 40: 'Financials',
 45: 'IT',
 50: 'Communication Service',
 55: 'Utilities',
 60: 'Real Estate'}
```

In [58]:

```
def fit_clustered(formula, data, cluster_col="gvkey"):
    base = smf.ols(formula, data=data, missing="drop")
    used_idx = base.data.row_labels                 # rows actually used in estimation
    groups = data.loc[used_idx, cluster_col]
    return base.fit(cov_type="cluster", cov_kwds={"groups": groups, "use_correction": True})

# 1. Simple Model (clustered)
m1 = fit_clustered("log_transaction_size_usd_nextyear ~ normalized_SRR_mean", final_data)

# 2. Controlled Model (clustered)
m2 = fit_clustered(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_mean + TobinQ + tenure_years + gender "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage + log_option_awards + log_revt_lag + log_at_lag",
    final_data
)

# 3. Fixed Effects Model (clustered)
m3 = fit_clustered(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_mean + TobinQ + tenure_years + gender "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage + log_option_awards + log_revt_lag + log_at_lag + C(gsector)",
    final_data
)

# Compare coefficients + clustered SEs + p-values
print("\n==========================================")
print(f"1. Simple:        coef={m1.params['normalized_SRR_mean']:.3f}  se={m1.bse['normalized_SRR_mean']:.3f}  p={m1.pvalues['normalized_SRR_mean']:.3g}")
print(f"2. + Controls:    coef={m2.params['normalized_SRR_mean']:.3f}  se={m2.bse['normalized_SRR_mean']:.3f}  p={m2.pvalues['normalized_SRR_mean']:.3g}")
print(f"3. + Sector FE:   coef={m3.params['normalized_SRR_mean']:.3f}  se={m3.bse['normalized_SRR_mean']:.3f}  p={m3.pvalues['normalized_SRR_mean']:.3g}")
print("------------------------------------------")
print("==========================================")
```

```
==========================================
1. Simple:        coef=119.290  se=59.052  p=0.0434
2. + Controls:    coef=165.831  se=68.988  p=0.0162
3. + Sector FE:   coef=172.698  se=61.891  p=0.00526
------------------------------------------
==========================================
```

In [59]:

```
def fit_clustered(formula, data, cluster_col="gvkey"):
    base = smf.ols(formula, data=data, missing="drop")
    used_idx = base.data.row_labels  # rows actually used in estimation
    groups = data.loc[used_idx, cluster_col]
    return base.fit(cov_type="cluster", cov_kwds={"groups": groups, "use_correction": True})

# 1. Simple Model (clustered)
m1 = fit_clustered(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_median",
    final_data
)

# 2. Controlled Model (clustered)
m2 = fit_clustered(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_median + TobinQ + tenure_years + gender "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage + log_option_awards + log_revt_lag + log_at_lag",
    final_data
)

# 3. Fixed Effects Model (clustered)
m3 = fit_clustered(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_median + TobinQ + tenure_years + gender "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage + log_option_awards + log_revt_lag + log_at_lag + C(gsector)",
    final_data
)

# Compare coefficients + clustered SEs + p-values
print("\n==========================================")
print(f"1. Simple: coef={m1.params['normalized_SRR_median']:.3f} "
      f"   se={m1.bse['normalized_SRR_median']:.3f} "
      f"  p={m1.pvalues['normalized_SRR_median']:.3g}")
print(f"2. + Controls: coef={m2.params['normalized_SRR_median']:.3f} "
      f"  se={m2.bse['normalized_SRR_median']:.3f} "
      f"  p={m2.pvalues['normalized_SRR_median']:.3g}")
print(f"3. + Sector FE: coef={m3.params['normalized_SRR_median']:.3f} "
      f"  se={m3.bse['normalized_SRR_median']:.3f} "
      f"  p={m3.pvalues['normalized_SRR_median']:.3g}")
print("------------------------------------------")
print("==========================================")
```

```
==========================================
1. Simple: coef=119.769    se=59.169   p=0.043
2. + Controls: coef=166.688   se=69.135   p=0.0159
3. + Sector FE: coef=173.743   se=62.073   p=0.00513
------------------------------------------
==========================================
```

Normalized SRR mean and median is significant in 3 case at 5% level sigificance

3.3 Regression with Interation term

This part is to answer: **How the SRR effect differs in sector X relative to a base sector Y (Energy) ?**

In [60]:

```
#Current year Model with Interaction Term
# map codes -> names (as categorical so statsmodels prints nice levels)
final_data = final_data.copy()
final_data["gsector_name"] = final_data["gsector"].map(sector_names).astype("category")

m_pool = smf.ols(
    "log_transaction_size_usd ~ normalized_SRR_median * C(gsector_name) + TobinQ + tenure_years + gender "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage + log_option_awards + log_revt_lag + log_at_lag",
    data=final_data
).fit()
m_pool.summary()
```

Out[60]:

OLS Regression Results

Dep. Variable:	log_transaction_size_usd	**R-squared:**	0.130
Model:	OLS	**Adj. R-squared:**	0.101
Method:	Least Squares	**F-statistic:**	4.502
Date:	Fri, 30 Jan 2026	**Prob (F-statistic):**	1.96e-14
Time:	23:13:55	**Log-Likelihood:**	-2255.5
No. Observations:	965	**AIC:**	4575.
Df Residuals:	933	**BIC:**	4731.
Df Model:	31		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.1516	0.879	0.172	0.863	-1.574	1.877
C(gsector_name)[T.Materials]	-0.1909	0.595	-0.321	0.748	-1.358	0.976
C(gsector_name)[T.Industrials]	0.2769	0.499	0.554	0.579	-0.703	1.257
C(gsector_name)[T.Commercial Discretionary]	0.4487	0.500	0.898	0.369	-0.532	1.430
C(gsector_name)[T.Consumer Staples]	-0.1462	0.562	-0.260	0.795	-1.249	0.956
C(gsector_name)[T.Healthcare]	1.0742	0.532	2.019	0.044	0.030	2.118
C(gsector_name)[T.Financials]	1.3179	0.612	2.154	0.032	0.117	2.519
C(gsector_name)[T.IT]	0.5610	0.487	1.151	0.250	-0.396	1.518
C(gsector_name)[T.Communication Service]	1.3263	0.569	2.331	0.020	0.210	2.443
C(gsector_name)[T.Utilities]	-0.2022	1.070	-0.189	0.850	-2.302	1.897
C(gsector_name)[T.Real Estate]	0.8656	1.135	0.763	0.446	-1.361	3.092
normalized_SRR_median	6.9553	173.279	0.040	0.968	-333.106	347.017
normalized_SRR_median:C(gsector_name)[T.Materials]	8.8071	523.337	0.017	0.987	-1018.246	1035.860
normalized_SRR_median:C(gsector_name)[T.Industrials]	59.1163	299.617	0.197	0.844	-528.886	647.118
normalized_SRR_median:C(gsector_name)[T.Commercial Discretionary]	-27.0577	197.951	-0.137	0.891	-415.538	361.422
normalized_SRR_median:C(gsector_name)[T.Consumer Staples]	665.7225	799.716	0.832	0.405	-903.729	2235.174
normalized_SRR_median:C(gsector_name)[T.Healthcare]	-284.5174	348.854	-0.816	0.415	-969.147	400.112
normalized_SRR_median:C(gsector_name)[T.Financials]	-145.3779	401.641	-0.362	0.717	-933.602	642.846
normalized_SRR_median:C(gsector_name)[T.IT]	131.9512	363.636	0.363	0.717	-581.689	845.591
normalized_SRR_median:C(gsector_name)[T.Communication Service]	-1026.0331	1523.515	-0.673	0.501	-4015.946	1963.880
normalized_SRR_median:C(gsector_name)[T.Utilities]	818.6222	3515.223	0.233	0.816	-6080.038	7717.282
normalized_SRR_median:C(gsector_name)[T.Real Estate]	-891.7859	1710.460	-0.521	0.602	-4248.580	2465.009
TobinQ	0.1453	0.058	2.502	0.013	0.031	0.259
tenure_years	0.0056	0.012	0.459	0.646	-0.019	0.030
gender	0.2324	0.355	0.654	0.513	-0.465	0.930
log_stock_awards	0.0097	0.034	0.289	0.773	-0.057	0.076
EBIT_Margin	1.1060	0.443	2.494	0.013	0.236	1.976
shrown_excl_opts_pct	0.0180	0.026	0.692	0.489	-0.033	0.069
Leverage	0.0430	0.529	0.081	0.935	-0.994	1.080
log_option_awards	-0.0431	0.022	-1.962	0.050	-0.086	1.95e-05
log_revt_lag	0.0138	0.152	0.090	0.928	-0.285	0.312
log_at_lag	0.3680	0.143	2.571	0.010	0.087	0.649

Omnibus:	24.893	**Durbin-Watson:**	1.941
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	13.591
Skew:	-0.087	**Prob(JB):**	0.00112
Kurtosis:	2.445	**Cond. No.**	7.86e+05

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 7.86e+05. This might indicate that there are
strong multicollinearity or other numerical problems.

The interation between SRR and sector is insignificant

In [61]:

```
#Next year transaction Model with Interaction Term
# map name codes
final_data = final_data.copy()
final_data["gsector_name"] = final_data["gsector"].map(sector_names).astype("category")



formula = (
    "log_transaction_size_usd_nextyear ~ normalized_SRR_median * C(gsector_name) + TobinQ + tenure_years + gender "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage + log_option_awards + log_revt_lag + log_at_lag"
)

# Build model with missing="drop" so we can align clusters to the used sample
base = smf.ols(formula, data=final_data, missing="drop")
used_idx = base.data.row_labels
groups = final_data.loc[used_idx, "gvkey"]

m_pool = base.fit(
    cov_type="cluster",
    cov_kwds={"groups": groups, "use_correction": True}
)

pd.set_option("display.max_rows", None)
pd.set_option("display.max_columns", None)
pd.set_option("display.width", None)
pd.set_option("display.max_colwidth", None)

display(m_pool.summary())
```

OLS Regression Results

Dep. Variable:	log_transaction_size_usd_nextyear	**R-squared:**	0.125
Model:	OLS	**Adj. R-squared:**	0.096
Method:	Least Squares	**F-statistic:**	7.429
Date:	Fri, 30 Jan 2026	**Prob (F-statistic):**	1.21e-20
Time:	23:13:55	**Log-Likelihood:**	-2258.7
No. Observations:	965	**AIC:**	4581.
Df Residuals:	933	**BIC:**	4737.
Df Model:	31		
Covariance Type:	cluster		

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	0.4992	0.925	0.539	0.590	-1.314	2.313
C(gsector_name)[T.Materials]	-0.1965	0.431	-0.456	0.649	-1.042	0.649
C(gsector_name)[T.Industrials]	-0.2226	0.465	-0.479	0.632	-1.133	0.688
C(gsector_name)[T.Commercial Discretionary]	-0.0924	0.437	-0.211	0.833	-0.949	0.764
C(gsector_name)[T.Consumer Staples]	-0.5587	0.476	-1.173	0.241	-1.492	0.375
C(gsector_name)[T.Healthcare]	0.5495	0.541	1.017	0.309	-0.510	1.609
C(gsector_name)[T.Financials]	1.0100	0.646	1.563	0.118	-0.256	2.276
C(gsector_name)[T.IT]	0.3133	0.444	0.705	0.481	-0.557	1.184
C(gsector_name)[T.Communication Service]	0.9030	0.415	2.174	0.030	0.089	1.717
C(gsector_name)[T.Utilities]	-0.3120	0.809	-0.386	0.700	-1.897	1.273
C(gsector_name)[T.Real Estate]	1.4157	0.795	1.781	0.075	-0.142	2.973
normalized_SRR_median	-113.8639	36.435	-3.125	0.002	-185.275	-42.453
normalized_SRR_median:C(gsector_name)[T.Materials]	252.3609	301.812	0.836	0.403	-339.179	843.901
normalized_SRR_median:C(gsector_name)[T.Industrials]	597.9551	163.406	3.659	0.000	277.684	918.226
normalized_SRR_median:C(gsector_name)[T.Commercial Discretionary]	266.8777	41.864	6.375	0.000	184.825	348.930
normalized_SRR_median:C(gsector_name)[T.Consumer Staples]	683.7818	626.805	1.091	0.275	-544.734	1912.298
normalized_SRR_median:C(gsector_name)[T.Healthcare]	358.6223	148.552	2.414	0.016	67.466	649.778
normalized_SRR_median:C(gsector_name)[T.Financials]	371.3690	413.149	0.899	0.369	-438.389	1181.127
normalized_SRR_median:C(gsector_name)[T.IT]	799.5035	206.039	3.880	0.000	395.675	1203.332
normalized_SRR_median:C(gsector_name)[T.Communication Service]	1597.5113	768.723	2.078	0.038	90.841	3104.181
normalized_SRR_median:C(gsector_name)[T.Utilities]	-1260.9793	2425.867	-0.520	0.603	-6015.591	3493.632
normalized_SRR_median:C(gsector_name)[T.Real Estate]	-1592.8423	423.305	-3.763	0.000	-2422.504	-763.180
TobinQ	0.1311	0.056	2.356	0.018	0.022	0.240
tenure_years	0.0035	0.018	0.200	0.841	-0.031	0.038
gender	0.2779	0.291	0.954	0.340	-0.293	0.849
log_stock_awards	-0.0069	0.040	-0.174	0.862	-0.085	0.071
EBIT_Margin	0.6509	0.404	1.610	0.107	-0.141	1.443
shrown_excl_opts_pct	0.0134	0.029	0.464	0.643	-0.043	0.070
Leverage	0.4185	0.605	0.692	0.489	-0.767	1.604
log_option_awards	-0.0340	0.022	-1.530	0.126	-0.077	0.010
log_revt_lag	0.1585	0.197	0.805	0.421	-0.228	0.545
log_at_lag	0.2343	0.182	1.290	0.197	-0.122	0.590

Omnibus:	25.986	**Durbin-Watson:**	1.917
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	14.243
Skew:	-0.100	**Prob(JB):**	0.000808
Kurtosis:	2.440	**Cond. No.**	7.86e+05

Notes:
[1] Standard Errors are robust to cluster correlation (cluster)
[2] The condition number is large, 7.86e+05. This might indicate that there are
strong multicollinearity or other numerical problems.

The SRR coefficent is significantly more positive in `Industrials` , `Consumer Discretionary` , `IT` , `Communication Service` , `Real Estate` , `Healthcare`

Interprete: As SRR increases, the change in transaction size is higher for Industrial, Comsumer Discretionary, IT, Real Estate and Healthcare than Energy as base Sector

normalized SRR is statistically significant

3.4 Regression for M&A spending next 2 year

This part is to answer the question: **Does SRR this year is associated to deal size next 2 year?**

In [62]:

```
m1_next2year_nocl = simple_ols(
    final_data,
    dependent_var="log_transaction_size_usd_next2year",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"]
)

print("Simple regression on log_transaction_size_usd_next2year, nonrobust: \n")

display(m1_next2year_nocl)
```

```
Simple regression on log_transaction_size_usd_next2year, nonrobust:
```

	coef	std_err	p_value	t_stat	R_squared	adj_R_squared	n
model							
SRR_median	0.906986	0.421931	0.031835	2.149607	0.004775	0.003742	965
SRR_mean	0.927837	0.433233	0.032471	2.141660	0.004740	0.003707	965
normalized_SRR_mean	-54.839609	74.716548	0.463146	-0.733969	0.000559	-0.000479	965
normalized_SRR_median	-53.856119	74.688385	0.471037	-0.721078	0.000540	-0.000498	965

Raw SRR is statistically significant but normalized SRR is not.

In [63]:

```
m1_next2years = simple_ols_clustered(
    final_data,
    dependent_var="log_transaction_size_usd_next2year",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    cluster_col="gvkey"
)

print("Simple regression on log_transaction_size_usd_next2year, firm-clustered SE (gvkey):\n")
display(m1_next2years)
```

```
Simple regression on log_transaction_size_usd_next2year, firm-clustered SE (gvkey):
```

	coef	std_err	p_value	t_stat	R_squared	adj_R_squared	n_obs	n_clusters
model								
SRR_median	0.906986	0.482188	0.059975	1.880979	0.004775	0.003742	965	222
SRR_mean	0.927837	0.500589	0.063812	1.853493	0.004740	0.003707	965	222
normalized_SRR_mean	-54.839609	50.238379	0.275014	-1.091588	0.000559	-0.000479	965	222
normalized_SRR_median	-53.856119	50.312806	0.284428	-1.070426	0.000540	-0.000498	965	222

SRR is no statistically siginicant

In [64]:

```
#Regression with basic controls

m2_sameyear_nocl, models_basic = ols_with_controls(
    data=final_data,
    dependent_var="log_transaction_size_usd_next2year",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
)

print("Regression with Controls Results on log_transaction_size_usd_next2year with Controls:, no clustering\n")
display(m2_sameyear_nocl)
```

```
Regression with Controls Results on log_transaction_size_usd_next2year with Controls:, no clustering
```

	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	F_Stat	Prob_F_Stat
0	SRR_median	0.638849	0.424394	1.505321	0.132573	-0.194007	1.471704	0.075443	0.063789	6.473495	3.572870e-11
1	SRR_mean	0.607953	0.436309	1.393400	0.163824	-0.248286	1.464192	0.075128	0.063470	6.444323	4.116359e-11
2	normalized_SRR_mean	-15.125348	72.947316	-0.207346	0.835784	-158.281465	128.030768	0.073284	0.061603	6.273605	9.419396e-11
3	normalized_SRR_median	-13.940456	72.921235	-0.191171	0.848432	-157.045388	129.164477	0.073278	0.061596	6.273025	9.445882e-11

In [65]:

```
m2_next2year, models_basic_cl = ols_with_controls_clustered(
    data=final_data,
    dependent_var="log_transaction_size_usd_next2year",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    cluster_col="gvkey"
)

print("Regression with Controls Results on log_transaction_size_usd_next2year (clustered by gvkey):\n")
display(m2_next2year)
```

```
Regression with Controls Results on log_transaction_size_usd_next2year (clustered by gvkey):
```

	Independent_Variable	coef	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	N_Clusters
0	SRR_median	0.638849	0.405720	1.574605	0.115348	-0.156348	1.434045	0.075443	0.063789	222
1	SRR_mean	0.607953	0.425424	1.429051	0.152990	-0.225863	1.441769	0.075128	0.063470	222
2	normalized_SRR_mean	-15.125348	53.067998	-0.285018	0.775630	-119.136713	88.886016	0.073284	0.061603	222
3	normalized_SRR_median	-13.940456	53.066138	-0.262700	0.792782	-117.948174	90.067263	0.073278	0.061596	222

SRR is not associated with deal size in next 2 years

In [66]:

```
# ---- inputs ----

m3_next2year_nocl = ols_with_controls_and_fe(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="nonrobust"
)



results_m3_2year = fe_results_table_nocl(m3_next2year_nocl)
print("Regression with Controls and Fixed Effects Results on log_transaction_size_usd_nextyear:,  no cluster \n")
display(results_m3_2year)
```

```
Regression with Controls and Fixed Effects Results on log_transaction_size_usd_nextyear:,  no cluster
```

	FE_spec	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	F_Stat	Prob_F_Stat
0	FE1: sector	SRR_median	0.657448	0.427284	1.538669	0.124221	-0.181090	1.495987	0.112743	0.092022	5.440885	2.241001e-14
1	FE1: sector	SRR_mean	0.640093	0.440191	1.454126	0.146245	-0.223775	1.503961	0.112505	0.091778	5.427955	2.490479e-14
2	FE1: sector	normalized_SRR_mean	172.317030	71.876537	2.397403	0.016706	31.260368	313.373693	0.115908	0.095260	5.613612	5.461195e-15
3	FE1: sector	normalized_SRR_median	173.368918	71.840886	2.413235	0.016002	32.382219	314.355616	0.115979	0.095333	5.617504	5.290006e-15
4	FE2: sector + year	SRR_median	0.709219	0.443496	1.599157	0.110127	-0.161154	1.579593	0.123181	0.087200	3.423438	4.638564e-11
5	FE2: sector + year	SRR_mean	0.705351	0.458121	1.539662	0.123984	-0.193724	1.604427	0.123005	0.087016	3.417848	4.964575e-11
6	FE2: sector + year	normalized_SRR_mean	175.404662	72.946104	2.404579	0.016387	32.245807	318.563516	0.126216	0.090359	3.519953	1.429721e-11
7	FE2: sector + year	normalized_SRR_median	176.367161	72.903469	2.419187	0.015747	33.291979	319.442343	0.126282	0.090427	3.522062	1.393303e-11
8	FE3: firm	SRR_median	0.269903	0.569238	0.474147	0.635536	-0.847634	1.387439	0.409927	0.221847	2.179531	3.715197e-15
9	FE3: firm	SRR_mean	0.329885	0.586597	0.562370	0.574036	-0.821731	1.481500	0.410001	0.221944	2.180195	3.622877e-15
10	FE3: firm	normalized_SRR_mean	171.305315	84.815470	2.019741	0.043775	4.794354	337.816276	0.413022	0.225927	2.207558	1.282987e-15
11	FE3: firm	normalized_SRR_median	171.410135	84.805821	2.021207	0.043622	4.918115	337.902154	0.413026	0.225933	2.207601	1.280888e-15
12	FE4: sector + firm	SRR_median	0.269903	0.569238	0.474147	0.635536	-0.847634	1.387439	0.409927	0.221847	2.179531	3.715197e-15
13	FE4: sector + firm	SRR_mean	0.329885	0.586597	0.562370	0.574036	-0.821731	1.481500	0.410001	0.221944	2.180195	3.622877e-15
14	FE4: sector + firm	normalized_SRR_mean	171.305315	84.815470	2.019741	0.043775	4.794354	337.816276	0.413022	0.225927	2.207558	1.282987e-15
15	FE4: sector + firm	normalized_SRR_median	171.410135	84.805821	2.021207	0.043622	4.918115	337.902154	0.413026	0.225933	2.207601	1.280888e-15
16	FE5: year	SRR_median	0.992681	0.437069	2.271223	0.023360	0.134933	1.850429	0.102804	0.075965	3.830359	1.686592e-10
17	FE5: year	SRR_mean	1.036224	0.450456	2.300387	0.021645	0.152203	1.920245	0.102931	0.076095	3.835638	1.603300e-10
18	FE5: year	normalized_SRR_mean	165.682358	73.034411	2.268552	0.023523	22.352204	309.012512	0.102792	0.075953	3.829879	1.694378e-10
19	FE5: year	normalized_SRR_median	166.483325	72.999211	2.280618	0.022795	23.222251	309.744399	0.102845	0.076007	3.832052	1.659417e-10

In [67]:

```
m3_next2year_clustered = ols_with_controls_and_fe_clusterd(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["SRR_median", "SRR_mean", "normalized_SRR_mean", "normalized_SRR_median"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="cluster",
    cluster_col="gvkey"
)

m3_next2year_clustered_table = fe_results_table(m3_next2year_clustered, data=final_data, cluster_col="gvkey")

print("Regression with Controls and Fixed Effects (clustered by firm gvkey):\n")
display(m3_next2year_clustered_table)
```

```
Regression with Controls and Fixed Effects (clustered by firm gvkey):
```

	FE_spec	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	N_Clusters	N_Obs
0	FE1: sector	SRR_median	0.657448	0.437373	1.503175	0.132794	-0.199787	1.514684	0.112743	0.092022	222	965
1	FE1: sector	SRR_mean	0.640093	0.455255	1.406010	0.159721	-0.252190	1.532376	0.112505	0.091778	222	965
2	FE1: sector	normalized_SRR_mean	172.317030	61.952161	2.781453	0.005412	50.893026	293.741035	0.115908	0.095260	222	965
3	FE1: sector	normalized_SRR_median	173.368918	62.141144	2.789922	0.005272	51.574514	295.163321	0.115979	0.095333	222	965
4	FE2: sector + year	SRR_median	0.709219	0.432620	1.639360	0.101138	-0.138700	1.557138	0.123181	0.087200	222	965
5	FE2: sector + year	SRR_mean	0.705351	0.455450	1.548693	0.121456	-0.187313	1.598016	0.123005	0.087016	222	965
6	FE2: sector + year	normalized_SRR_mean	175.404662	62.926814	2.787439	0.005313	52.070373	298.738951	0.126216	0.090359	222	965
7	FE2: sector + year	normalized_SRR_median	176.367161	63.112424	2.794492	0.005198	52.669084	300.065238	0.126282	0.090427	222	965
8	FE3: firm	SRR_median	0.269903	0.637434	0.423421	0.671988	-0.979445	1.519250	0.409927	0.221847	222	965
9	FE3: firm	SRR_mean	0.329885	0.660269	0.499622	0.617342	-0.964219	1.623988	0.410001	0.221944	222	965
10	FE3: firm	normalized_SRR_mean	171.305315	96.941310	1.767103	0.077211	-18.696162	361.306792	0.413022	0.225927	222	965
11	FE3: firm	normalized_SRR_median	171.410135	96.965549	1.767743	0.077104	-18.638850	361.459119	0.413026	0.225933	222	965
12	FE4: sector + firm	SRR_median	0.269903	0.641839	0.420515	0.674109	-0.988079	1.527884	0.409927	0.221847	222	965
13	FE4: sector + firm	SRR_mean	0.329885	0.664832	0.496192	0.619759	-0.973162	1.632931	0.410001	0.221944	222	965
14	FE4: sector + firm	normalized_SRR_mean	171.305315	97.611265	1.754975	0.079264	-20.009249	362.619879	0.413022	0.225927	222	965
15	FE4: sector + firm	normalized_SRR_median	171.410135	97.635672	1.755610	0.079155	-19.952265	362.772534	0.413026	0.225933	222	965
16	FE5: year	SRR_median	0.992681	0.420627	2.360002	0.018275	0.168267	1.817095	0.102804	0.075965	222	965
17	FE5: year	SRR_mean	1.036224	0.435375	2.380070	0.017309	0.182904	1.889544	0.102931	0.076095	222	965
18	FE5: year	normalized_SRR_mean	165.682358	70.134503	2.362352	0.018159	28.221257	303.143458	0.102792	0.075953	222	965
19	FE5: year	normalized_SRR_median	166.483325	70.258864	2.369570	0.017809	28.778482	304.188169	0.102845	0.076007	222	965

The association between SRR and next-year acquisition size is not robust for the raw SRR measures.

Overall, SRR is most strongly related to deal size in next-year models

4. Robustness check

To check whether the SRR–M&A relationship still hold when we make reasonable alternative assumptions

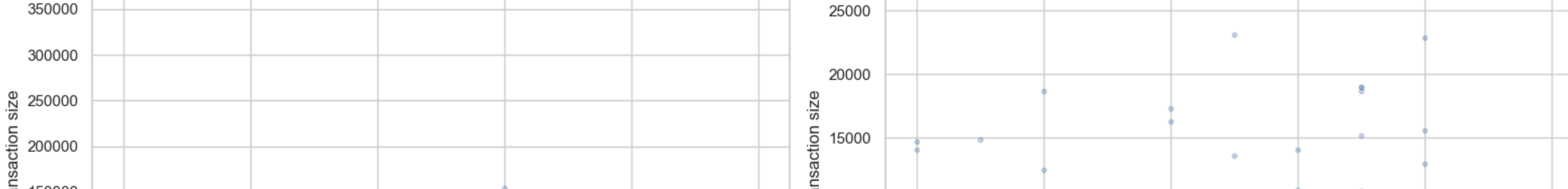
4.1 Winsorize SRR only among observations with transaction volume at 2%

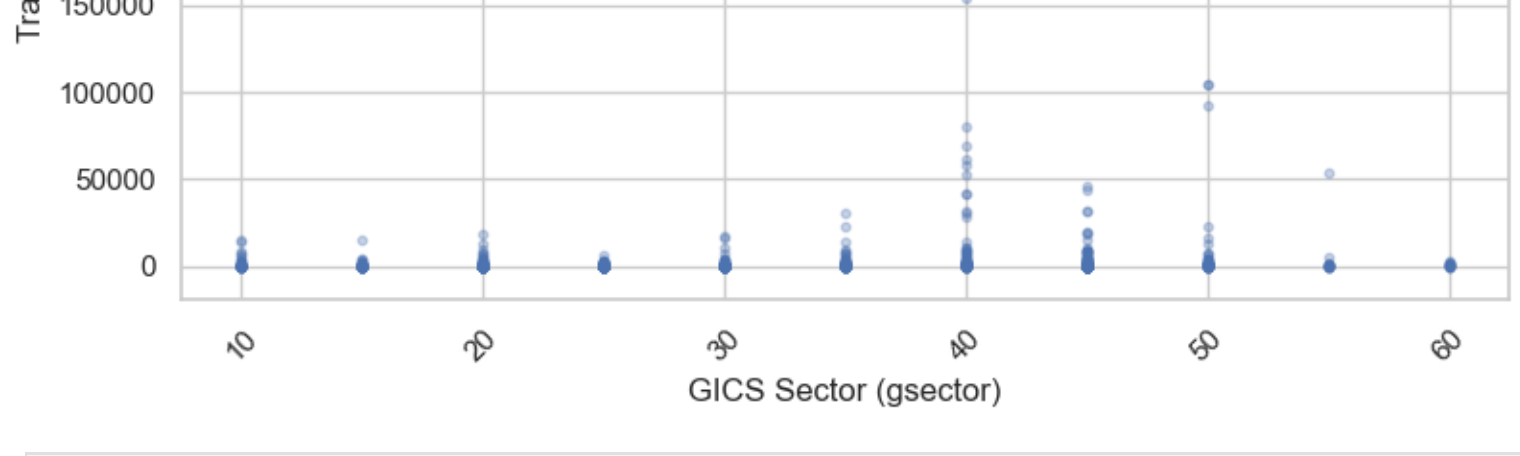
Purpose: to mitigate the influence of extreme transaction values and ensure that estimated relationships are not driven by a small number of unusually large or small deals, we trim the transaction size distribution by excluding observations below the 2nd percentile and above the 98th percentile. This improves the stability of OLS estimates and provides a robustness check that the SRR–deal size association holds for the bulk of the sample

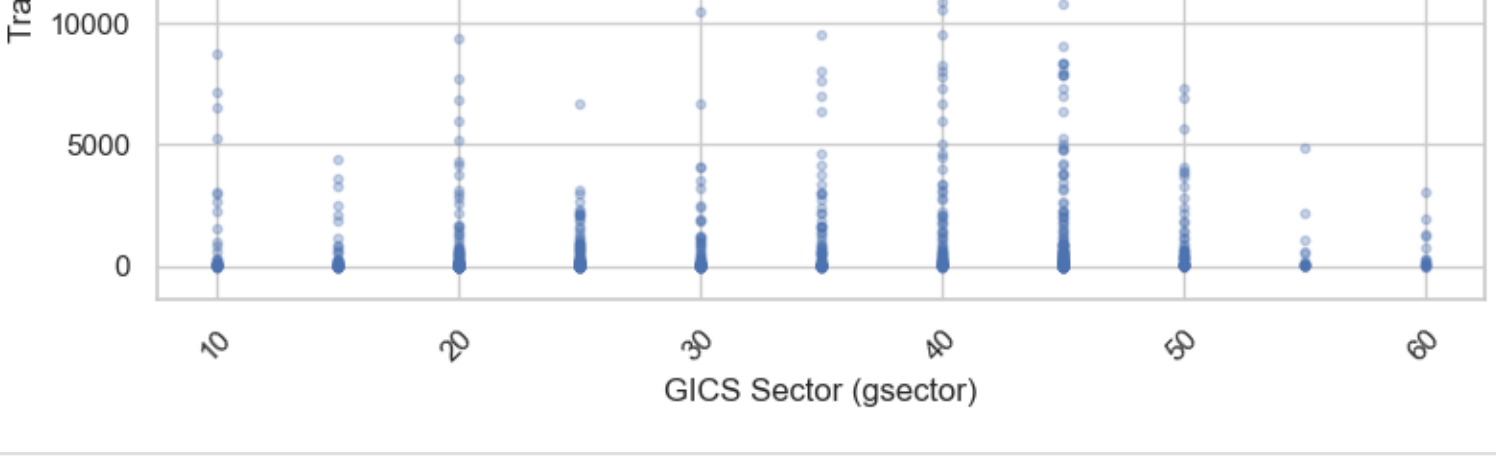
Result: normalized_SRR is positively and significantly associated with contemporaneous acquisition size. The estimate remains stable and statistically significant after controlling for firm and CEO characteristics and when including sector fixed effects

In [68]:

```
#remove outlier at the smallest 2% and largest 2%

q2 = final_data["transactionsize_usd"].quantile(0.02)
q98 = final_data["transactionsize_usd"].quantile(0.98)

final_data_filtered = final_data[(final_data["transactionsize_usd"] >= q2) &
                    (final_data["transactionsize_usd"] <= q98)]

# Create 2 subplots side by side
fig, axes = plt.subplots(1, 2, figsize=(16, 5))  # 1 row, 2 columns

# --- Before filtered ---
axes[0].scatter(final_data["gsector"], final_data["transactionsize_usd"],
                alpha=0.3, s=10)
axes[0].set_xlabel("GICS Sector (gsector)")
axes[0].set_ylabel("Transaction size")
axes[0].set_title("Before Filter")
axes[0].tick_params(axis='x', rotation=45)

# --- After filtered ---
axes[1].scatter(final_data_filtered["gsector"], final_data_filtered["transactionsize_usd"],
                alpha=0.3, s=10)
axes[1].set_xlabel("GICS Sector (gsector)")
axes[1].set_ylabel("Transaction size")
axes[1].set_title("After Filter")
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```







In [69]:
```
# --- filter outliers (2% / 98%) ---
q2  = final_data["log_transaction_size_usd_nextyear"].quantile(0.02)
q98 = final_data["log_transaction_size_usd_nextyear"].quantile(0.98)

final_data_filtered = final_data[
    (final_data["log_transaction_size_usd_nextyear"] >= q2) &
    (final_data["log_transaction_size_usd_nextyear"] <= q98)
].copy()

# --- helper: fit with firm-clustered SE ---
def fit_clustered(formula, data, cluster_col="gvkey"):
    base = smf.ols(formula, data=data, missing="drop")
    used_idx = base.data.row_labels
    groups = data.loc[used_idx, cluster_col]
    return base.fit(cov_type="cluster", cov_kwds={"groups": groups, "use_correction": True})

# 1) Simple model (clustered)
m1 = fit_clustered("log_transaction_size_usd_nextyear ~ normalized_SRR_median", final_data_filtered)

# 2) Controls (clustered)
m2 = fit_clustered(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_median + TobinQ + tenure_years + gender "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage + log_option_awards + log_revt_lag + log_at_lag",
    final_data_filtered
)


# 3) Sector FE (clustered)
m3 = fit_clustered(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_median + TobinQ + tenure_years + gender "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct "
    "+ Leverage + log_option_awards + log_revt_lag + log_at_lag + C(gsector)",
    final_data_filtered
)


# --- Compare coef + clustered SE + p-value ---
print("\n========================================")
print(f"1. Simple:       coef={m1.params['normalized_SRR_median']:.3f}  se={m1.bse['normalized_SRR_median']:.3f}  p={m1.pvalues['normalized_SRR_median']:.3g}")
print(f"2. + Controls:   coef={m2.params['normalized_SRR_median']:.3f}  se={m2.bse['normalized_SRR_median']:.3f}  p={m2.pvalues['normalized_SRR_median']:.3g}")
print(f"3. + Sector FE:  coef={m3.params['normalized_SRR_median']:.3f}  se={m3.bse['normalized_SRR_median']:.3f}  p={m3.pvalues['normalized_SRR_median']:.3g}")
print("----------------------------------------")
print("========================================")


# Optional: full summaries
# print(m1.summary()); print(m2.summary()); print(m3.summary())
```

```
========================================
1. Simple:       coef=128.932  se=58.944  p=0.0287
2. + Controls:   coef=169.829  se=67.840  p=0.0123
3. + Sector FE:  coef=174.912  se=62.522  p=0.00515
----------------------------------------
========================================
```

4.2 Lag corrected SRR by 1 year (Causality)

Result: No causality is seen

In [70]:
```
# sort panel
final_data = final_data.sort_values(["gvkey", "publish_year"]).copy()

# 1-year lag of SRR measures (within firm)

final_data["normalized_SRR_mean_lag1"] = final_data.groupby("gvkey")["normalized_SRR_mean"].shift(1)
final_data["normalized_SRR_median_lag1"] = final_data.groupby("gvkey")["normalized_SRR_median"].shift(1)

# Helper: OLS with FE + firm-clustered SEs
def fit_fe_clustered(formula, data, cluster_col="gvkey"):
    base = smf.ols(formula, data=data, missing="drop")
    used_idx = base.data.row_labels
    groups = data.loc[used_idx, cluster_col]
    return base.fit(cov_type="cluster", cov_kwds={"groups": groups, "use_correction": True})

# Model with controls + FE
# # Firm + year FE
m_lag_firm = fit_fe_clustered(
    "log_transaction_size_usd_nextyear ~ normalized_SRR_mean_lag1 + TobinQ + tenure_years + gender "
    "+ age + log_stock_awards + EBIT_Margin + shrown_excl_opts_pct + Leverage "
    "+ log_option_awards + log_revt_lag + log_at_lag "
    "+ C(gsector) + C(publish_year)",
    final_data
)

print(m_lag_firm.summary())

print("\n========================================")

var = "normalized_SRR_mean_lag1"

print("coef :", m_lag_firm.params[var])
print("se   :", m_lag_firm.bse[var])
print("pval :", m_lag_firm.pvalues[var])
```

```
                                  OLS Regression Results
=============================================================================================
Dep. Variable:     log_transaction_size_usd_nextyear   R-squared:                       0.141
Model:                                           OLS   Adj. R-squared:                  0.095
Method:                                Least Squares   F-statistic:                     5.779
Date:                               Fri, 30 Jan 2026   Prob (F-statistic):           6.72e-16
Time:                                       23:13:56   Log-Likelihood:                -1740.9
No. Observations:                                743   AIC:                             3560.
Df Residuals:                                    704   BIC:                             3740.
Df Model:                                         38
Covariance Type:                             cluster
=============================================================================================
                                coef    std err          z      P>|z|      [0.025      0.975]
---------------------------------------------------------------------------------------------
Intercept                    -0.8260      1.353     -0.611      0.541      -3.477       1.825
C(gsector)[T.15.0]           -0.2911      0.552     -0.528      0.598      -1.372       0.790
C(gsector)[T.20.0]           -0.1515      0.531     -0.285      0.776      -1.193       0.890
C(gsector)[T.25.0]           -0.1621      0.522     -0.311      0.756      -1.185       0.860
C(gsector)[T.30.0]           -0.7124      0.448     -1.589      0.112      -1.591       0.166
C(gsector)[T.35.0]            0.6022      0.516      1.168      0.243      -0.409       1.613
C(gsector)[T.40.0]            1.2721      0.655      1.944      0.052      -0.011       2.555
C(gsector)[T.45.0]            0.7294      0.491      1.484      0.138      -0.234       1.693
C(gsector)[T.50.0]            1.4682      0.439      3.343      0.001       0.608       2.329
C(gsector)[T.55.0]           -0.6924      0.886     -0.782      0.434      -2.428       1.043
C(gsector)[T.60.0]            0.8424      0.746      1.130      0.259      -0.619       2.304
C(publish_year)[T.2008.0]     1.9747      1.045      1.889      0.059      -0.074       4.024
C(publish_year)[T.2009.0]     0.4813      1.187      0.406      0.685      -1.845       2.808
C(publish_year)[T.2010.0]     1.4397      0.577      2.495      0.013       0.309       2.571
C(publish_year)[T.2011.0]     1.5313      0.752      2.036      0.042       0.057       3.005
C(publish_year)[T.2012.0]     1.6855      0.747      2.256      0.024       0.221       3.150
C(publish_year)[T.2013.0]     1.1074      0.750      1.476      0.140      -0.363       2.577
C(publish_year)[T.2014.0]     1.8556      0.750      2.475      0.013       0.386       3.325
C(publish_year)[T.2015.0]     1.4779      0.731      2.022      0.043       0.045       2.911
C(publish_year)[T.2016.0]     1.7085      0.738      2.315      0.021       0.262       3.155
C(publish_year)[T.2017.0]     1.7434      0.813      2.144      0.032       0.149       3.337
C(publish_year)[T.2018.0]     1.0730      0.680      1.578      0.114      -0.259       2.405
C(publish_year)[T.2019.0]     1.6743      0.844      1.984      0.047       0.020       3.328
C(publish_year)[T.2020.0]     1.9855      0.960      2.069      0.039       0.105       3.866
C(publish_year)[T.2021.0]     1.8484      0.876      2.111      0.035       0.132       3.564
C(publish_year)[T.2022.0]     1.7746      0.889      1.996      0.046       0.032       3.517
C(publish_year)[T.2023.0]     0.3346      1.041      0.321      0.748      -1.706       2.375
normalized_SRR_mean_lag1    -14.9984     94.315     -0.159      0.874    -199.853     169.856
TobinQ                        0.0902      0.079      1.140      0.254      -0.065       0.245
tenure_years                  0.0198      0.018      1.087      0.277      -0.016       0.055
gender                        0.2790      0.354      0.789      0.430      -0.414       0.972
age                           0.0077      0.019      0.409      0.683      -0.029       0.045
log_stock_awards             -0.0276      0.052     -0.526      0.599      -0.130       0.075
EBIT_Margin                   1.0086      0.429      2.352      0.019       0.168       1.849
shrown_excl_opts_pct         -0.0179      0.030     -0.590      0.555      -0.077       0.041
Leverage                      0.3575      0.784      0.456      0.648      -1.179       1.894
log_option_awards            -0.0389      0.027     -1.432      0.152      -0.092       0.014
log_revt_lag                  0.2694      0.229      1.176      0.240      -0.180       0.718
log_at_lag                    0.0664      0.210      0.315      0.753      -0.346       0.479
==============================================================================
Omnibus:                       25.191   Durbin-Watson:                   2.013
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               12.193
Skew:                          -0.056   Prob(JB):                      0.00225
Kurtosis:                       2.382   Cond. No.                     7.36e+04
==============================================================================

Notes:
[1] Standard Errors are robust to cluster correlation (cluster)
[2] The condition number is large, 7.36e+04. This might indicate that there are
strong multicollinearity or other numerical problems.

========================================
coef : -14.998420750649615
se   : 94.31526242444318
pval : 0.873649710733196
```

In [71]:
```
# sort panel
final_data = final_data.sort_values(["gvkey", "publish_year"]).copy()

# 1-year lag of SRR measures (within firm)

final_data["normalized_SRR_mean_lag1"] = final_data.groupby("gvkey")["normalized_SRR_mean"].shift(1)
final_data["normalized_SRR_median_lag1"] = final_data.groupby("gvkey")["normalized_SRR_median"].shift(1)

# Helper: OLS with FE + firm-clustered SEs
def fit_fe_clustered(formula, data, cluster_col="gvkey"):
    base = smf.ols(formula, data=data, missing="drop")
    used_idx = base.data.row_labels
    groups = data.loc[used_idx, cluster_col]
    return base.fit(cov_type="cluster", cov_kwds={"groups": groups, "use_correction": True})

# Model with controls + FE
# # Firm + year FE
m_lag_firm = fit_fe_clustered(
    "normalized_SRR_mean_lag1 ~ log_transaction_size_usd_nextyear+ TobinQ + tenure_years + gender "
    "+ age + log_stock_awards + EBIT_Margin + shrown_excl_opts_pct + Leverage "
    "+ log_option_awards + log_revt_lag + log_at_lag "
    "+ C(gsector) + C(publish_year)",
    final_data
)

print(m_lag_firm.summary())

print("\n========================================")

var = "log_transaction_size_usd_nextyear"

print("coef :", m_lag_firm.params[var])
print("se   :", m_lag_firm.bse[var])
print("pval :", m_lag_firm.pvalues[var])
```

```
                                     OLS Regression Results
==================================================================================================
Dep. Variable:     normalized_SRR_mean_lag1   R-squared:                       0.094
Model:                                  OLS   Adj. R-squared:                  0.045
Method:                       Least Squares   F-statistic:                     3.600
Date:                      Fri, 30 Jan 2026   Prob (F-statistic):           6.39e-09
Time:                              23:13:56   Log-Likelihood:                 4234.0
No. Observations:                       743   AIC:                            -8390.
Df Residuals:                           704   BIC:                            -8210.
Df Model:                                38
Covariance Type:                    cluster
==================================================================================================
                                        coef    std err          z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------------------------
Intercept                             0.0008      0.001      1.203      0.229      -0.000       0.002
C(gsector)[T.15.0]                   -0.0005      0.001     -0.809      0.419      -0.002       0.001
C(gsector)[T.20.0]                   -0.0005      0.001     -0.741      0.458      -0.002       0.001
C(gsector)[T.25.0]                   -0.0007      0.001     -1.112      0.266      -0.002       0.001
C(gsector)[T.30.0]                   -0.0007      0.001     -1.140      0.254      -0.002       0.000
C(gsector)[T.35.0]                   -0.0005      0.001     -0.861      0.389      -0.002       0.001
C(gsector)[T.40.0]                   -0.0004      0.001     -0.625      0.532      -0.002       0.001
C(gsector)[T.45.0]                   -0.0007      0.001     -1.095      0.274      -0.002       0.001
C(gsector)[T.50.0]                   -0.0008      0.001     -1.258      0.208      -0.002       0.000
C(gsector)[T.55.0]                   -0.0009      0.001     -1.360      0.174      -0.002       0.000
C(gsector)[T.60.0]                   -0.0004      0.001     -0.580      0.562      -0.002       0.001
C(publish_year)[T.2008.0]             0.0009      0.001      0.919      0.358      -0.001       0.003
C(publish_year)[T.2009.0]          4.538e-05      0.000      0.217      0.829      -0.000       0.000
C(publish_year)[T.2010.0]          3.175e-05      0.000      0.155      0.877      -0.000       0.000
C(publish_year)[T.2011.0]          7.693e-05      0.000      0.395      0.693      -0.000       0.000
C(publish_year)[T.2012.0]             0.0001      0.000      0.543      0.587      -0.000       0.001
C(publish_year)[T.2013.0]             0.0001      0.000      0.494      0.621      -0.000       0.000
C(publish_year)[T.2014.0]             0.0003      0.000      1.262      0.207      -0.000       0.001
C(publish_year)[T.2015.0]             0.0005      0.000      1.817      0.069   -3.67e-05       0.001
C(publish_year)[T.2016.0]             0.0005      0.000      1.589      0.112      -0.000       0.001
C(publish_year)[T.2017.0]             0.0004      0.000      1.488      0.137      -0.000       0.001
C(publish_year)[T.2018.0]             0.0004      0.000      1.239      0.215      -0.000       0.000
C(publish_year)[T.2019.0]          7.961e-06      0.000      0.041      0.967      -0.000       0.000
C(publish_year)[T.2020.0]         -2.804e-05      0.000     -0.170      0.865      -0.000       0.000
C(publish_year)[T.2021.0]         -2.364e-05      0.000     -0.129      0.897      -0.000       0.000
C(publish_year)[T.2022.0]         -1.606e-05      0.000     -0.082      0.935      -0.000       0.000
C(publish_year)[T.2023.0]         -4.328e-05      0.000     -0.216      0.829      -0.000       0.000
log_transaction_size_usd_nextyear -1.553e-06   9.79e-06     -0.159      0.874   -2.07e-05    1.76e-05
TobinQ                            -1.529e-05   1.33e-05     -1.152      0.249   -4.13e-05    1.07e-05
tenure_years                      -1.368e-06   5.47e-06     -0.250      0.803   -1.21e-05    9.36e-06
gender                             9.626e-05      0.000      0.914      0.361      -0.000       0.000
age                                3.647e-06   4.85e-06      0.752      0.452   -5.86e-06    1.31e-05
log_stock_awards                  -4.658e-06   1.03e-05     -0.450      0.653   -2.49e-05    1.56e-05
EBIT_Margin                           0.0001      0.000      1.086      0.277   -9.31e-05       0.000
shrown_excl_opts_pct               7.977e-07   7.58e-06      0.105      0.916   -1.41e-05    1.57e-05
Leverage                           5.019e-05      0.000      0.295      0.768      -0.000       0.000
log_option_awards                 -9.819e-07   9.25e-06     -0.106      0.915   -1.91e-05    1.71e-05
log_revt_lag                       1.449e-05   6.61e-05      0.219      0.826      -0.000       0.000
log_at_lag                         -3.12e-05   5.96e-05     -0.523      0.601      -0.000    8.56e-05
==============================================================================
Omnibus:                      996.505   Durbin-Watson:                   1.315
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           204359.096
Skew:                           6.959   Prob(JB):                         0.00
Kurtosis:                      83.046   Cond. No.                     4.90e+03
==============================================================================

Notes:
[1] Standard Errors are robust to cluster correlation (cluster)
[2] The condition number is large, 4.9e+03. This might indicate that there are
strong multicollinearity or other numerical problems.

========================================
coef : -1.5534014698289653e-06
se   : 9.793808660257866e-06
pval : 0.8739664946788703
```

In [72]:
```
m_lag_firm = fit_fe_clustered(
    "normalized_SRR_mean_lag1 ~ log_transaction_size_usd_nextyear+ C(gsector)",
    final_data
)

print(m_lag_firm.summary())

print("\n========================================")

var = "log_transaction_size_usd_nextyear"

print("coef :", m_lag_firm.params[var])
print("se   :", m_lag_firm.bse[var])
print("pval :", m_lag_firm.pvalues[var])
```

```
                                     OLS Regression Results
==================================================================================================
Dep. Variable:     normalized_SRR_mean_lag1   R-squared:                       0.035
Model:                                  OLS   Adj. R-squared:                  0.020
Method:                       Least Squares   F-statistic:                     4.235
Date:                      Fri, 30 Jan 2026   Prob (F-statistic):           1.52e-05
Time:                              23:13:56   Log-Likelihood:                 4210.6
No. Observations:                       743   AIC:                            -8397.
Df Residuals:                           731   BIC:                            -8342.
Df Model:                                11
Covariance Type:                    cluster
==================================================================================================
                                        coef    std err          z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------------------------
Intercept                             0.0009      0.001      1.506      0.132      -0.000       0.002
C(gsector)[T.15.0]                   -0.0005      0.001     -0.774      0.439      -0.002       0.001
C(gsector)[T.20.0]                   -0.0004      0.001     -0.634      0.526      -0.002       0.001
C(gsector)[T.25.0]                   -0.0006      0.001     -1.008      0.313      -0.002       0.001
C(gsector)[T.30.0]                   -0.0006      0.001     -1.077      0.282      -0.002       0.000
C(gsector)[T.35.0]                   -0.0005      0.001     -0.751      0.452      -0.002       0.001
C(gsector)[T.40.0]                   -0.0004      0.001     -0.598      0.550      -0.002       0.001
C(gsector)[T.45.0]                   -0.0006      0.001     -1.017      0.309      -0.002       0.001
C(gsector)[T.50.0]                   -0.0007      0.001     -1.238      0.216      -0.002       0.000
C(gsector)[T.55.0]                   -0.0007      0.001     -1.168      0.243      -0.002       0.000
C(gsector)[T.60.0]                   -0.0004      0.001     -0.606      0.545      -0.002       0.001
log_transaction_size_usd_nextyear -3.355e-06    1.1e-05     -0.304      0.761    -2.5e-05    1.83e-05
==============================================================================
Omnibus:                      996.968   Durbin-Watson:                   1.264
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           189090.297
Skew:                           7.006   Prob(JB):                         0.00
Kurtosis:                      79.887   Cond. No.                         95.1
==============================================================================

Notes:
[1] Standard Errors are robust to cluster correlation (cluster)

========================================
coef : -3.354600741055933e-06
se   : 1.1045354566101878e-05
pval : 0.7613477277754338
```

In [73]:
```
m_lag_firm = fit_fe_clustered(
    "normalized_SRR_mean_lag1 ~ log_transaction_size_usd_nextyear",
    final_data
)

print(m_lag_firm.summary())

print("\n========================================")

var = "log_transaction_size_usd_nextyear"

print("coef :", m_lag_firm.params[var])
print("se   :", m_lag_firm.bse[var])
print("pval :", m_lag_firm.pvalues[var])
```

```
                                     OLS Regression Results
==================================================================================================
Dep. Variable:     normalized_SRR_mean_lag1   R-squared:                       0.000
Model:                                  OLS   Adj. R-squared:                 -0.001
Method:                       Least Squares   F-statistic:                   0.09695
Date:                      Fri, 30 Jan 2026   Prob (F-statistic):              0.756
Time:                              23:13:56   Log-Likelihood:                 4197.5
No. Observations:                       743   AIC:                            -8391.
Df Residuals:                           741   BIC:                            -8382.
Df Model:                                 1
Covariance Type:                    cluster
==================================================================================================
                                        coef    std err          z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------------------------
Intercept                             0.0004   7.85e-05      4.982      0.000       0.000       0.001
log_transaction_size_usd_nextyear -3.279e-06   1.05e-05     -0.311      0.756   -2.39e-05    1.74e-05
==============================================================================
Omnibus:                     1034.526   Durbin-Watson:                   1.214
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           226422.198
Skew:                           7.465   Prob(JB):                         0.00
Kurtosis:                      87.207   Cond. No.                         12.0
==============================================================================

Notes:
[1] Standard Errors are robust to cluster correlation (cluster)

========================================
coef : -3.279222153391897e-06
se   : 1.0531405040206354e-05
pval : 0.7555151305312023
```

In [74]:
```
df = final_data.sort_values(["gvkey", "publish_year"]).copy()

# Lead of Y (t+1) within firm
df["log_transaction_size_usd_nextyear"] = (
    df.groupby("gvkey")["log_transaction_size_usd"].shift(-1)
)

# Lag of Y (t)
# If you want explicit lag:
df["log_transaction_size_usd_lag1"] = (
    df.groupby("gvkey")["log_transaction_size_usd"].shift(0)  # same-year
)

# Lag of SRR (t-1 relative to nextyear is t)
# If your normalized_SRR_mean_lag1 is already computed correctly, skip this.
df["normalized_SRR_mean_lag1"] = (
    df.groupby("gvkey")["normalized_SRR_mean"].shift(0)  # same-year relative to nextyear
)

# Drop rows where nextyear is missing (last year per firm) + any missing regressors
needed = [
    "log_transaction_size_usd_nextyear",
    "log_transaction_size_usd",
    "normalized_SRR_mean_lag1",
    "TobinQ", "tenure_years", "gender", "age",
    "log_stock_awards", "EBIT_Margin", "shrown_excl_opts_pct", "Leverage",
    "log_option_awards", "log_revt_lag", "log_at_lag",
    "gsector", "publish_year", "gvkey"
]
d = df.dropna(subset=needed).copy()


# Unrestricted: add SRR lag
f_unrestricted = (
    "log_transaction_size_usd_nextyear ~ "
    "log_transaction_size_usd "
    "+ normalized_SRR_mean_lag1 "
    "+ TobinQ + tenure_years + gender + age "
    "+ log_stock_awards + EBIT_Margin + shrown_excl_opts_pct + Leverage "
    "+ log_option_awards + log_revt_lag + log_at_lag "
    "+ C(gsector) + C(publish_year)"
)

m_u = smf.ols(f_unrestricted, data=d).fit(cov_type="cluster",
                                          cov_kwds={"groups": d["gvkey"]})


print(m_u.summary())
print("coef SRR:", m_u.params["normalized_SRR_mean_lag1"])
print("pval SRR:", m_u.pvalues["normalized_SRR_mean_lag1"])
print("coef lagged log_transaction_size_usd:", m_u.params["log_transaction_size_usd"])
print("pval lagged log_transaction_size_usd:", m_u.pvalues["log_transaction_size_usd"])
```

```
                                  OLS Regression Results
=============================================================================================
Dep. Variable:     log_transaction_size_usd_nextyear   R-squared:                       0.142
Model:                                           OLS   Adj. R-squared:                  0.096
Method:                                Least Squares   F-statistic:                     11.72
Date:                               Fri, 30 Jan 2026   Prob (F-statistic):           1.09e-30
Time:                                       23:13:56   Log-Likelihood:                -1733.5
No. Observations:                                743   AIC:                             3545.
Df Residuals:                                    704   BIC:                             3725.
Df Model:                                         38
Covariance Type:                             cluster
=============================================================================================
                                coef    std err          z      P>|z|      [0.025      0.975]
---------------------------------------------------------------------------------------------
Intercept                     0.4896      1.588      0.308      0.758      -2.623       3.602
C(gsector)[T.15.0]           -0.2745      0.564     -0.487      0.627      -1.380       0.831
C(gsector)[T.20.0]            0.1088      0.542      0.201      0.841      -0.953       1.170
C(gsector)[T.25.0]            0.2383      0.556      0.429      0.668      -0.851       1.328
C(gsector)[T.30.0]           -0.3787      0.547     -0.692      0.489      -1.451       0.693
C(gsector)[T.35.0]            0.8566      0.569      1.505      0.132      -0.259       1.972
C(gsector)[T.40.0]            1.2828      0.697      1.840      0.066      -0.084       2.649
C(gsector)[T.45.0]            0.4231      0.486      0.870      0.384      -0.530       1.376
C(gsector)[T.50.0]            1.1968      0.463      2.587      0.010       0.290       2.104
C(gsector)[T.55.0]           -0.5228      0.750     -0.697      0.486      -1.992       0.947
C(gsector)[T.60.0]            0.3477      0.871      0.399      0.690      -1.360       2.055
C(publish_year)[T.2008.0]    -0.5383      0.938     -0.574      0.566      -2.377       1.301
C(publish_year)[T.2009.0]    -0.2214      1.090     -0.203      0.839      -2.358       1.915
C(publish_year)[T.2010.0]     0.6465      1.132      0.571      0.568      -1.573       2.866
C(publish_year)[T.2011.0]    -0.0315      1.061     -0.030      0.976      -2.110       2.047
C(publish_year)[T.2012.0]    -0.0032      1.066     -0.003      0.998      -2.092       2.086
C(publish_year)[T.2013.0]     0.1358      1.020      0.133      0.894      -1.863       2.134
C(publish_year)[T.2014.0]     0.2262      0.990      0.229      0.819      -1.713       2.166
C(publish_year)[T.2015.0]    -0.1317      1.036     -0.127      0.899      -2.162       1.899
C(publish_year)[T.2016.0]     0.3318      0.964      0.344      0.731      -1.557       2.221
C(publish_year)[T.2017.0]    -0.0843      0.995     -0.085      0.933      -2.035       1.867
C(publish_year)[T.2018.0]    -0.3573      1.045     -0.342      0.732      -2.405       1.691
C(publish_year)[T.2019.0]     0.2090      1.038      0.201      0.840      -1.826       2.244
C(publish_year)[T.2020.0]    -0.4710      1.098     -0.429      0.668      -2.623       1.681
C(publish_year)[T.2021.0]     0.0375      0.977      0.038      0.969      -1.878       1.953
C(publish_year)[T.2022.0]    -0.6125      1.087     -0.563      0.573      -2.743       1.518
log_transaction_size_usd      0.0185      0.054      0.346      0.729      -0.086       0.124
normalized_SRR_mean_lag1    150.8487    136.412      1.106      0.269    -116.513     418.211
TobinQ                        0.1770      0.060      2.939      0.003       0.059       0.295
tenure_years                  0.0062      0.018      0.341      0.733      -0.030       0.042
gender                        0.4520      0.304      1.487      0.137      -0.144       1.048
age                          -0.0016      0.018     -0.091      0.928      -0.037       0.034
log_stock_awards             -0.0288      0.046     -0.623      0.533      -0.119       0.062
EBIT_Margin                   1.0735      0.473      2.268      0.023       0.146       2.001
shrown_excl_opts_pct         -0.0166      0.037     -0.450      0.653      -0.089       0.056
Leverage                      0.1616      0.687      0.235      0.814      -1.185       1.508
log_option_awards            -0.0433      0.028     -1.536      0.125      -0.099       0.012
log_revt_lag                  0.0968      0.211      0.460      0.646      -0.316       0.510
log_at_lag                    0.2929      0.195      1.499      0.134      -0.090       0.676
==============================================================================
Omnibus:                       19.899   Durbin-Watson:                   2.043
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               10.271
Skew:                          -0.027   Prob(JB):                      0.00588
Kurtosis:                       2.426   Cond. No.                     7.30e+04
==============================================================================

Notes:
[1] Standard Errors are robust to cluster correlation (cluster)
[2] The condition number is large, 7.3e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
coef SRR: 150.84871921056254
pval SRR: 0.26879797556962715
coef lagged log_transaction_size_usd: 0.01853971953166924
pval lagged log_transaction_size_usd: 0.7292362135276723
```

There is no evidence for SRR Granger-causes deal size next year (p=0.296).

4.3 Winsorize SRR only among observations with transaction volume at 1%

In [75]:
```
def winsorize_series(s, p_low=0.01, p_high=0.99):
    lo, hi = s.quantile([p_low, p_high])
    return s.clip(lower=lo, upper=hi)
mask = final_data["transactionsize_usd_nextyear"].notna()


final_data.loc[mask, "normalized_SRR_mean_w"] = winsorize_series(final_data.loc[mask, "normalized_SRR_mean"], 0.01, 0.99)
final_data.loc[mask, "normalized_SRR_median_w"] = winsorize_series(final_data.loc[mask, "normalized_SRR_median"], 0.01, 0.99)

#Regression with basic controls
control_vars = ["TobinQ", "tenure_years", "gender", "age", "log_stock_awards", "log_stock_awards",
                "EBIT_Margin","shrown_excl_opts_pct","Leverage","log_option_awards","log_revt_lag","log_at_lag"]
model_with_controls, models_basic = ols_with_controls_clustered(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["normalized_SRR_mean_w", "normalized_SRR_median_w"],
    control_vars=control_vars,
)
print("Regression with Winsorized SRR Controls Results on log_transaction_size_usd_nextyear with Controls:\n")
display(model_with_controls)
```

```
Regression with Winsorized SRR Controls Results on log_transaction_size_usd_nextyear with Controls:
```

	Independent_Variable	coef	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	N_Clusters
0	normalized_SRR_mean_w	353.064675	114.935280	3.071856	0.002127	127.795665	578.333685	0.09469	0.083278	222
1	normalized_SRR_median_w	355.463552	114.805733	3.096218	0.001960	130.448449	580.478655	0.09480	0.083390	222

Across all 2 SRR measures, the coefficient stays positive and statistically significant after winsorization and controls. The SRR–deal-size link is not just caused by a few outlier interviews.

In [76]:
```
# ---- inputs ----

models_fe = ols_with_controls_and_fe_clusterd(
    data=final_data,
    dependent_var="log_transaction_size_usd_nextyear",
    independent_vars=["normalized_SRR_mean_w", "normalized_SRR_median_w"],
    control_vars=control_vars,
    fe_specs=fe_specs,
    cov_type="nonrobust"
)

results_df = fe_results_table(models_fe, data=final_data, cluster_col="gvkey")
print("Regression with Winsorized SRR Controls and Fixed Effects Results on log_transaction_size_usd_nextyear:\n")
display(results_df)
```

```
Regression with Winsorized SRR Controls and Fixed Effects Results on log_transaction_size_usd_nextyear:
```

	FE_spec	Independent_Variable	Coefficient	Std_Error	T_Stat	P_Value	[0.025	0.975]	R_Squared	Adj_R_Squared	N_Clusters	N_Obs
0	FE1: sector	normalized_SRR_mean_w	357.328150	131.144202	2.724697	0.006555	99.959555	614.696745	0.117469	0.096858	222	965
1	FE1: sector	normalized_SRR_median_w	360.367342	130.953025	2.751883	0.006039	103.373929	617.360754	0.117607	0.096999	222	965
2	FE2: sector + year	normalized_SRR_mean_w	382.594141	135.222584	2.829366	0.004765	117.215881	647.972400	0.128296	0.092524	222	965
3	FE2: sector + year	normalized_SRR_median_w	385.343439	134.996836	2.854463	0.004407	120.408217	650.278661	0.128429	0.092662	222	965
4	FE3: firm	normalized_SRR_mean_w	347.958907	146.313680	2.378171	0.017655	60.713766	635.204048	0.414278	0.227584	222	965
5	FE3: firm	normalized_SRR_median_w	348.241984	146.282764	2.380608	0.017539	61.057540	635.426429	0.414287	0.227596	222	965
6	FE4: sector + firm	normalized_SRR_mean_w	347.958907	146.313680	2.378171	0.017655	60.713766	635.204048	0.414278	0.227584	222	965
7	FE4: sector + firm	normalized_SRR_median_w	348.241984	146.282764	2.380608	0.017539	61.057540	635.426429	0.414287	0.227596	222	965
8	FE5: year	normalized_SRR_mean_w	364.240113	134.679747	2.704491	0.006965	99.930882	628.549345	0.104854	0.078076	222	965
9	FE5: year	normalized_SRR_median_w	366.481840	134.487971	2.725016	0.006550	102.548970	630.414710	0.104960	0.078185	222	965

The estimated association between normalized SRR and next-year acquisition size remains positive and statistically significant across multiple fixed-effect specifications.

4.4 Check multicollinearity

Purpose:To assess whether correlation among regressors inflates estimation uncertainty, we compute variance inflation factors (VIFs) for the main explanatory and control variables.

Intuition: High VIF values indicate strong multicollinearity, which can increase standard errors and produce unstable coefficient estimates across specifications.

Result: Variance inflation factors (VIFs) for the main explanatory variable and all controls are low (all < 2.1), indicating that multicollinearity is not a material concern

In [77]:
```
# Check multicollinearity by VIF

def compute_vif(df, features):
    X = df[features].assign(constant=1)  # add constant for intercept
    vif_data = pd.DataFrame()
    vif_data["feature"] = features
    vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(len(features))]
    return vif_data
features = [
    "normalized_SRR_mean", "TobinQ", "tenure_years", "gender", "CF", "age", "Aqc_Ratio",
    "shrown_excl_opts_pct", "revt_lag", "Leverage",
    "log_option_awards", "log_stock_awards", "opt_unex_exer_est_val"
]
vif_results = compute_vif(final_data, features)
print(vif_results)
```

```
                  feature       VIF
0     normalized_SRR_mean  1.008127
1                  TobinQ  1.472676
2            tenure_years  1.788360
3                  gender  1.055481
4                      CF  1.291133
5                     age  1.524452
6               Aqc_Ratio  1.083126
7    shrown_excl_opts_pct  2.076961
8                revt_lag  1.098426
9                Leverage  1.045056
10      log_option_awards  1.142242
11       log_stock_awards  1.361907
12  opt_unex_exer_est_val  1.434540
```

Insight

According to our model transaction next year, we find a positive relationship between the level of CEO narcisism (SRR normalized mean and median) and the transaction size (significant at level 5%). The significance hold under 5% in single regression, muliple regression and also when putting fix effect to control for time invariant variables or when remove 0.1% outlier in the data

The model same year regression and next 2 year regression show the relationship between SRR and transaction size is not significant

SRR is statistical significant and positive associated with the acquisition activity next year

In the regression, we also find that some control factor are significantly with firm acquisition Three variables emerge as highly significant predictors:

1. Organizational Scale: Lagged total assets (log_at_lag) confirms that a firm's existing balance sheet capacity is the strongest anchor for future deal size or large firm are about to conduct more merge and aquisition
2. Operating Efficiency: The EBIT_Margin shows a robust positive association. This suggests that highly profitable firms leverage their superior cash-flow generation to execute larger-scale acquisitions.
3. Growth Opportunities: Tobin's Q remains a significant positive driver, indicating that firms with high market-to-book valuations—often interpreted as having a "mandate for growth"—consistently pursue larger targets.

Interaction effect of SRR and sector show that The coefficient is significantly more positive in `Industrials`, `Consumer Discretionary`, `IT`, `Communication Service`, `Real Estate`, `Healthcare`. As SRR increases, the change in transaction size is higher for Industrial, Consumer Discretionary, IT, Real Estate and Healthcare than Energy as base Sector

The model demonstrates that traditional governance and demographic variables have negligible predictive power regarding transaction size

1. CEO Characteristics: Neither gender nor tenure_years impact the scale of transactions.
2. Compensation Structures: The magnitude of stock awards and option awards does not appear to incentivize larger deal tickets.
3. Capital Structure: Current leverage does not act as a statistically significant constraint on the following year's acquisition size.

Reference

Liu, Yue (Lucy) and Taffler, Richard J., Damned Out of Their Own Mouths: CEO Narcissism in M&A Decision-Making and Its Impact on Firm Performance (February 15, 2009). Available at SSRN: https://ssrn.com/abstract=1344119 or http://dx.doi.org/10.2139/ssrn.1344119